ДИСЦИПЛИНА **Анализ и концептуальное моделирование систем**

(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **практической и прикладной информатики**

(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекционные материалы**

(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Пяткин Василий Владимирович**

(фамилия, имя, отчество)

СЕМЕСТР **8, 2023-2024**

(указать семестр обучения, учебный год)

АНАЛИЗ И КОНЦЕПТУАЛЬНОЕ МОДЕЛИРОВАНИЕ СИСТЕМ

Доц. Каф. ППИ, к.э.н. Пяткин В.В.

Online-edu.mirea.ru

ЛЕКЦИЯ 1.

Основные понятия системного анализа и теории систем.

Цель и задачи курса:

Цель курса – изучение методов исследования систем, методов обследования организаций, изучение современных методов и средств моделирования и проектирования информационных систем на базе унифицированного языка моделирования UML.

Задачи курса:

- введение в теорию моделирования;
- обзор и сравнение основных подходов к разработке программного обеспечения;
- обзор языка UML, его средств и возможностей;
- изучение языка UML применительно к моделированию и проектированию информационных систем.

ПЛАН ЛЕКЦИОННЫХ ЗАНЯТИЙ:

Лекция 1. Основные понятия системного анализа и теории систем.

Лекция 2. Введение в теорию моделирования. Классификация моделей.

Лекция 3. Объектно-ориентированное моделирование.

Лекция 4. Основы унифицированного языка моделирования.

Лекция 5-6. Модели анализа, проектирования, реализации. (4 ч.)

Лекция 7. Исследование систем управления.

Лекция 8. Оценивание систем.

ПЛАН ПРАКТИЧЕСКИХ РАБОТ:

Практическая работа №1. Описание функционала системы.

Практическая работа №2. Описание функций системы через диаграмму вариантов использования.

Практическая работа №3. Построение UML – модели системы. Диаграмма классов анализа.

Практическая работа №4. Построение UML – модели системы. Диаграмма последовательности.

Практическая работа №5. Построение UML – модели системы. Диаграмма классов.

Практическая работа №6. Построение UML – модели системы. Диаграмма деятельности.

Практическая работа №7. Построение UML – модели системы. Диаграммы компонентов, развертывания.

ЛЕКЦИЯ 1.

Основные понятия системного анализа и теории систем.

СОДЕРЖАНИЕ ЛЕКЦИИ:

- Основные положения системного анализа. Цель системного анализа.
- Принципы системного анализа.
- Понятие информационной системы. Классификация систем.
- Методы исследования систем.

1. Основные положения системного анализа. Понятие системного анализа.

Дисциплина анализ и концептуальное моделирование систем (АКМС) часть более общей дисциплины – системного анализа

Системный анализ — это научно-методологическая дисциплина, которая изучает принципы, методы и средства исследования *сложных объектов* посредством представления их в качестве *систем* и *анализа* этих систем.

Таким образом, в системном анализе любой объект рассматривается с учётом его системного характера, то есть не как единое целое, а как комплекс взаимосвязанных составных элементов, их свойств и процессов.

1. Основные положения системного анализа. Понятие системного анализа.

Дисциплина анализ и концептуальное моделирование систем (АКМС) часть более общей дисциплины – системного анализа

Наиболее широкое распространение **системный анализ** получил при исследовании искусственных систем, в теории и практике ***управления*** — при выработке, принятии и обосновании решений, связанных с проектированием, созданием и управлением сложными, многоуровневыми и многокомпонентными искусственными системами.

*Для понимания сущности системного анализа рассмотрим понятие **системы** и понятие **анализа***

1. Основные положения системного анализа. Понятие системного анализа.

Понятие анализа как научного метода

Анализ (образовано от греческого слова: αναλυσις — разделение)

Анализ — это процедура мысленного или материального разделения целостного объекта (предмета, явления, процесса) на составляющие части (признаки, свойства, отношения) с целью их изучения.)

Анализ — органичная составная часть всякого научного исследования, являющаяся обычно его первой стадией, когда исследователь переходит от нерасчлененного описания изучаемого объекта к выявлению его строения, состава, а также его свойств и признаков.

Проблемы анализа как научного метода

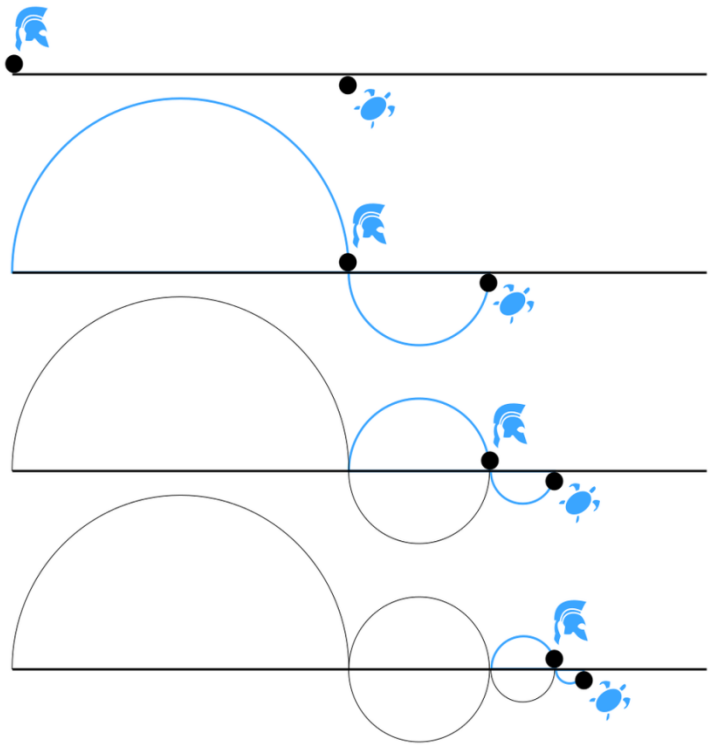


Проблемы анализа как научного метода

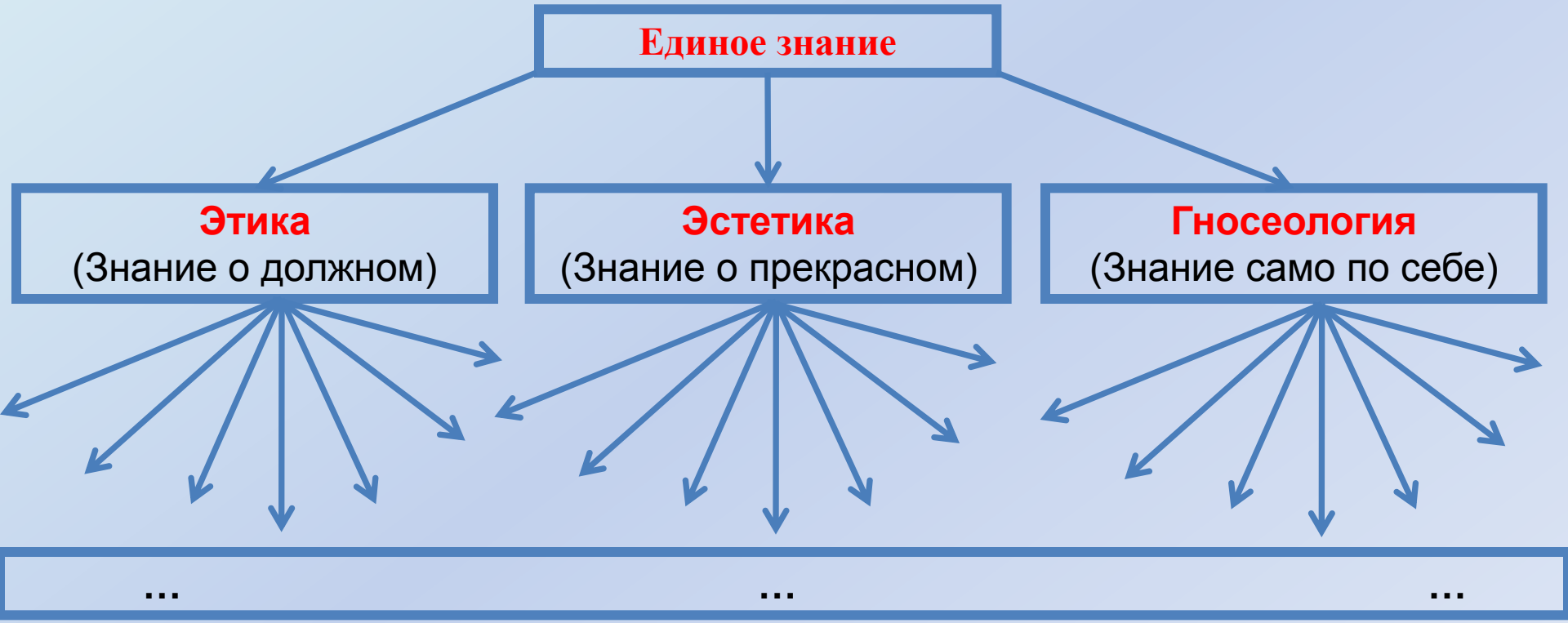
Ахилесс и Черепаха – одна из апорий древнегреческого философа Зенона:

Медлительнейшее - когда оно бежит - никогда не будет догнано быстрейшим
(«Физика» Аристотель)





Проблемы анализа как научного метода



Необходимость интеграции (лат. integratio востановление, восполнение) то есть объединить знания по отношению к какому-либо множеству элементов, составляющих систему или способных образовать целостность.

1. Основные положения системного анализа. Понятие системного анализа.

Понятие синтеза как как научного метода противоположного и дополняющего метод анализа

Процедурой, обратной анализу, является *синтез*, который представляет собой соединение ранее выделенных частей (признаков, свойств, отношений) объекта в единое целое

Синтез – не простое суммирование, а смысловое соединение.

Если просто соединить явления, между ними не возникнет системы связей, образуется лишь хаотическое накопление отдельных фактов.

Анализ и синтез представляют собой наиболее элементарные приёмы познания.

1. Основные положения системного анализа. Понятие системного анализа.

Место анализа и синтеза как методов познания



Основные общенаучные методы естествознания

Место анализа и синтеза как методов познания



Анализ - разложение объекта на составляющие его части.
Синтез - объединение познанных в результате анализа элементов в единое целое.

Обобщение - процесс мысленного перехода от единичного к о общему.
Абстрагирование - мысленное внесение определенных изменений в изучаемый объект.

Индукция - процесс выведения общего положения из ряда единичных фактов.
Дедукция - процесс аналитического рассуждения от общего к частному.

Аналогия – вероятное заключение о сходстве двух предметов в каком-либо признаке.
Моделирование - воспроизведение свойств объекта на специально устроенном его аналоге - модели.

Исторический метод - воспроизведение истории объекта с учетом всех деталей и случайностей.
Логический метод - логическое воспроизведение истории изучаемого объекта.

Классификация - распределение объектов по классам в зависимости от их общих признаков.

1. Основные положения системного анализа. Понятие системного анализа.

Анализ и синтез

Анализ и **синтез** играют фундаментальную роль в *познавательном процессе* и осуществляются на всех его ступенях.

В мыслительных операциях они выступают как логические приёмы *мышления* совершающиеся при помощи *абстрактных понятий* и тесно связанные с другими мыслительными операциями.

Объективной предпосылкой познавательных операций анализа и синтеза является **структурность** материальных объектов, способность их элементов к перегруппировке, объединению и разъединению.

1. Основные положения системного анализа. Понятие системного анализа.

Понятие концепции

Концепция – это комплекс взглядов на что-либо, связанных между собой и образующих единую систему; определённый способ понимания, трактовки каких-либо явлений; основная точка зрения, руководящая идея для их освещения; система взглядов на явления — в мире, природе, обществе

Концепция – это система взглядов на что-нибудь; то или иное понимание чего-нибудь, основная мысль; теоретическое построение; генеральный замысел, определяющий стратегию действий при осуществлении реформ, проектов, планов, реформ, программ;

Понятие системы

Понятие системы

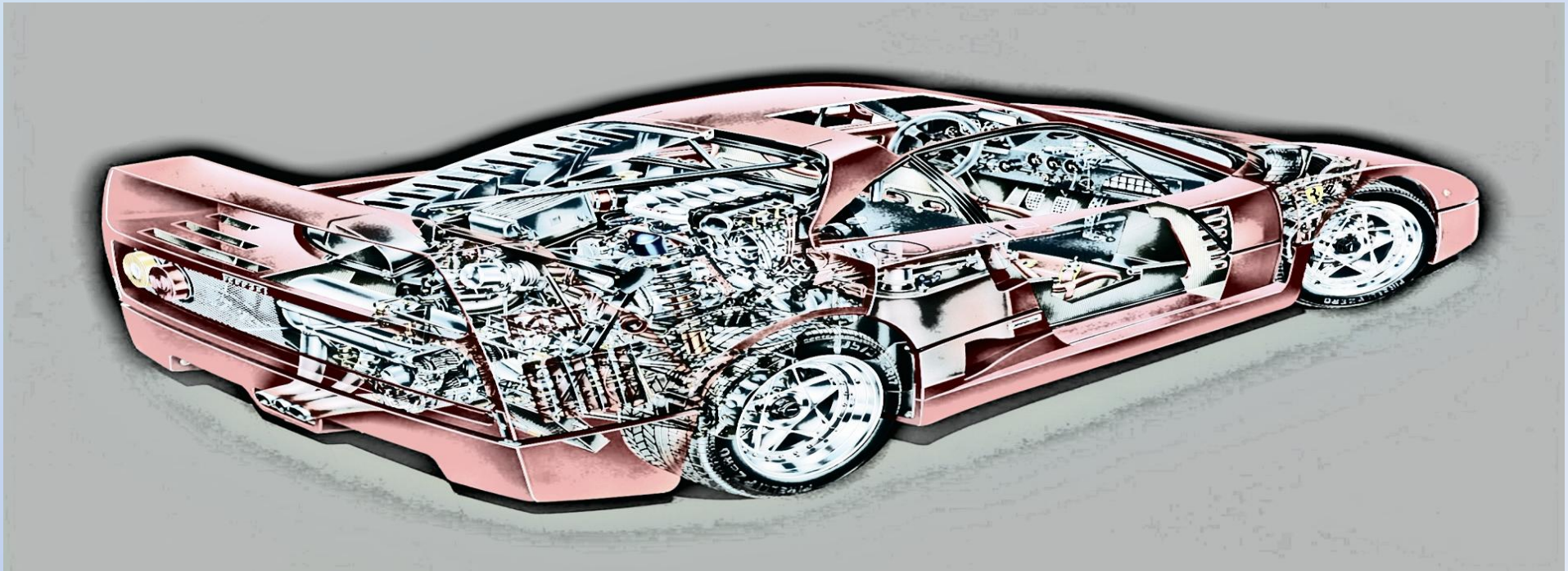
Система – это совокупность взаимодействующих и взаимосвязанных элементов, образующих единое целое.

Под системой понимают любой объект, который одновременно рассматривается и как единое целое, и как объединенная в интересах достижения поставленных целей совокупность разнородных элементов

Система — совокупность элементов, находящихся в определённых отношениях друг с другом и со средой (Л. фон Берталанфи).

Признаки (свойства) системы:

- целостность и членимость
- связанность, связь
- наличие целей, организация
- эмерджентность системные свойства



Системность

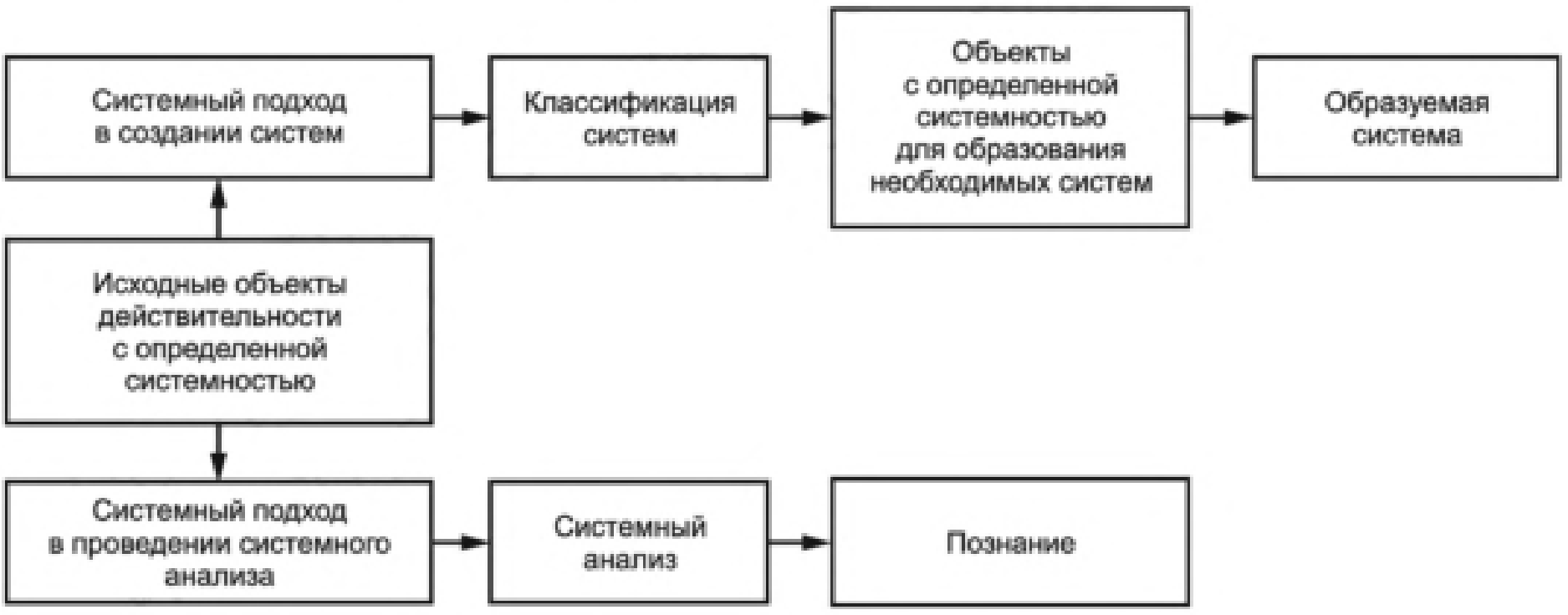
Системность — это упорядоченность и взаимосвязь элементов целого в объектах действительности.

Системность — это свойство определенных объектов действительности, позволяющее:

- с его применением при системном подходе образовывать необходимые системы, например такие, к которым в технической деятельности могут быть отнесены системы «человек-информация», «человек-машина»;

- элементы системы рассматривать в их взаимосвязи и в развитии;

- проводить системный анализ.



ГОСТ Р 43.0.30—2022 Информационное обеспечение техники и операторской деятельности. Системность.

Системность

Системность познавательной деятельности человека является одним из объективных факторов возникновения и развития системных понятий.

Естественный, спонтанный рост системности деятельности человека сопровождается, дополняется осознанием этой системности, а затем ускоряется ее сознательным повышением. Знания и соблюдение принципов системности в деятельности возрастают. Алгоритмизация любого вида деятельности — важный способ повышения ее системности.

Одной из особенностей познания человека является наличие **аналитического и синтетического образов мышления.**

Анализ состоит в разделении целого на части, в представлении сложного в виде совокупности более простых компонентов.

Для того чтобы познать целое, сложное, необходим и обратный процесс — синтез.

Расчлененность мышления (на анализ и синтез) и взаимосвязанность этих частей являются признаками **системности познания**.

ГОСТ Р 43.0.30—2022 Информационное обеспечение техники и операторской деятельности. Системность.

Системность

Системность как всеобщее свойство действительности представлена на рисунке



ГОСТ Р 43.0.30—2022 Информационное обеспечение техники и операторской деятельности. Системность.

Становление системного анализа
Развитие системных представлений

Предтечей современного системного анализа был А. А. Богданов, предложивший в начале XX века концепцию всеобщей организационной науки — *тектологии*, послужившей предтечей общей теории систем Л. фон Берталанфи. Основная идея теории Богданова заключается в том, что все существующие объекты и процессы имеют определённый уровень организованности, который тем выше, чем сильнее свойства целого отличаются от простой суммы свойств комплектующих элементов.

Именно анализ свойств целого и его частей был впоследствии заложен в качестве основной характеристики понятия сложной системы.

Наряду с этим, Богданов изучает не только статическое состояние структур, а занимается исследованием динамического поведения объектов, уделяет внимание вопросам развития организации, подчёркивает значение обратных связей, указывает на необходимость учёта собственных целей организации, отмечает роль открытых систем. При этом он уделяет особое внимание роли моделирования и математического анализа как потенциальных методов решения задач теории организации.

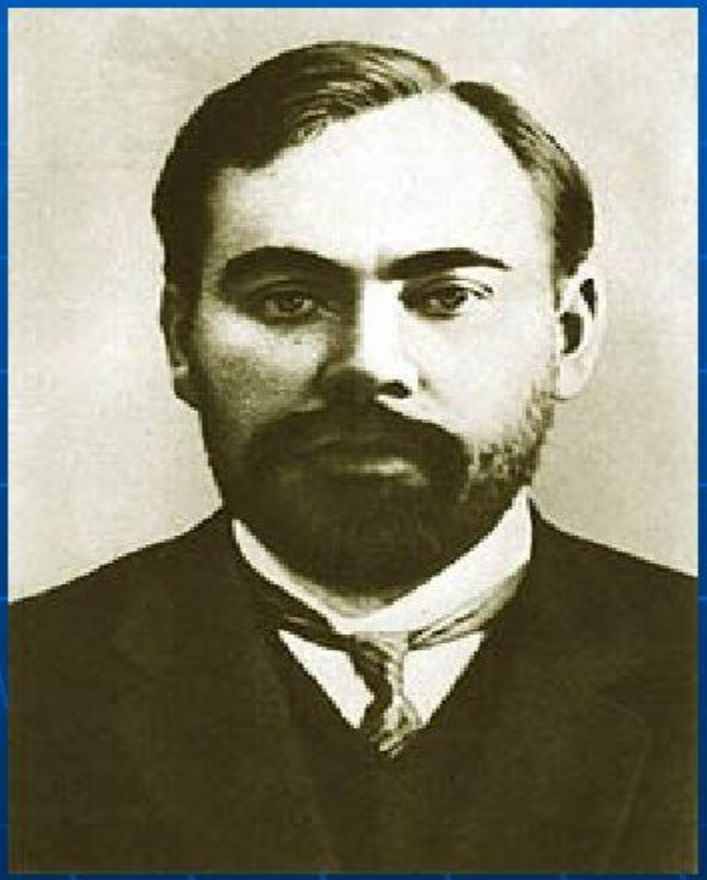
13 А.А. Богданов и Тектология



Алекса́ндр Алекса́ндрович Богда́нов (настоящая фамилия — **Малино́вский**; другие псевдонимы — *Ве́рнер*, *Макси́мов*, *Рядово́й*) (10 (22) августа 1873, Соколка, Гродненская губерния — 7 апреля 1928, Москва) — врач, экономист, философ, политический деятель, учёный-естествоиспытатель. Член РСДРП в 1896—1909, большевик, с 1905 член ЦК. Глава группы «Вперёд», организатор партийных школ в Болонье и на Капри. С 1918 — идеолог Пролеткульта. Выдвинул идею создания науки об общих принципах организации — тектологии, предвосхитил некоторые положения кибернетики. С 1926 — организатор и директор первого в мире Института переливания крови; погиб, производя на себе опыт.

15 Основоположники ТС и СА

ОСНОВАТЕЛИ СИСТЕМНОГО АНАЛИЗА



А.А.БОГДАНОВ



Л. фон БЕРТАЛАНФИ

16 Основание теории систем 2

Людвиг фон Берталанфи (1901-1972)



- Биофизик, психолог, физиолог,
- Создал эвристическую модель организма как открытой системы, стремящейся к устойчивому состоянию.
- Привнес из термодинамики необратимых процессов: Закрытая система определяется кинетически обратимым равновесием, открытая – динамически необратимым состоянием устойчивости
- Соединив органические и термодинамические направления, предложил общую теорию систем как метатеорию, позволяющую строить модели во всех научных дисциплинах, в т.ч. и в политологии.
- В отличие от кибернетики – изучение нестационарных структур и динамики самоорганизации сложных систем

17 Развитие теории систем Берталанфи

Школа «Системного подхода» (Л.фон Берталанфи)



Системность (организация – «открытая» система, взаимодействующая со средой)

Целевой подход (управление организацией «от целей»)

Упор на принятие решений (выбор оптимального управленческого решения)

Использование моделей при принятии решений

Использование компьютеров

Понятие системного анализа.

Системный анализ – совокупность методологических средств, используемых для подготовки и обоснования решений по сложным проблемам политического, военного, социального, экономического, научного, технического характера.

Задачи методов системного анализа:

- *поиск* альтернативных вариантов решения проблемы

- *выявление и анализ* масштабов неопределённости по каждому из вариантов

- *анализ возможных вариантов* по тем или иным критериям эффективности

Цель системного анализа.

Особенности организации системы



Цель системного анализа

Цель системного анализа



Целью применения системного анализа является повышение степени обоснованности принимаемого решения, расширение множества вариантов, среди которых производится обоснованный выбор.

Становление системного анализ

Предшественником современного системного анализа был А. А. Богданов, предложивший в начале XX века концепцию всеобщей организационной науки — *тектологии*, послужившей предтечей общей теории систем Л. фон Берталанфи.

Основная идея теории Богданова заключается в том, что все существующие объекты и процессы имеют определённый уровень организованности, который тем выше, чем сильнее свойства целого отличаются от простой суммы свойств комплектующих элементов.

Именно анализ свойств целого и его частей был впоследствии заложен в качестве основной характеристики понятия сложной системы.

Наряду с этим, Богданов изучает не только статическое состояние структур, а занимается исследованием динамического поведения объектов, уделяет внимание вопросам развития организации, подчёркивает значение обратных связей, указывает на необходимость учёта собственных целей организации, отмечает роль открытых систем. При этом он уделяет особое внимание роли моделирования и математического анализа как потенциальных методов решения задач теории организации.

ЛЕКЦИЯ 2.
Введение в теорию моделирования. Классификация моделей.

СОДЕРЖАНИЕ ЛЕКЦИИ:

- Понятие моделирования и модели.
- Принципы моделирования и классификация моделей.
- Метамоделирование.
- Методы моделирования систем.
- Этапы моделирования.

Понятие модели и моделирования

- Модель — это удобное, упрощённое представление объекта, процесса, явления, ситуации.
- Модель это — система, исследование которой служит средством для получения информации о другой системе; представление некоторого реального процесса, устройства или концепции.
- Модель это — абстрактное описание на некотором формальном языке некоторых аспектов системы, важных с точки зрения цели моделирования.
- Моделирование – это абстрактное описание на некотором формальном языке некоторых аспектов системы, важных с точки зрения цели моделирования.

Классификация видов моделей систем



1. Понятие моделирования и модели.

Модель – это абстрактное описание на некотором формальном языке некоторых аспектов системы, важных с точки зрения цели моделирования.

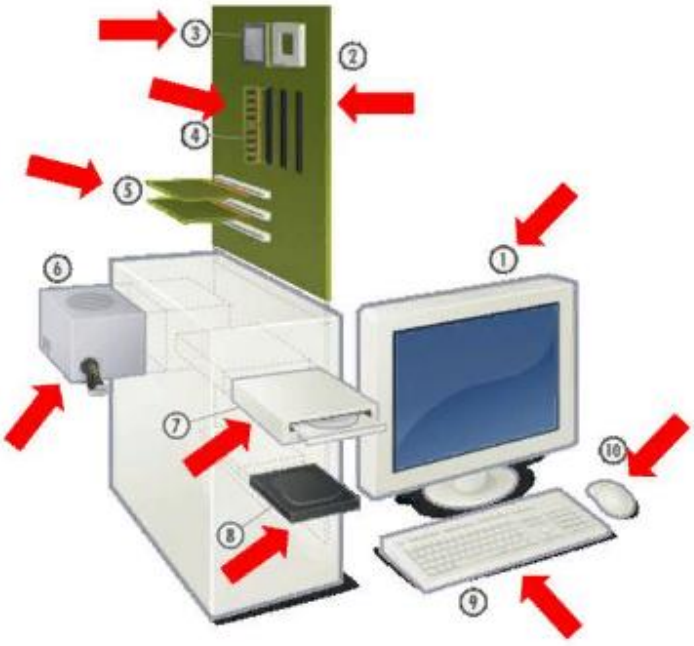
Классификация моделей по точке зрения на систему



Моделирование - это замещение одного объекта другим с целью получения информации о важнейших свойствах объекта-оригинала.

Статические модели.

В *статических* моделях система представляется неизменной во времени. Такие модели удобны, когда нужно описать структуру системы, то есть из каких объектов она состоит, как эти объекты связаны с друг с другом и каковы свойства этих объектов.



1. Монитор
2. Материнская плата
3. Процессор
4. ОЗУ
5. Карты расширения
6. Блок питания
7. Дисковод CD, DVD
8. Винчестер
9. Клавиатура
10. Мышь

Динамические модели.

Динамические модели содержат информацию о поведении системы и ее составных частей. Для описания поведения обычно используются записанные в виде формул, схем или компьютерных программ соотношения, позволяющие вычислить параметры системы и ее объектов, как функции времени.



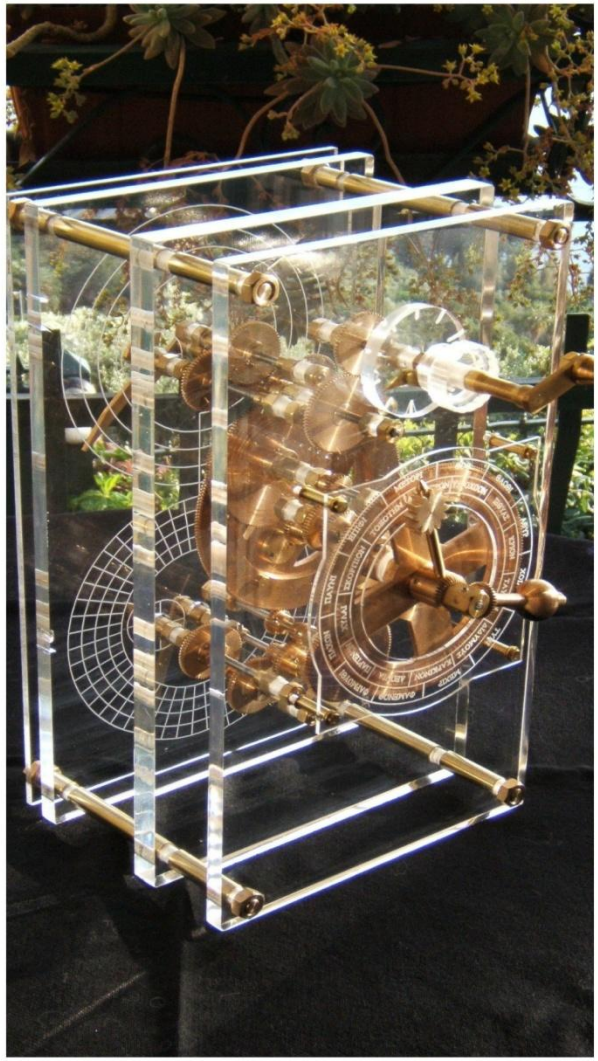


Детерминированное моделирование

Детерминированные Модели

Модели, в которых установлено взаимно-однозначное соответствие между переменными описывающими объект или явление

Антикитерский механизм







Стохастическое моделирование

Стохастические Модели

Модели имеющие переменные, изменения которых включают случайную (вероятностную) компоненту.

Рыночная динамика

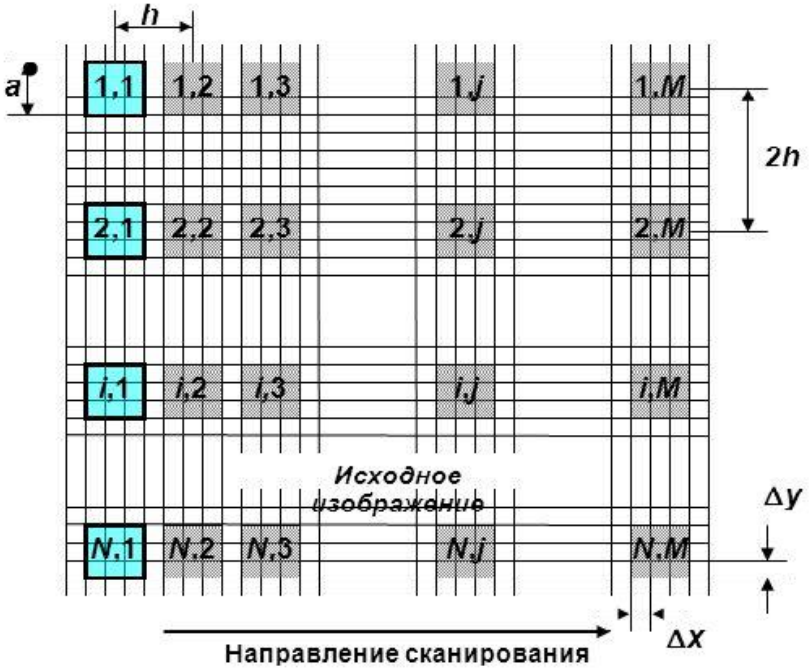


Дискретное моделирование

Дискретные Модели

Модели, описывающие поведение объекта только в отдельные промежутки времени это

Дискретная модель процесса сканирования



Моделирование сканирования изображения фотоприемной линейки.

$$x(t) = x_0 + j(\Delta t) \times h \times \Delta x$$

$$y(t) = y_0 + q(T) \times \Delta y$$

$T = M \times \Delta t$ - время кадра

Δt – время опроса N фотодатчиков

$j = 1, 2, \ldots, M$

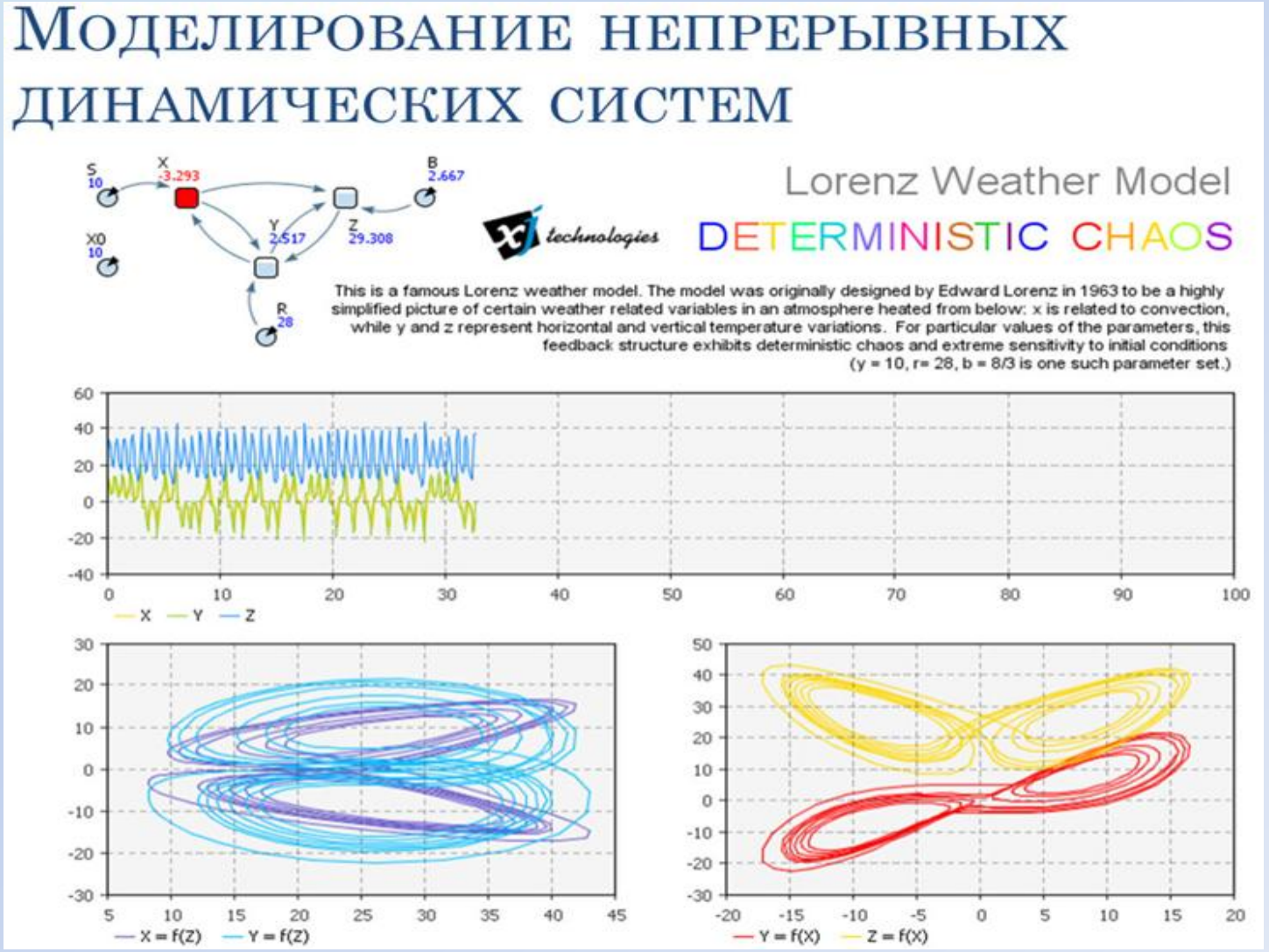
$q = 1, 2, \ldots, 2h$

За время кадра T изображение смещается на Δy

Непрерывное моделирование

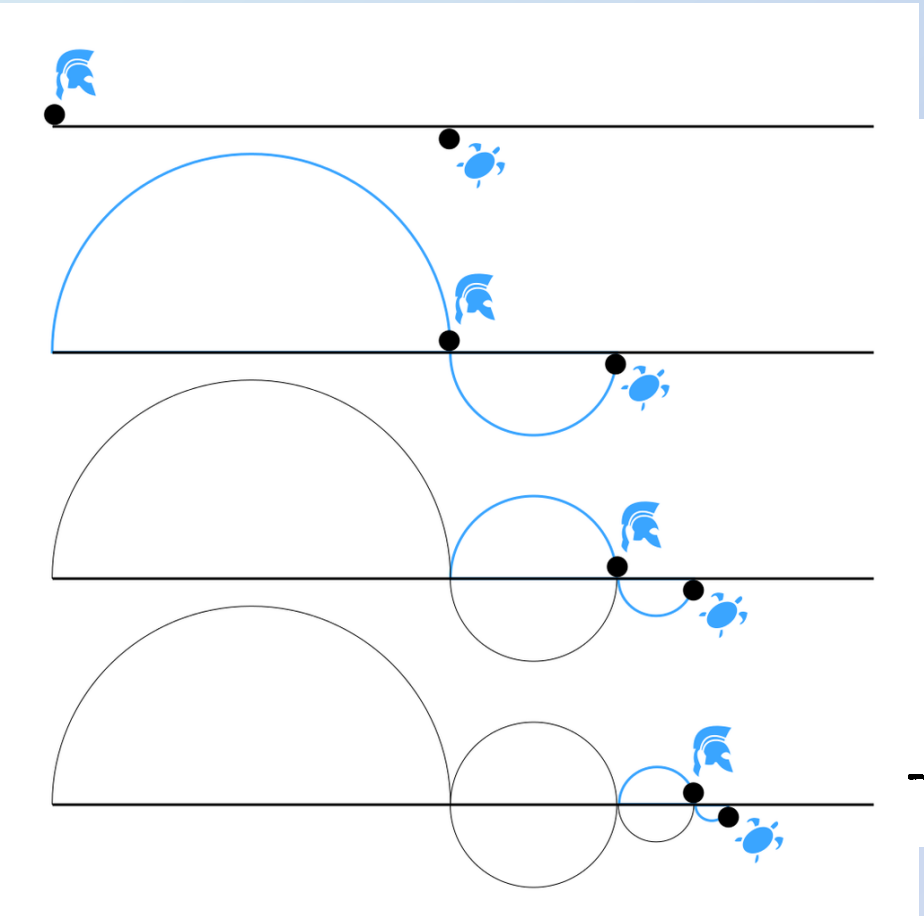
Непрерывные модели

Непрерывные модели описывают поведение объекта для всех промежутков времени. Состояние таких моделей изменяется непрерывно во времени.



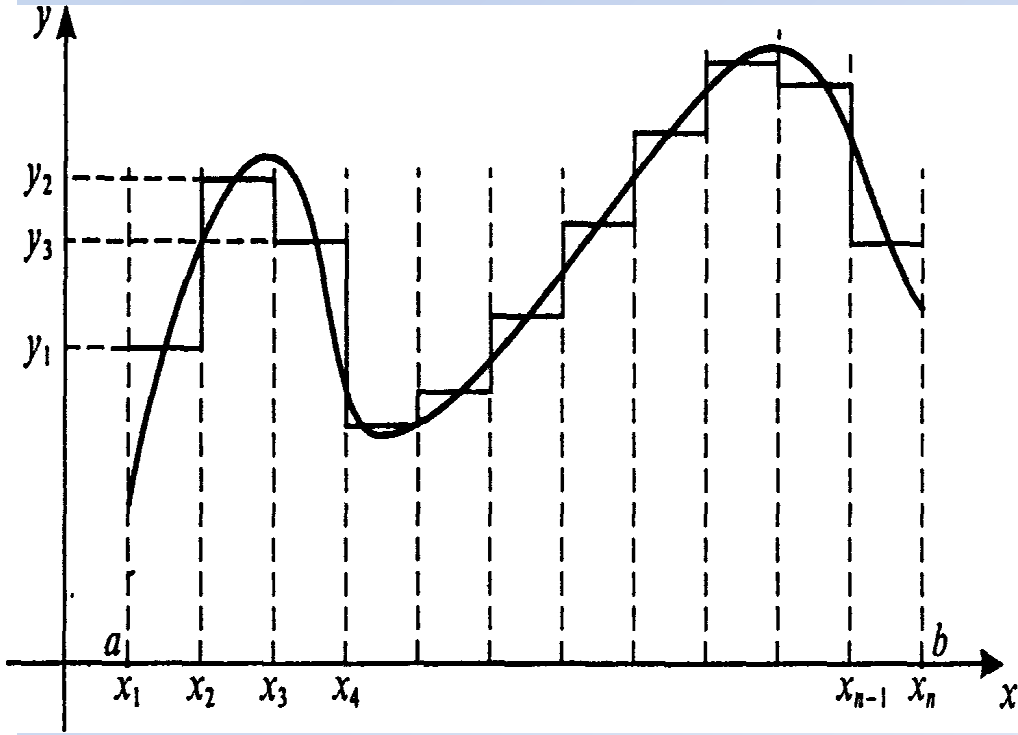
Дискретное и непрерывное моделирование
Проблемы

Дискретная модель движения:



Если движение непрерывно – дает ложное представление о процессе

Дискретизация сигнала

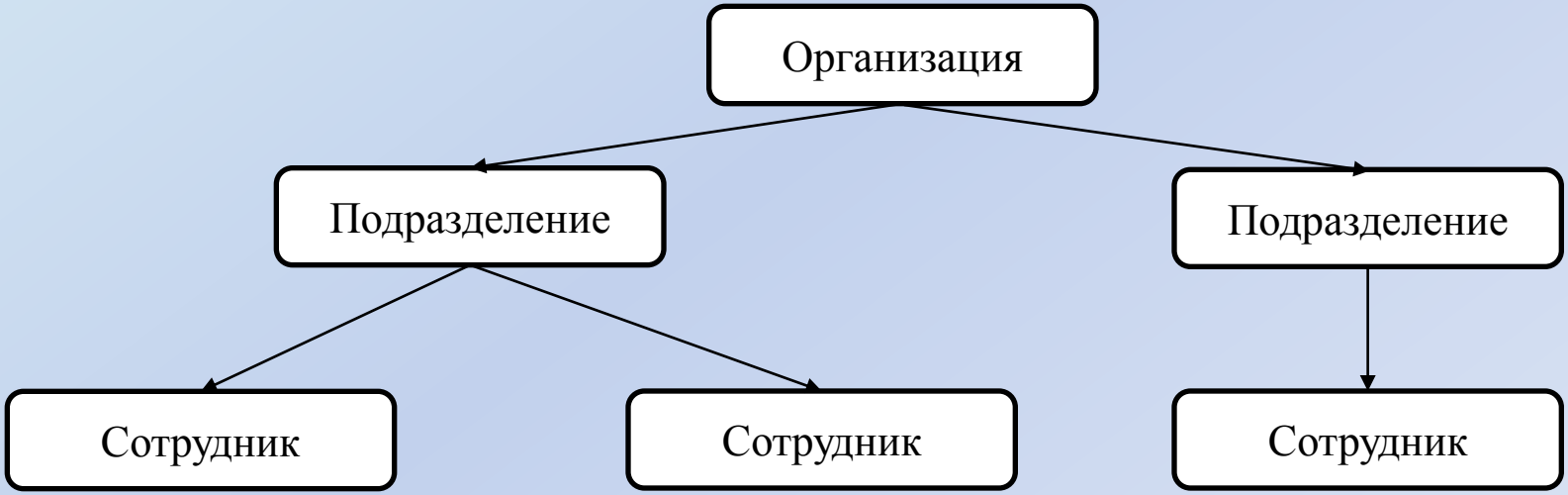


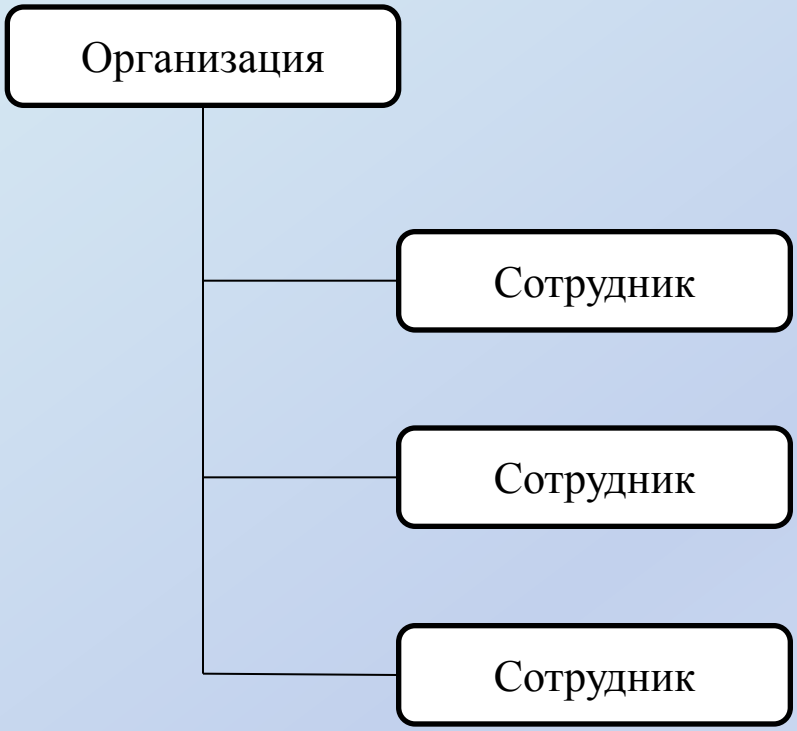
Дискретизация сигнала неизбежно приводит к потере информации, снижению точности

Метамоделирование.

Метамодель – это модель языка моделирования.

Пример простых моделей:



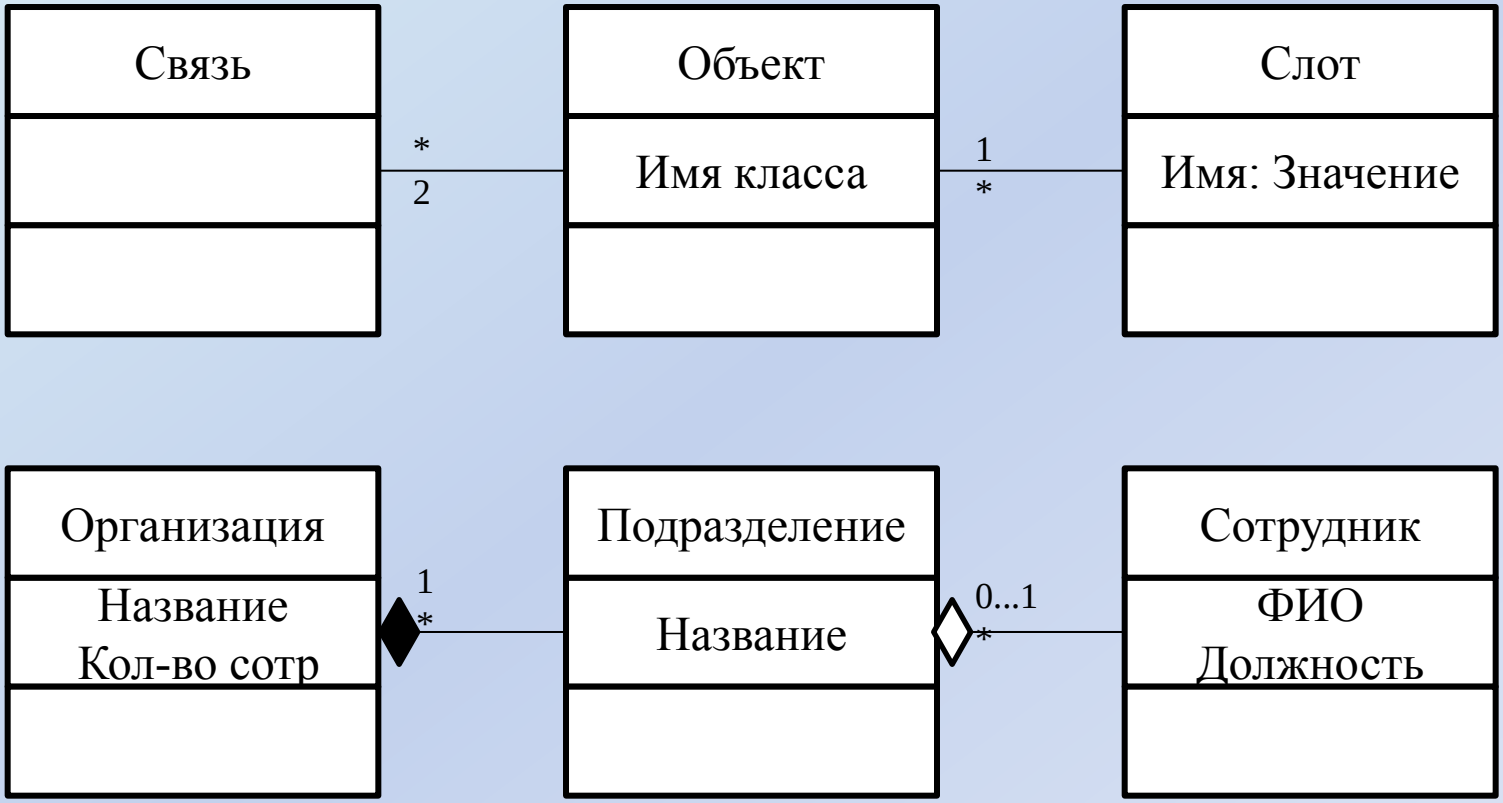


Организация

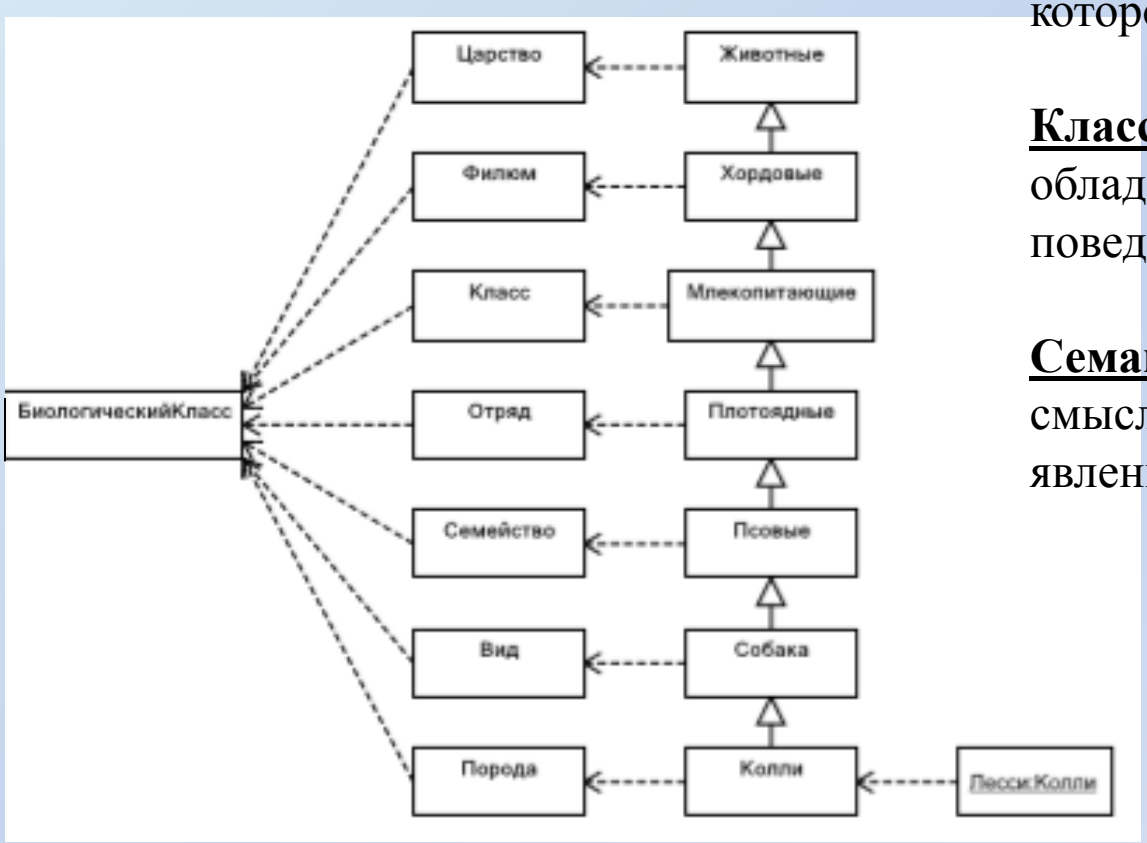
Кол-во сотрудников = 3

Элементы: объект, связь и слот (количество сотрудников)

Пример метамодели:



Пример многоуровневого онтологического моделирования



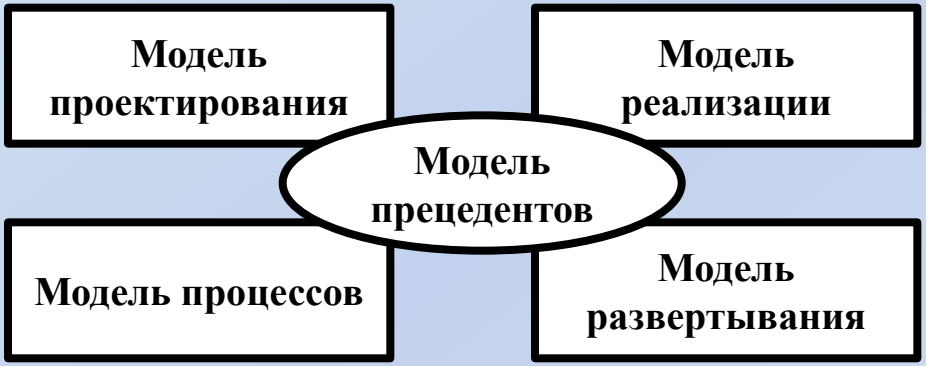
Метакласс – это класс, экземплярами которого являются другие классы.

Класс – это совокупность объектов, обладающих схожей структурой, поведением и семантикой.

Семантика – это суть, значение, смысл некоторого предмета или явления.

2. Принципы моделирования и классификация моделей.

Классификация Буча



Словарь, функциональность

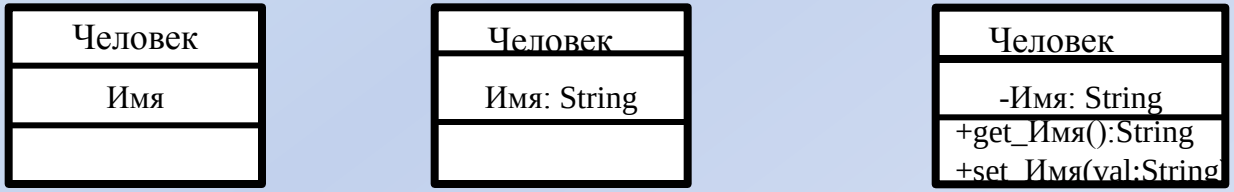
Производительность, масштабируемость, пропускная способность

Сборка системы, управление конфигурацией

Топология системы, поставка, установка

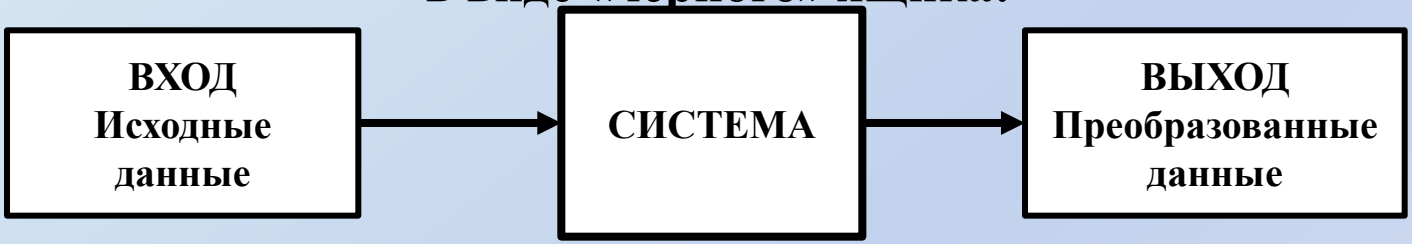
Классификация по степени абстракции:

- *концептуальные модели* – высокоуровневый взгляд на задачу в терминах предметной области;
- *модели спецификации* – определяют внешний вид и внешнее поведение системы;
- *модели реализации* – отражают внутреннее устройство системы, конкретный способ реализации внешнего облика и наблюдаемого поведения.



концептуальная модель модель спецификации модель реализации

Графическое изображение модели системы в виде «черного» ящика:



Моделирование *– это метод научного познания, заключающийся в изучении некоторого объекта посредством его модели.*

Реализация процесса моделирования основана на:

- формализованном представлении системы;
- решении задачи принятия решений.

{Ф} *

где {} – множество альтернатив;
Ф – принцип выбора;
** – выбранная альтернатива, одна или несколько в каком-то смысле равноценных.*

Варианты принятия решений

Вариант 1: Оптимальный выбор	Вариант 2: Выбор	Вариант 3: Общая задача принятия решения
Множество альтернатив {χ} и принцип выбора Ф определенны. Приложение Ф к {χ} не зависит от субъективных обстоятельств	Множество альтернатив {χ} достоверно определено, принцип выбора Ф не может быть формализован. Результат выбора зависит от того, кто и на основе какой информации принимает решение	Множество альтернатив {χ} не имеет определенных границ, принцип выбора Ф не определен и даже не может быть зафиксирован

{χ}– множество альтернатив и Ф – принцип выбора



В зависимости от количества равноправных специалистов принимающих решение, различают:

- задачи индивидуального принятия решения или выбора;
- задачи группового принятия решения или выбора.

В зависимости от количества критериев, используемых для оценки исходов, различают:

- однокритериальные задачи;
- многокритериальные задачи.

В зависимости от конечных целей, различают задачи:

- выбора единственного варианта;
- выбора подмножества вариантов;
- упорядочения вариантов;
- классификации вариантов.

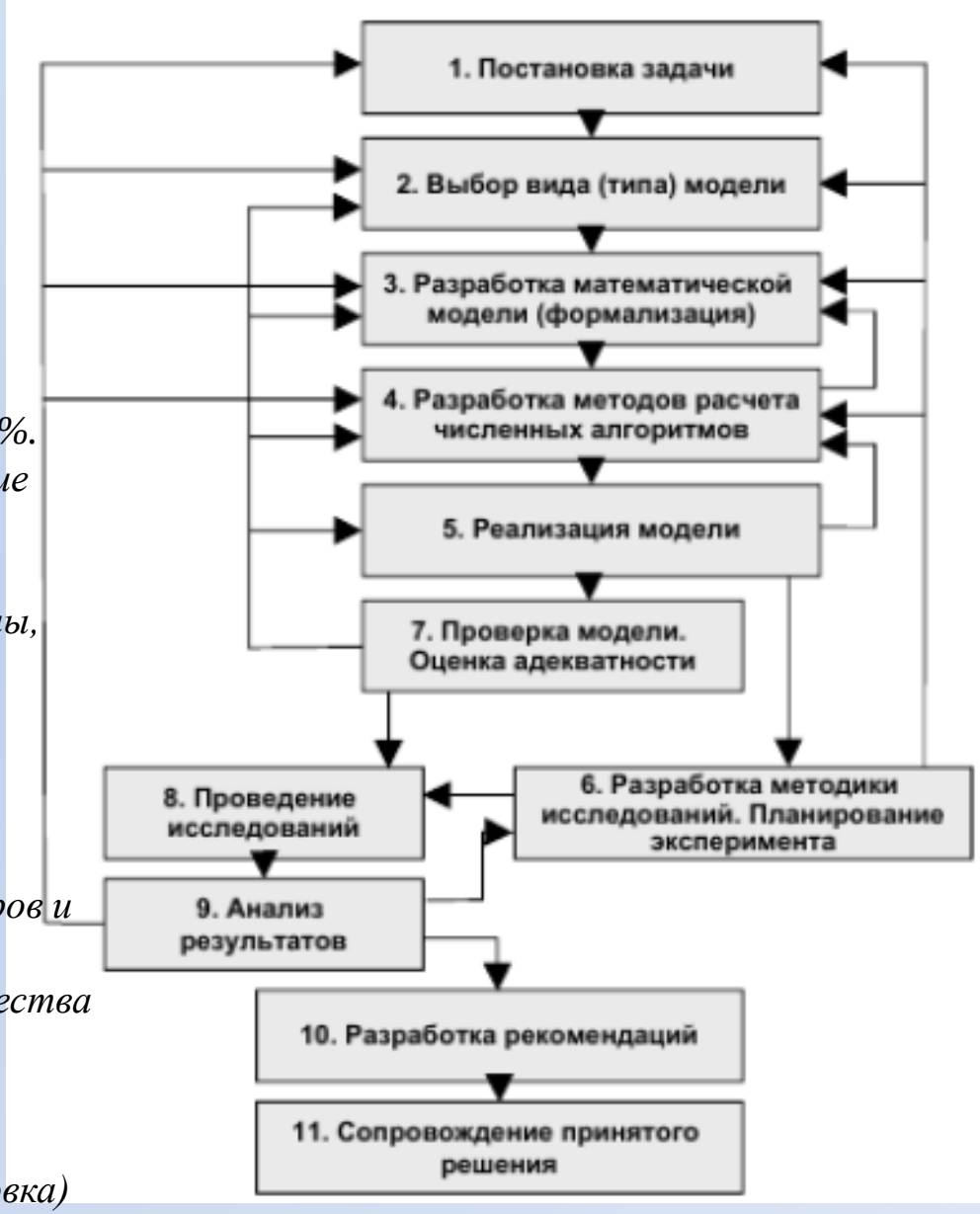
5. Этапы моделирования

оценка времени на отдельные этапы моделирования:

− постановка задачи – 40-50%;
− разработка модели – 20-30%;
− эксперимент, анализ результатов – 20-30%.

При постановке задачи решаются следующие взаимосвязанные задачи:

1. Уяснение цели (задачи) исследования.
2. Изучение объекта моделирования (системы, процесса).
3. Анализ доступной информации.
4. Выявление релевантных факторов.
5. Формулирование альтернатив.
6. Определение ограничений и допущений. Определение диапазона изменения параметров и переменных.
7. Выбор критерия, системы критериев качества достижения цели.
8. Установление масштаба предстоящего эксперимента.
9. Математическая постановка (формулировка) задачи.



4. Методы моделирования систем.

методы, направленные на активизацию интуиции и опыта специалистов	- методы организации сложных экспертиз; - метод экспертных оценок; - морфологические методы; - методы структуризации; - методы типа «Дельфи»; - методы типа «сценариев»; - методы типа «мозгового штурма» и выработки коллективных решений; - методы диалектической логики.
методы формализованного представления систем	- графические методы; - семиотические методы; - лингвистические методы; - логические методы; - теоретико-множественные методы; - статистические методы; - аналитические методы оптимизации.
специальные методы постепенной формализации задачи	- имитационное динамическое моделирование; - ситуационное моделирование; - структурно-лингвистическое моделирование; - когнитивный подход; - формализация моделей принятия решений; - информационный подход к анализу систем.

Методы организации сложных экспертиз

К ним относятся методы организации индивидуальных экспертиз, когда обращаются к квалифицированному специалисту в исследуемой области – эксперту. В этой методике выделяются группы критериев оценки и рекомендуется ввести весовые коэффициенты критериев. Введение критериев позволяет организовать опрос экспертов более дифференцированно, а весовые коэффициенты – повышают объективность результирующих оценок.

Методы структуризации

Для успешной подготовки решений особенно важно то, что метод структуризации позволяет расчленять сложную, трудноразрешимую задачу на совокупность относительно простых, для решения которых существуют проверенные приемы и методы.

Термин "дерево" предполагает использование иерархической структуры, полученной путем разделения общей цели на подцели.

Метод "дерева целей" ориентирован на получение относительно устойчивой структуры целей проблем, направлений. Для достижения этого при построении первоначального варианта структуры следует учитывать закономерности целеобразования и использовать принципы формирования иерархических структур.

Морфологические методы

Основная идея морфологического подхода – систематически находить все возможные варианты решения проблемы путем комбинирования выделенных элементов или их признаков.

Методы типа «мозгового штурма» и выработки коллективных решений

Методы данного типа преследуют основную цель – поиск новых идей, их широкое обсуждение и конструктивную критику. Основная гипотеза заключается в предположении, что среди большого числа идей имеются, по меньшей мере, несколько хороших.

При проведении обсуждений по исследуемой проблеме применяются следующие правила:

- сформулировать проблему в основных терминах, выделив единственный центральный пункт;
- не объявлять идею ложной и не прекращать исследование ни одной идеи;
- поддерживать идею любого рода, даже если ее уместность кажется вам в данное время сомнительной;
- оказывать поддержку и поощрение, чтобы освободить участников обсуждения от скованности.

Лингвистические методы

Лингвистические методы можно разделить на когнитивные и семантико-стилистические подгруппы. Анализируются ментальный и языковой феномены. А также проводятся исследования языковых, текстовых или дискурсивных феноменов.

Логические методы исследования базируются на применении в процессе исследований формальной логики. *Формальная логика* – наука о законах выводного знания, т.е. знания, полученного из ранее установленных и проверенных истин, без обращения в каждом конкретном случае к опыту, а только в результате применения законов и правил мышления. Формальная логика включает: традиционную логику; математическую логику.

Традиционная логика при получении новых (выводных) знаний использует следующие логические методы.

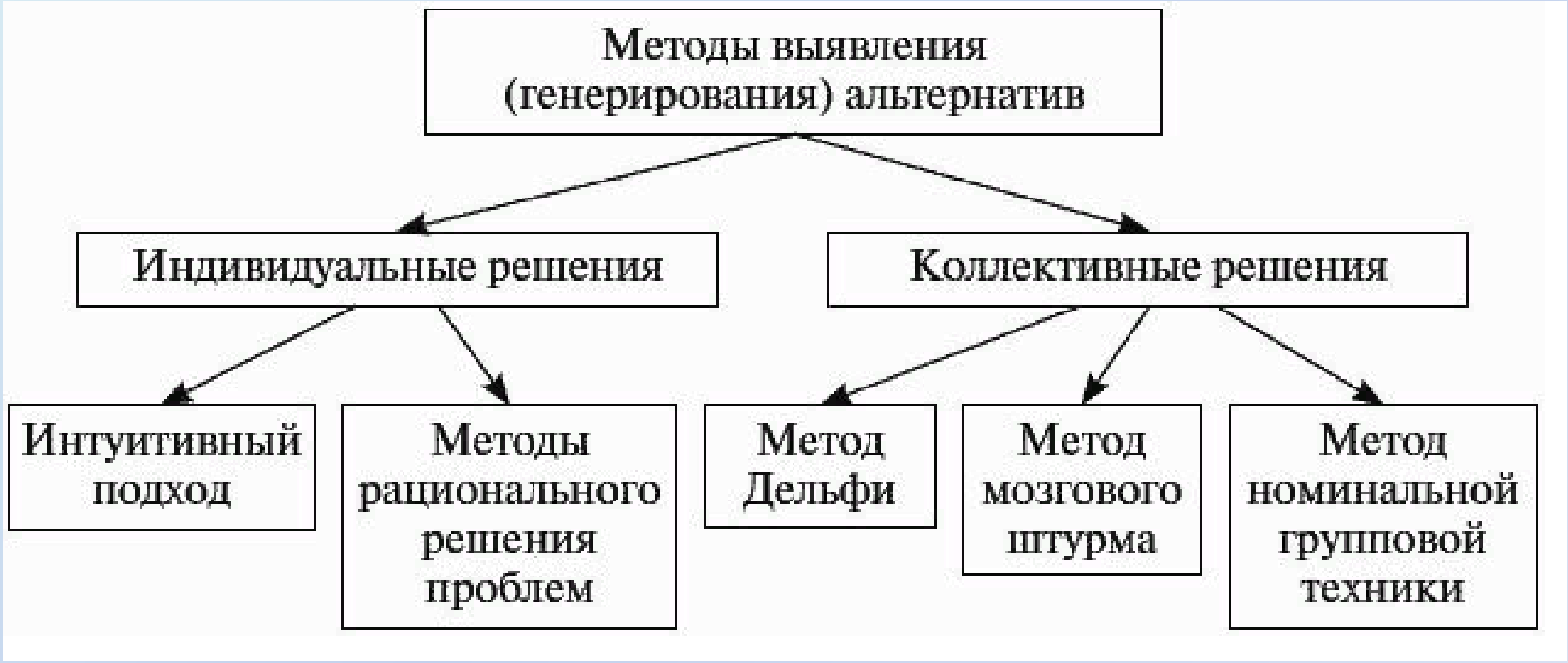
Анализ – логический метод расчленения целого на отдельные элементы с рассмотрением каждого из них в отдельности.

Синтез – объединение всех данных, полученных в результате анализа. Задача состоит в мысленном воспроизведении основных связей между элементами анализируемого целого.

Индукция – процесс движения мысли от частного к общему, от ряда факторов к закону.

Дедукция – процесс движения мысли от общего к единичному, от закона к отдельным его проявлениям.

Обобщение – метод научного познания, его можно рассматривать, как логический процесс перехода от единичного к общему, от менее общего к более общему знанию, установления общих свойств и признаков предметов, или как результат этого процесса: обобщенное понятие, суждение, закон, теория.


Методы выявления (генерирования) альтернатив
Индивидуальные решения
Коллективные решения
Интуитивный подход
Методы рационального решения проблем
Метод Дельфи
Метод мозгового штурма
Метод номинальной групповой техники

Метод Дельфи - многоэтапный метод, предусматривающий первоначальное изолированное вынесение экспертами своих суждений и дальнейшую многократную их корректировку на базе ознакомления каждого эксперта с суждениями других экспертов до тех пор, пока *величина разброса оценок не будет находиться в рамках* заранее устанавливаемого желаемого интервала варьирования оценок.

Получаемые посредством данных методик оценки носят статичный и одноразовый характер, в результате чего возникает необходимость повторного обращения к экспертам при составлении прогноза доли рынка на последующие периоды. Кроме того, метод внутреннего и внешнего экспертного прогнозирования характеризуется определенной степенью субъективности.



Метод мозгового штурма - метод психологической активизации, при котором участникам обсуждения предлагают высказывать возможно большее количество вариантов решения, в том числе самых фантастических.

Затем из общего числа высказанных идей отбирают *наиболее удачные*, которые могут быть использованы на практике. Является методом экспертного оценивания и широко используется во многих организациях для поиска нетрадиционных решений самых разнообразных задач.

Метод мозгового штурма был разработан Алексом Осборном в 1953 году. Классическая техника мозгового штурма, предложенная Осборном, основывается на двух основных принципах — «отсрочка вынесения приговора идее» и «из количества рождается качество».

Метод номинальных групп (МНГ) – один из способов коллективного принятия решений. По форме МНГ – это тщательно спланированный структурированный групповой процесс.

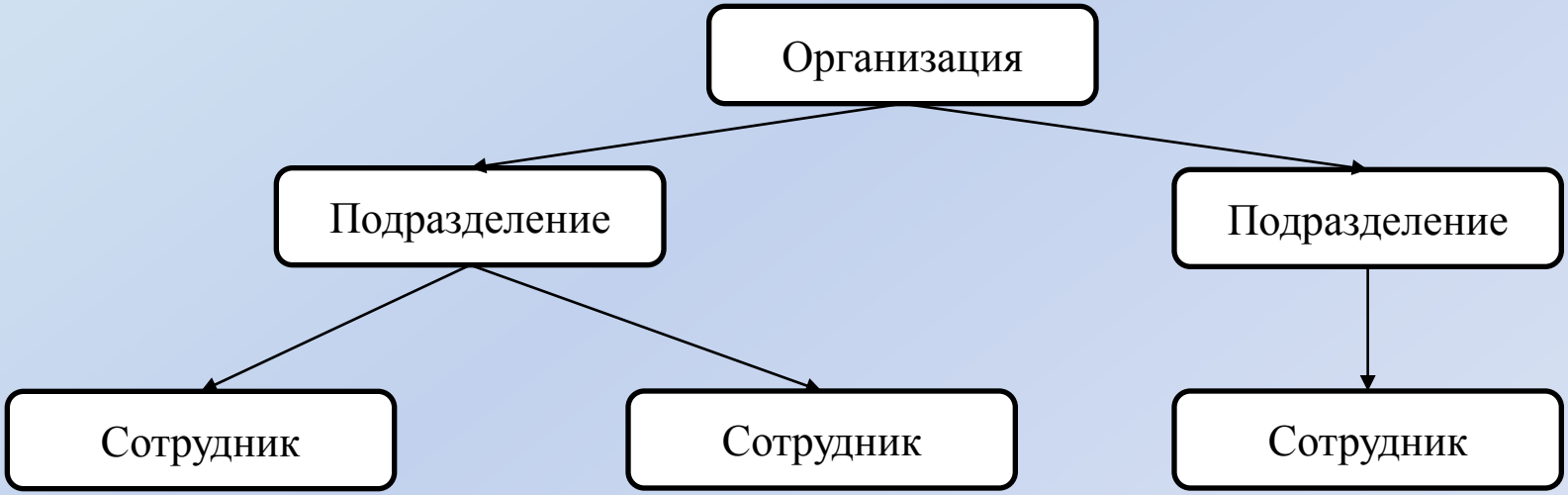
Данный метод полезен в тех случаях, когда необходимо выявить и сопоставить индивидуальные суждения, с тем чтобы получить решения, к которым один человек прийти не может. Кроме того, во время работы МНГ участники получают друг от друга информацию, которая во время обычных совещаний остается не услышанной. Это могут быть свежие идеи, новый взгляд на проблему, неожиданно верное решение.

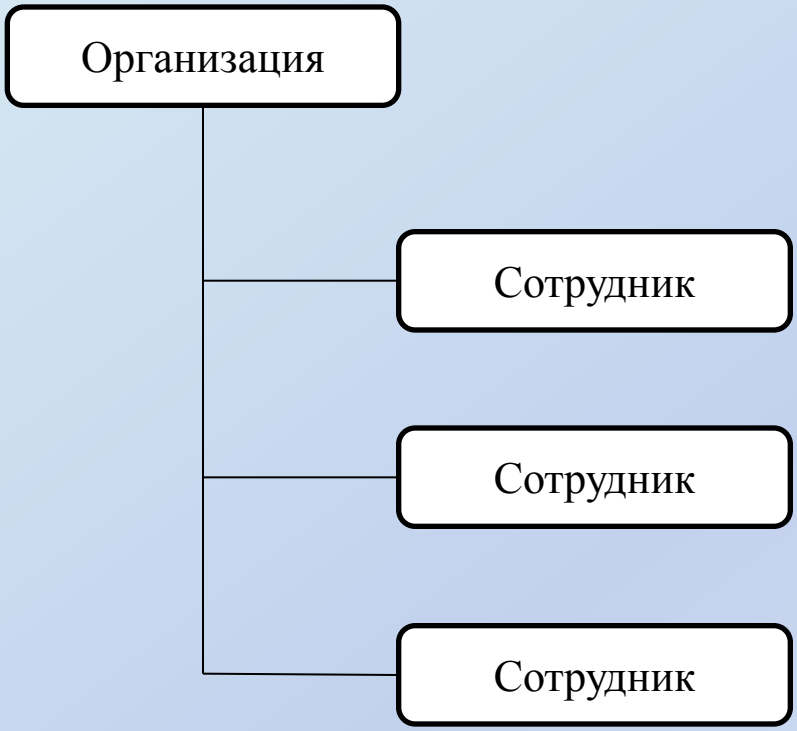
Метод разработан Андре Л. Дельбеком и Эндрю Ван де Веном на основе социально-психологических исследований совещаний по принятию решений и согласования групповых суждений.

Метамоделирование.

Метамодель – это модель языка моделирования.

Пример простых моделей:



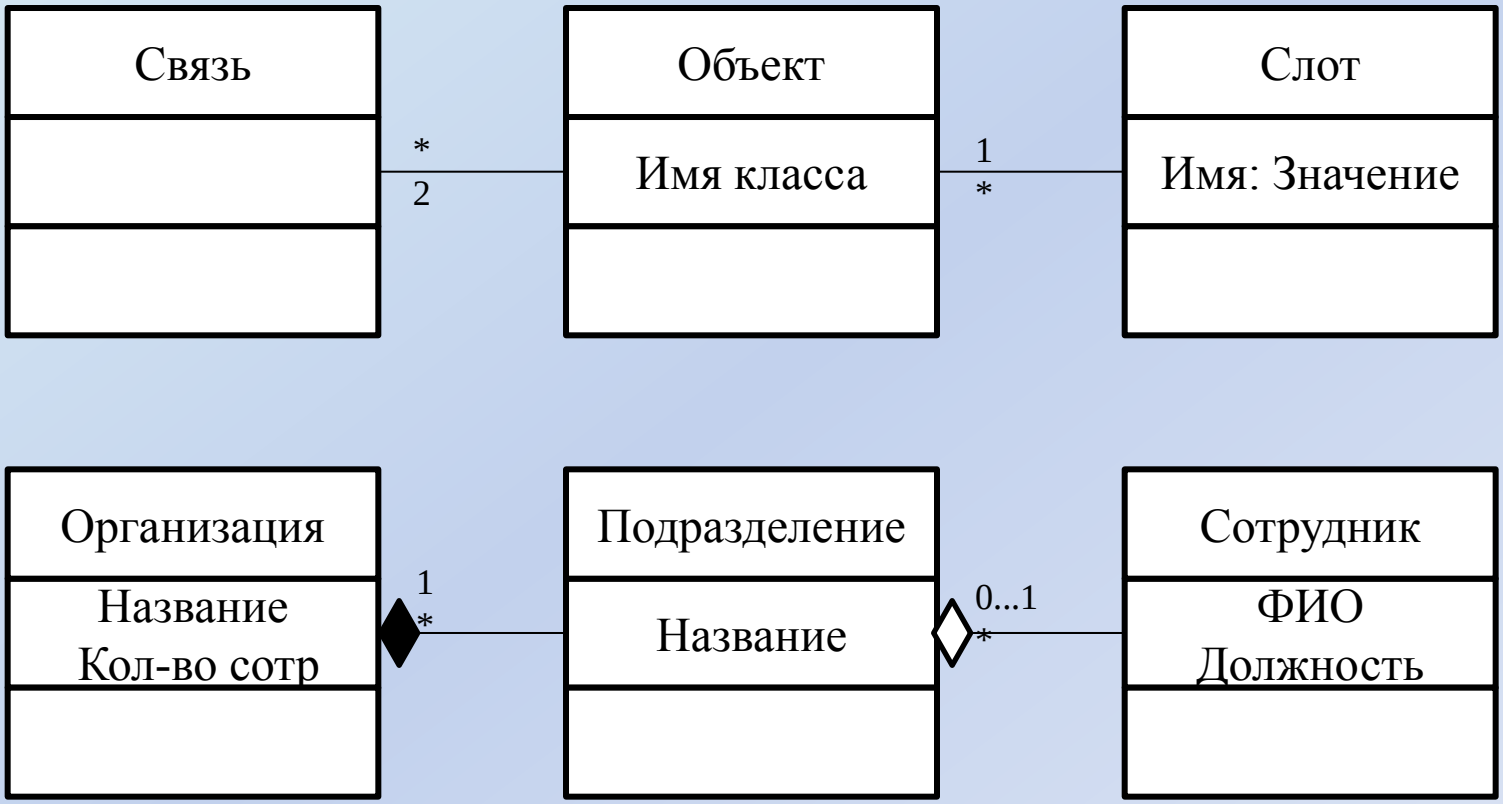


Организация

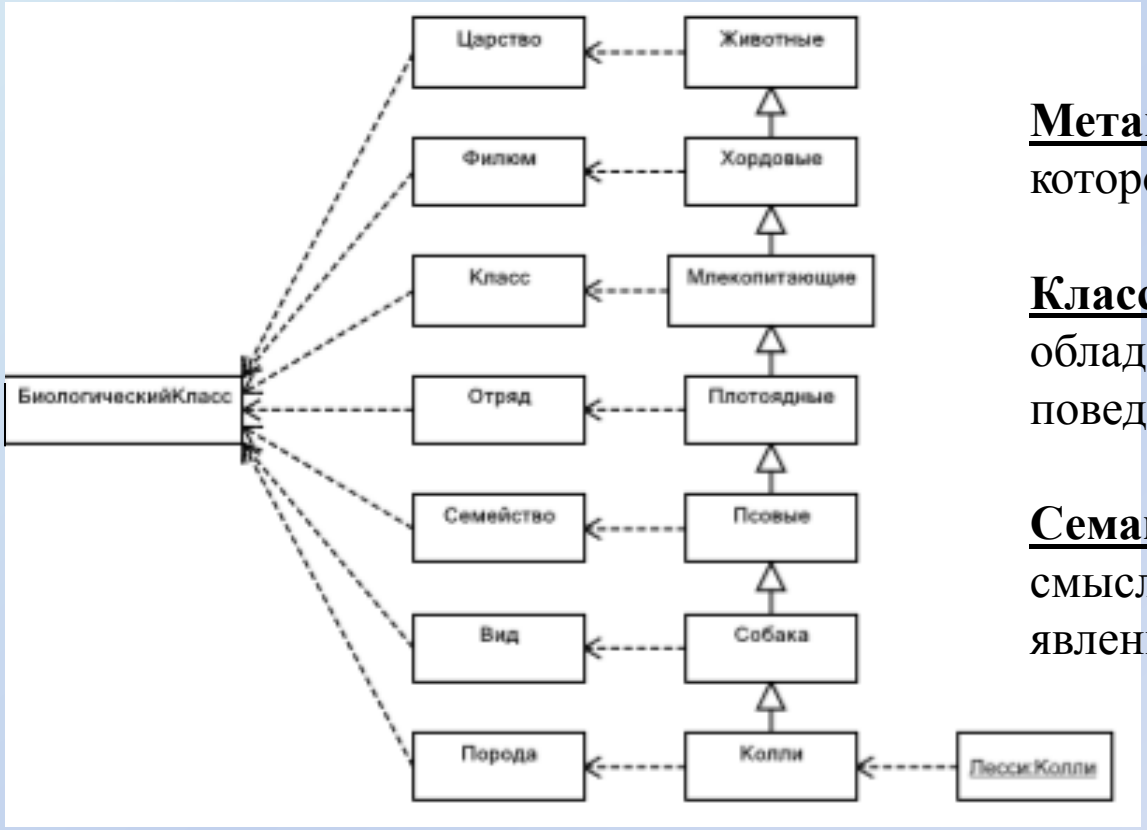
Кол-во сотрудников = 3

Элементы: объект, связь и слот (количество сотрудников)

Пример метамодели:



Пример многоуровневого онтологического моделирования



Онтологическое моделирование - это процесс построения, развития, обработки и использования онтологии предметной области.

Метакласс – это класс, экземплярами которого являются другие классы.

Класс – это совокупность объектов, обладающих схожей структурой, поведением и семантикой.

Семантика – это суть, значение, смысл некоторого предмета или явления.

Системный подход — это подход, при котором любая система (объект) рассматривается как совокупность взаимосвязанных элементов (компонентов), имеющая выход (цель), вход (ресурсы), связь с внешней средой, обратную связь.

Аспекты:

1. системно-элементного или системно-комплексного, состоящего в выявлении элементов, составляющих данную систему.

2. *системно-структурного,* заключающегося в выяснении внутренних связей и зависимостей между элементами данной системы и позволяющего получить представление о внутренней организации (строении) исследуемой системы;

3. системно-функционального, предполагающего выявление функций, для выполнения которых созданы и существуют соответствующие системы;

4. системно-целевого, означающего необходимость научного определения целей и подцелей системы, их взаимной увязки между собой;

5. *системно-ресурсного*, заключающегося в тщательном выявлении ресурсов, требующихся для функционирования системы, для решения системой той или иной проблемы;

6. *системно-интеграционного*, состоящего в определении совокупности качественных свойств системы, обеспечивающих её целостность и особенность;

7. *системно-коммуникационного*, означающего необходимость выявления внешних связей данной системы с другими, то есть её связей с окружающей средой;

8. *системно-исторического*, позволяющего выяснить условия во времени возникновения исследуемой системы, пройденные ею этапы, современное состояние, а также возможные перспективы развития.

Системный подход – основа методологии системного анализа.



При рассмотрении системы посредством системного подхода можно выделить некоторые особенности, которые коррелируются с принципами системного анализа и дополняют его:

1. Целостность системы.
2. Связность системы.
3. Изменение системы в процессе ее развития.
4. Превосходство общей (глобальной) цели.
5. Приоритетность структуры.
6. Децентрализация и централизация.
7. Модульность структуры системы.
8. Структурность системы.
9. Направление свертки: информация и управляющие воздействия свертываются, укрупняются при движении по иерархии снизу вверх.
10. Минимизация неопределенности переходов системы.
11. Достоверность информации.
12. Исполняемость команд.

Системный подход – основа методологии системного анализа

Выполнение действий при системном подходе осуществляется с применением их отдельных частей, более мелких действий, при этом составные части должны быть выполнены не в произвольном порядке, а в их определенной (алгоритмической) последовательности.

Определенная, подчиненная цели взаимозависимость составных частей действий при системном подходе является признаком системности. Сохраняя логическую принудительность последовательности действий, допускается, что в алгоритме данной деятельности могут присутствовать и такие действия, которые не формализованы; имеет значение то, чтобы этот этап алгоритма успешно выполнялся человеком, хотя и не осознанно.

При этом необходимым моментом является то, что:

- любая деятельность алгоритмична;

- не всегда алгоритм реальной деятельности осознается;

- в случае неудовлетворительности результатом деятельности возможную причину неудачи следует искать в несовершенстве алгоритма.

Это означает попытку выявить алгоритм, исследовать его, устранить недостатки, т. е. совершенствовать алгоритм и, следовательно, повышать системность деятельности.

ГОСТ Р 43.0.30—2022 Информационное обеспечение техники и операторской деятельности. Системность.

Проблемы разработки возможных путей развития системы можно разделить на три класса:




Уровень управления
Характер решаемых задач
Верхний Top-менеджмент
Директор информационной службы Chief Information Officer (CIO)
Стратегические задачи с высокой степенью неопределенности и риска, затрагивают предприятие в целом
Средний Middle-менеджмент
Начальники IT-подразделений компании
Слабо структурированные задачи в условиях неполной информации, затрагивают структурные подразделения IT-службы
Низший Low-менеджмент
IT-менеджеры, ответственные за "узкие" участки работы
Оперативные вопросы максимально формализованные, затрагивают операционную деятельность

	Признак деления	Классы прикладных программ
1	Комплексные системы (сетевые) предприятий	Малые и средние, средние, средние и крупные, корпоративные системы
2	Универсальные блоки и модули	Бухгалтерский учет, склад, учет труда и зарплаты, кадры, ведение договоров, первичные документы, документооборот
3	Отраслевые версии и спецмодули	Торговля, строительство, системы с функциями «западного» учета, бюджет, страхование, коммунальное хозяйство, промышленность, транспорт, индустрия сервиса, аудит
4	Аналитические программы	Ретроспективный анализ, прогнозный анализ, анализ полного состава ресурсов, анализ финансовых ресурсов
5	Системы для бизнеса	Бизнес-план, маркетинг, инвестиционные проекты, прогнозирование и моделирование, программы для директоров

Предметная область информационной системы

Предметная область - это часть реального мира, которая представляется, отображается и используется в ИС.

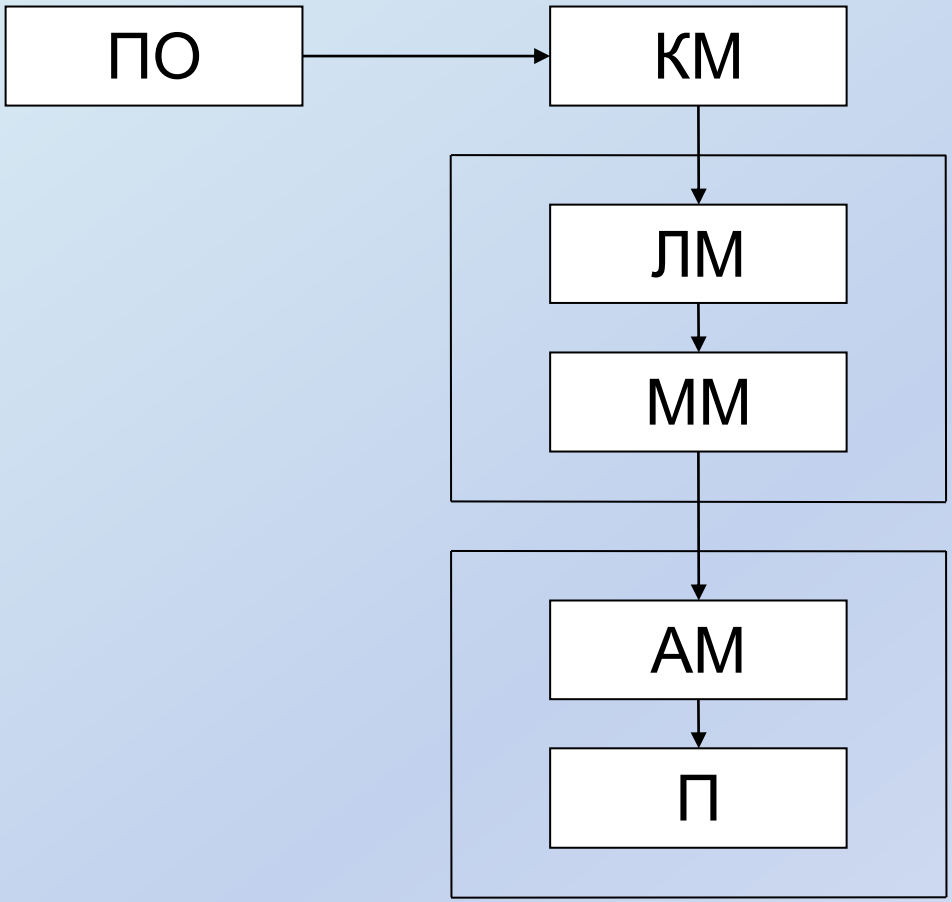
Отображение ПО представляется с помощью различных форм информации называемыми **информационными моделями.**

Модель - это объект, который в определенных задачах может заменять оригинал.

Модель предметной области – параметрическое формализованное представления процесса циркуляции и обработки информации в системе производства и управления, которая отображает действительность в упрощенном виде.

Предметная область представляется с помощью информационных моделей нескольких уровней.

Уровни информационных моделей



ПО – предметная область,
КМ – концептуальная модель,
ЛМ – логическая модель,
ММ – математическая модель,
АМ – алгоритмическая модель,
П – программа.

КМ (Концептуальная Модель) - это модель, которая отображает знания специалиста в предметной области о ее объектах и их взаимосвязях, процессах и результатах деятельности. На этом этапе могут быть использованы: тексты, таблицы, графики, графы, блок-схемы.

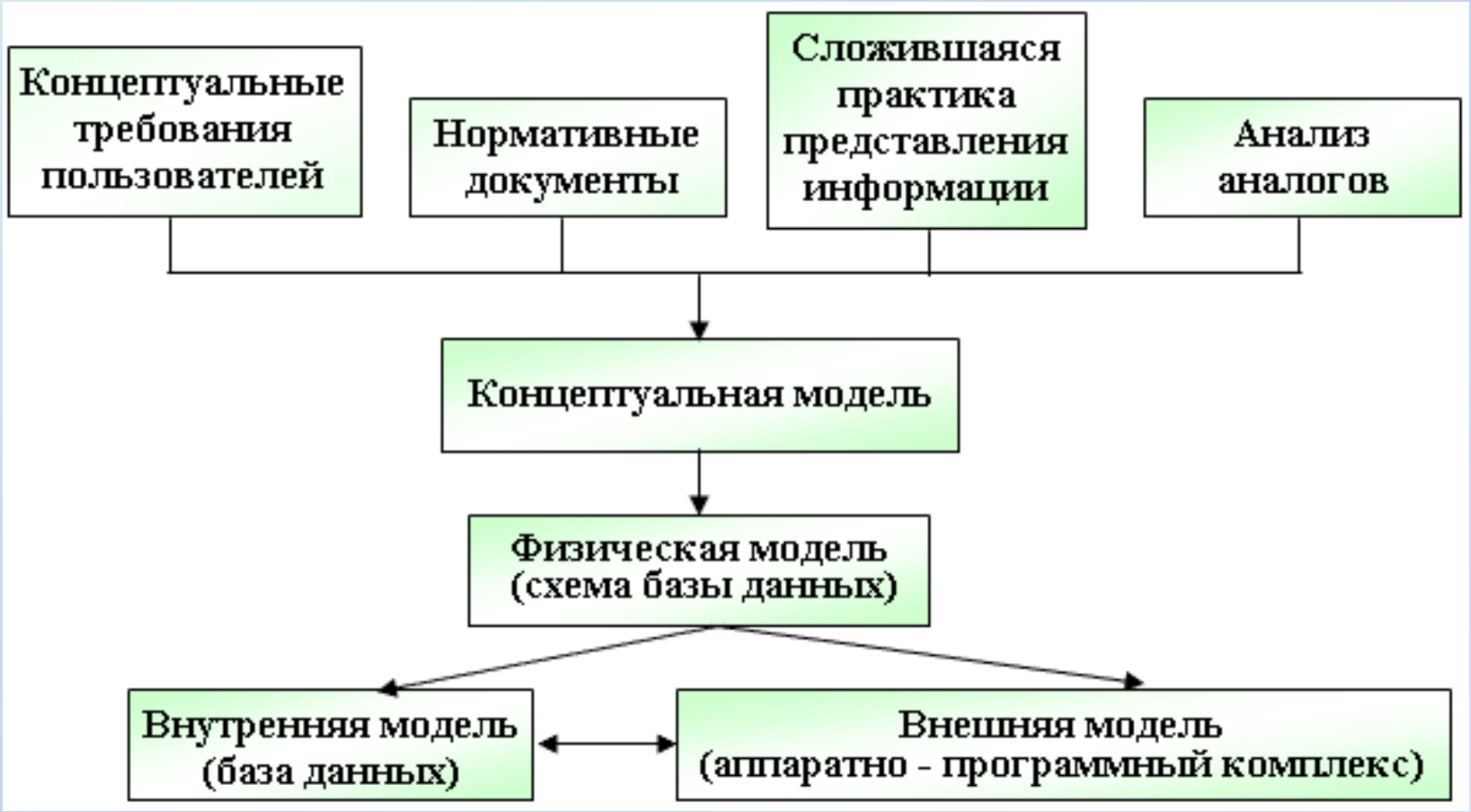
Логическая модель (ЛМ) – формируется из концептуальной путем выделения конкретной части, подлежащей управлению, ее детализации и формализации. ЛМ ПО выражается с помощью инструментария анализа и проектирования ИС.

Математическая модель (ММ). ЛМ, представляющая взаимосвязи в выделенной ПО с помощью математического аппарата.

Алгоритмическая модель (АМ). ММ, представленная в виде последовательности действий, реализующих достижение поставленной цели, преобразуется в АМ.

Программа (П). На основе АМ создается машинная программа, которая является той же АМ, только представленной на языке, понятном ЭВМ.

Схема формирования ИС

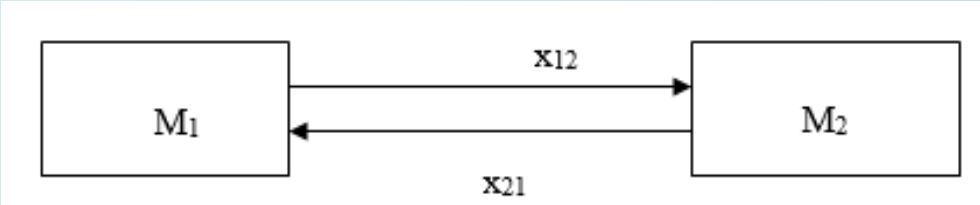


3. Понятие информационной системы.

Информационная система – это комплекс информационных ресурсов и технологий, предназначенный для сбора, хранения и обработки данных в рамках некоторой предметной области.

Элемент	- предел членения системы с точки зрения аспекта рассмотрения, решения конкретной задачи, поставленной цели
Подсистема	- выделение относительно независимой части системы, обладающей свойствами системы.
Компонент	- Часть системы, состоящая из совокупности однородных элементов

Элементы, связи, система, внешняя среда



Связь двух элементов

M –элемент
{M} совокупность элементов
$M_i \in \{M\}$- принадлежность элемента совокупности

Характеристики связи

- направление (направленные и ненаправленные);
- сила (сильные и слабые);
- характер (или вид).

- подчинение;
- порождение (или генетические);
- равноправные (или безразличные);
- управление.

Сложная система - система, состоящая из элементов разных типов и обладающая разнородными связями между ними.

$$\{M\}:\{\{M^1\},\{M^2\},...,\{M^R\}\} \qquad \{M\}=\bigcup_{r=1}^{R}\{M^r\}$$

Автоматизированные или вычислительные системы (АС, ВС).

АС - сложная система с определяющей ролью элементов двух типов:

- в виде технических средств;
- в виде действий человека.

$$\Sigma^A:\{\{M^T\},\{M^Ч\},\{x^T\},\{x^{T-Ч}\},\{x^Ч\},F\}$$

M^T - технические средства, в первую очередь ЭВМ;

$M^Ч$ - решения и другая активность человека;

$\{x^{T-Ч}\}$ - совокупность связей между человеком и техникой.

Структура системы. Декомпозиция. Иерархия.

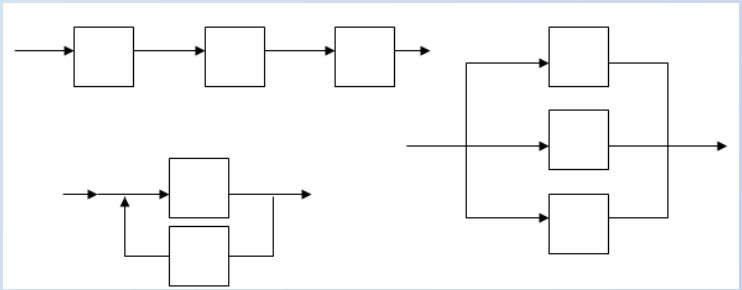
$$\sum\sum:\{\{\hat{M}\},\{\hat{x}\}\}$$

$\hat{M}$ - совокупность групп элементов (компонентов)

$\hat{x}$ - совокупность связей между этими группами.

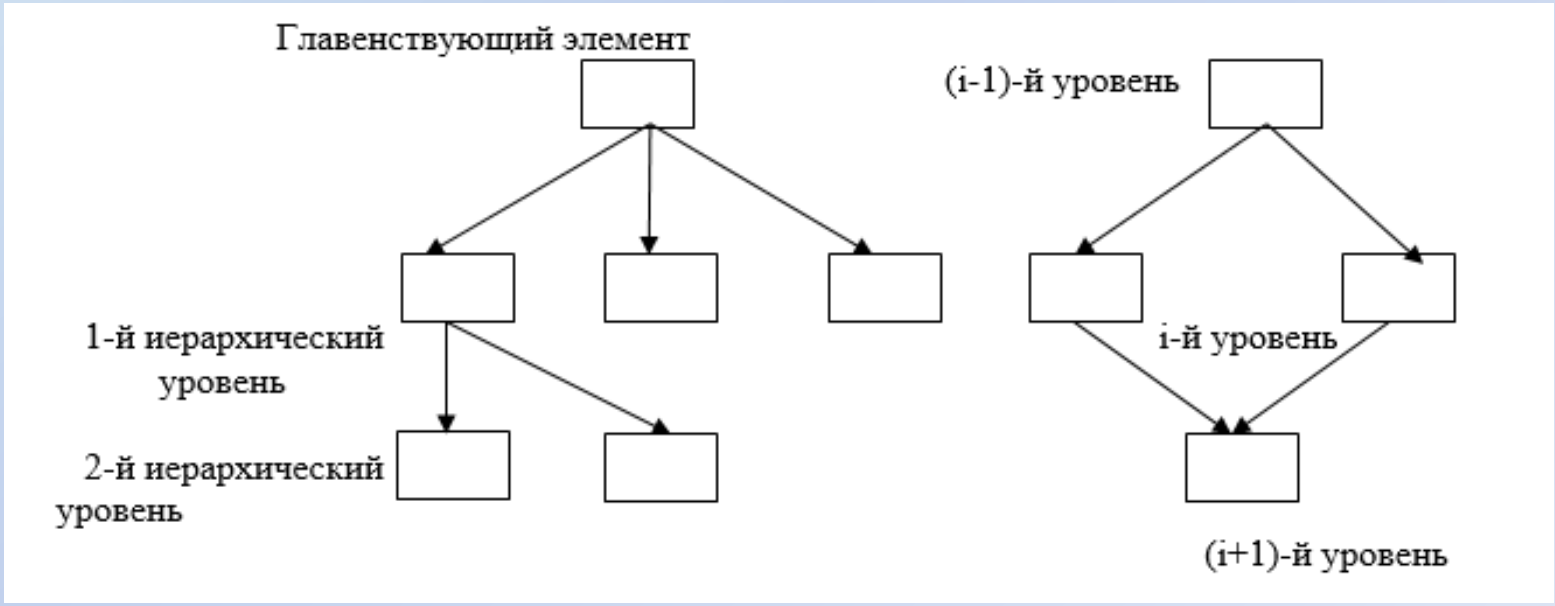


Декомпозицией называется деление системы на части, удобное для каких-либо операций с этой системой.



Простейшие типы связей

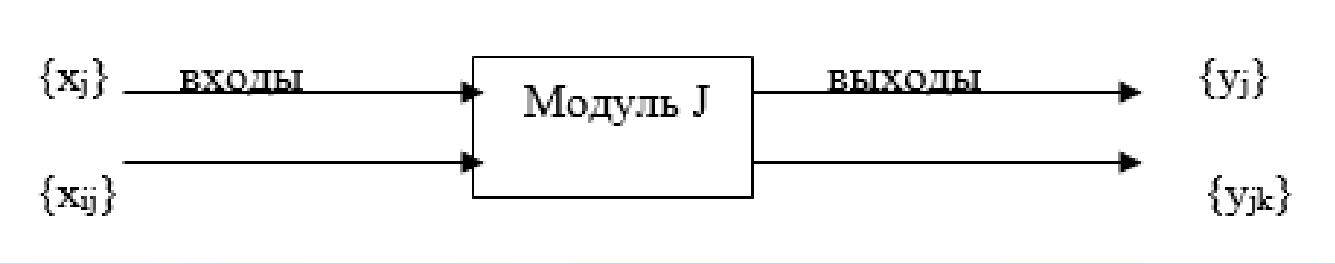
Иерархические структуры:



Древовидная

Ромбовидная

Модульное строение системы



x_j – внешние (от «не системы») воздействия на элементы модуля;

xij – воздействия от других элементов системы на элементы модуля;

y_{jk} – воздействия от элементов модуля на другие элементы системы;

y_j – воздействия от элементов модуля на «не систему».

$$(\{x_j\},\{x_{ij}\},J) \rightarrow (\{y_j\},\{y_{jk}\})$$

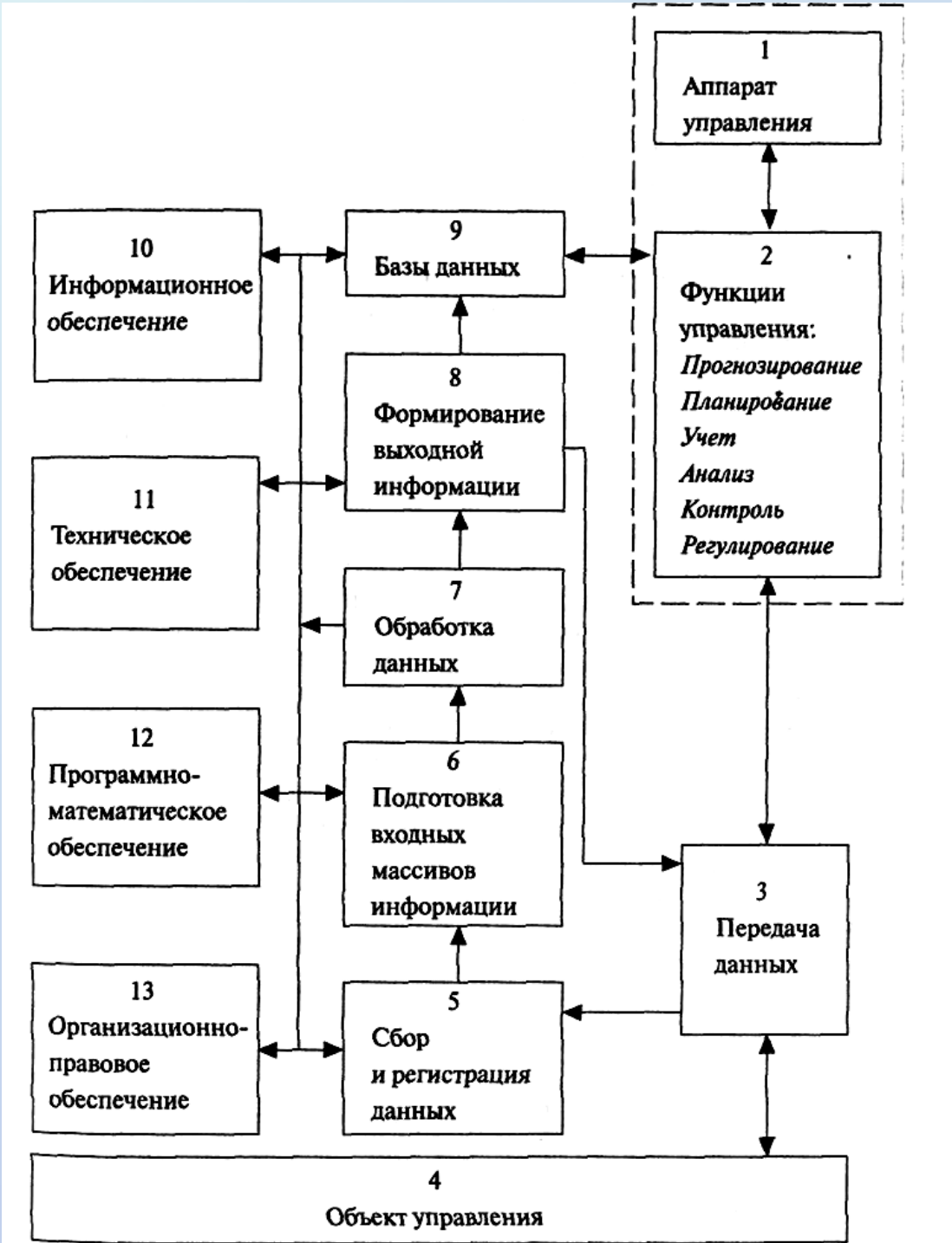
Процессы в системе

Процесс - набор состояний системы, соответствующий упорядоченному (непрерывному или дискретному) изменению некоторого параметра, определяющего характеристики системы.

$$S_{t_0 t}(z(t_0)) = z(t), z \in Z, t \in T$$

Z - значения характеристик

$S_{t_0 t}$ - некоторое правило перехода от состояния в момент t_0 к состоянию в момент $t > t_0$ через все его промежуточные непрерывные или дискретные значения


1
Аппарат управления
2
Функции управления:
Прогнозирование
Планирование
Учет
Анализ
Контроль
Регулирование
3
Передача данных
4
Объект управления
5
Сбор и регистрация данных
6
Подготовка входных массивов информации
7
Обработка данных
8
Формирование выходной информации
9
Базы данных
10
Информационное обеспечение
11
Техническое обеспечение
12
Программно-математическое обеспечение
13
Организационно-правовое обеспечение

ЛЕКЦИЯ 3.
Объектно-ориентированное моделирование.

Сущность структурного подхода. Базовые принципы структурного подхода

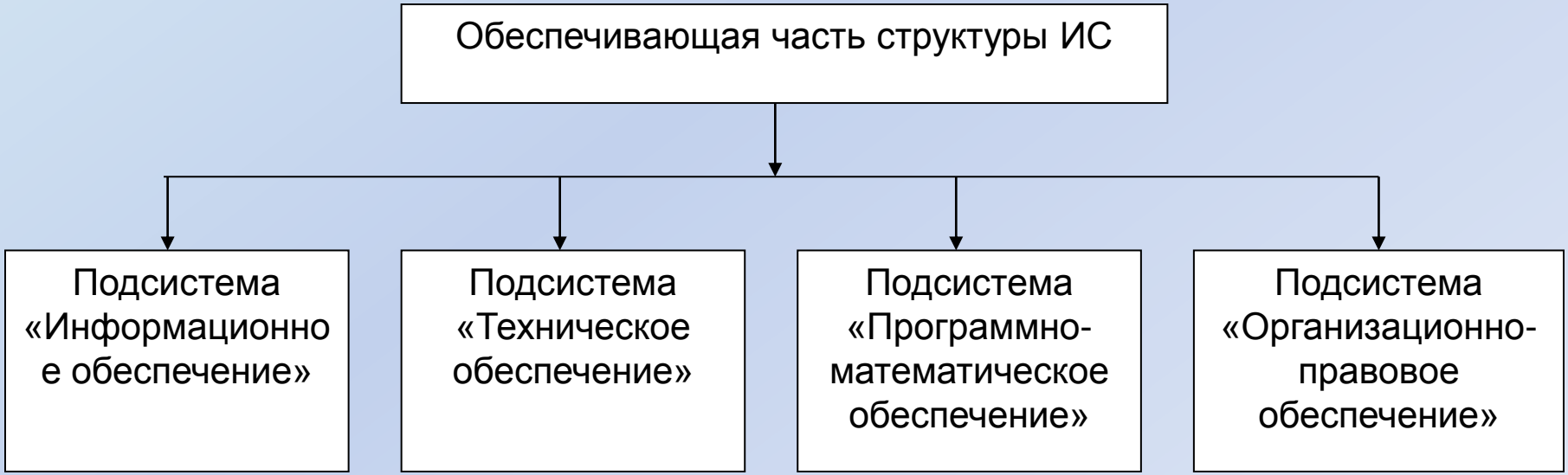
Принципы структурного похода:

1. **Разделяй и властвуй**. Для решения сложной задачи используется ее разбиение на несколько более простых подзадач.
2. **Иерархическое упорядочивание**. Решение задачи представляется в виде иерархии описаний (функций, потоков данных), где описания вышестоящих уровней детализируются при помощи описаний нижестоящих уровней.
3. **Абстрагирование**. При решении задачи рассматриваются только важные ее аспекты; все незначительное отбрасывается.
4. **Формализация**. Все принятые решения описываются формально.
5. **Непротиворечивость**. Никакая часть системы не может противоречить никакой другой. Все должно быть согласованно и находиться в соответствии.

Состав и структура информационной системы

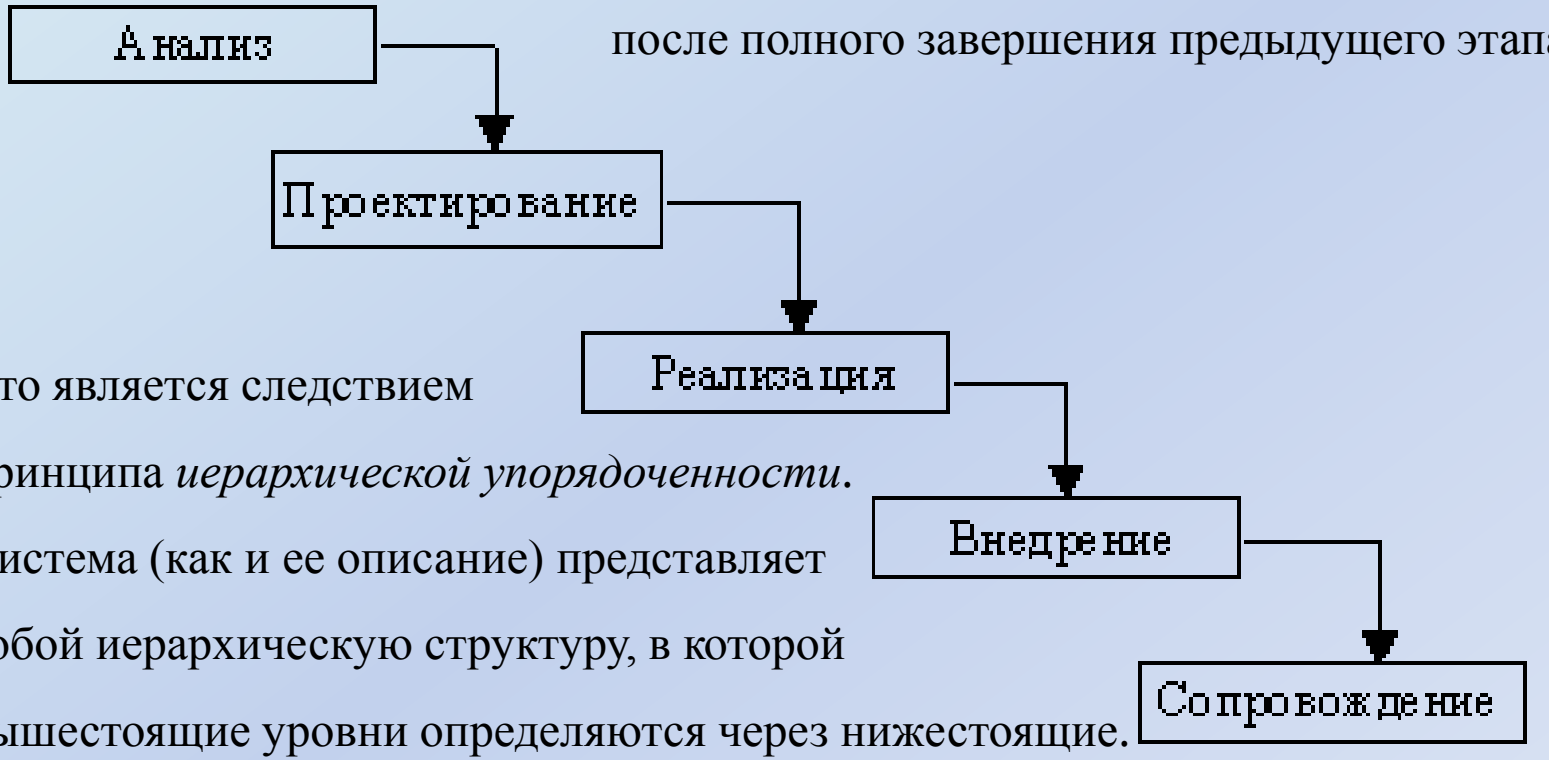
Структуру информационных систем составляет совокупность отдельных ее частей, называемых подсистемами.

Подсистема - это часть системы, выделенная по какому-либо признаку.



Модель процесса

Для структурного подхода характерен *каскадный жизненный цикл* системы, когда каждый последующий этап решения задачи выполняется после полного завершения предыдущего этапа.



Это является следствием принципа *иерархической упорядоченности*. Система (как и ее описание) представляет собой иерархическую структуру, в которой вышестоящие уровни определяются через нижестоящие. Любую вершину в иерархии можно считать определенной только в том случае, если определены все ее потомки.

Процессы в информационной системе



1. ввод информации из внешних или внутренних источников;
2. обработка входной информации и представление ее в удобном виде;
3. вывод информации для представления потребителям или передачи в другую систему;
4. обратная связь - это информация, переработанная людьми данной организации для коррекции входной информации.

Этапы

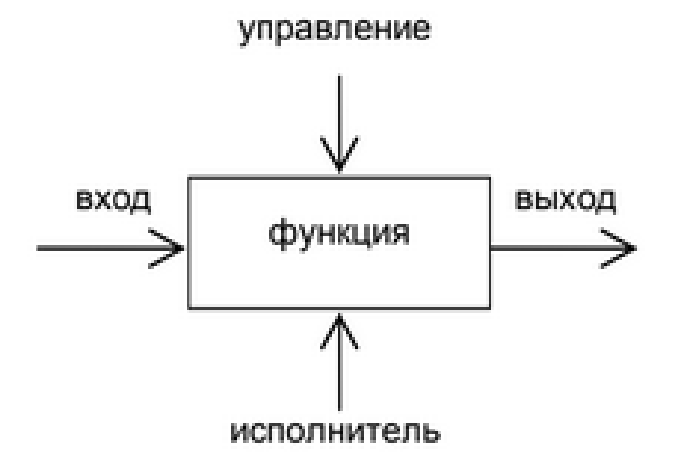
Зарождение данных - формирование первичных сообщений, которые фиксируют результаты определенных операций, свойства объектов и субъектов управления, параметры процессов, содержание нормативных и юридических актов и т.п.

Накопление и систематизация данных - организация такого их размещения, которое обеспечивало бы быстрый поиск и отбор нужных сведений, методическое обновление данных, защита их от искажений, потери и др.

Обработка данных - процессы, вследствие которых на основании прежде накопленных данных формируются новые виды данных: обобщающие, аналитические, рекомендательные, прогнозные. Производные данные тоже можно обрабатывать, получая более обобщенные сведения.

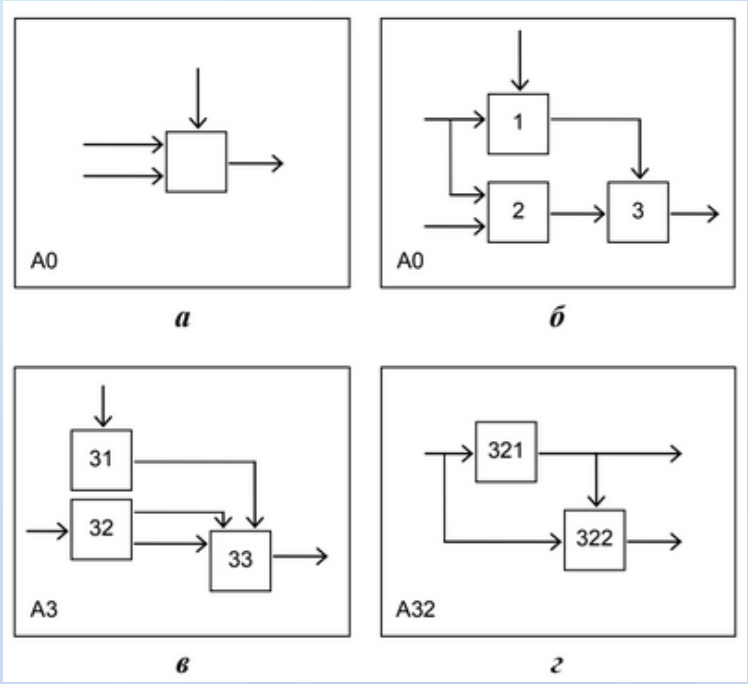
Отображение данных - представление их в форме, пригодной для восприятия человеком. Прежде всего - это вывод на печать, то есть создание документов на так называемых твердых (бумажных) носителях.

Методология *SADT* представляет собой совокупность методов, правил и процедур, предназначенных для построения функциональной модели объекта какой-либо предметной области. Функциональная модель *SADT* отображает функциональную структуру объекта, т.е. производимые им действия и связи между этими действиями.



Графическая нотация диаграмм SADT

Иерархия диаграмм



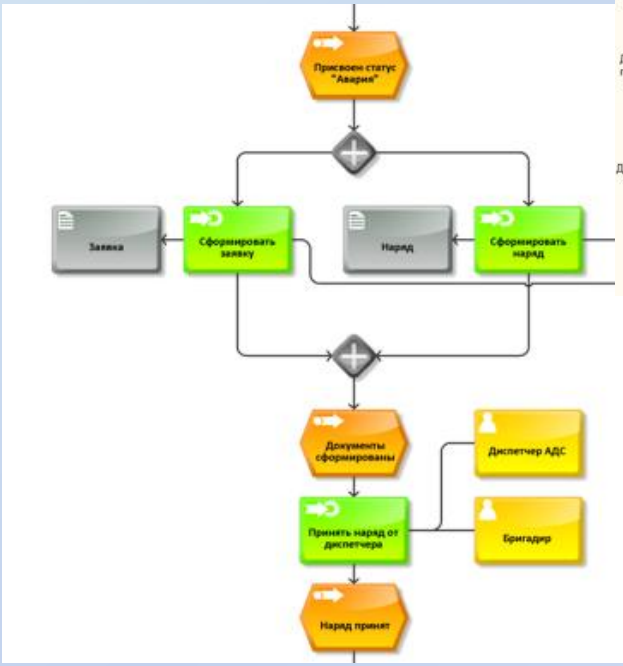
Детализация функций на дочерних диаграммах

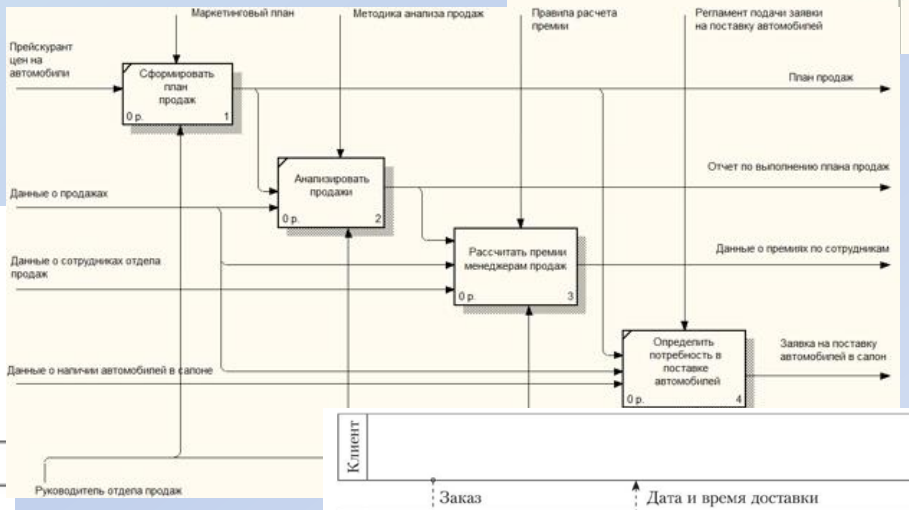
Сначала строится корневая диаграмма, на которой моделируемый объект (вся система в целом или какая-то ее подсистема) представляется в виде одной функции, для которой определяются все входные параметры, выходные величины, управляющая информация, а также исполнители (а).

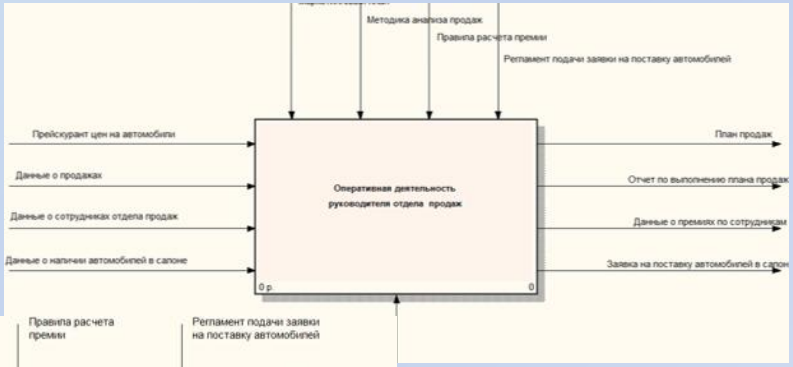
После этого полученная диаграмма детализируется путем разбиения основной функции на несколько подфункций(б).

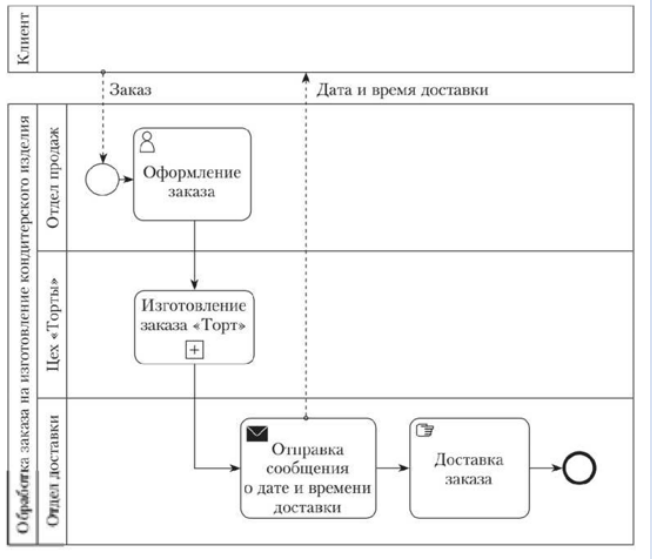
Далее процесс детализации рекурсивно повторяется для каждой нетривиальной подфункции (в, г).

Примеры реализации:









Сравнение существующих методик

*1. В **функциональных моделях** (DFD-диаграммах потоков данных, SADT-диаграммах) главными структурными компонентами являются **функции** (операции, действия, работы), которые на диаграммах связываются между собой потоками объектов.*

*2. **Достоинством** функциональных моделей является реализация структурного подхода к проектированию ИС по принципу "сверху-вниз", когда каждый функциональный блок может быть декомпозирован на множество подфункций и т.д., выполняя, таким образом, модульное проектирование ИС.*
Для функциональных моделей характерны процедурная строгость декомпозиции ИС и наглядность представления.

*3. При функциональном подходе объектные модели данных в виде **ER-диаграмм "объект — свойство — связь"** разрабатываются отдельно. Для проверки корректности моделирования предметной области между функциональными и объектными моделями устанавливаются взаимно однозначные связи.*

*4. Главный **недостаток** функциональных моделей заключается в том, что процессы и данные существуют отдельно друг от друга — помимо функциональной декомпозиции существует структура данных, находящаяся на втором плане. Кроме того, не ясны условия выполнения процессов обработки информации, которые динамически могут изменяться.*

*5. Перечисленные недостатки функциональных моделей снимаются в **объектно-ориентированных моделях**, в которых главным структурообразующим компонентом выступает **класс объектов с набором функций**, которые могут обращаться к атрибутам этого класса.*

Сущность объектно-ориентированного подхода

Первые объектно-ориентированные методы появились в начале 80-х гг. XX в., однако основные результаты были получены спустя 10 лет, когда Айвар Джекобсон, Джим Рембо и Грейди Буч представили свои подходы – *Objectory*, *OMT (Object Modeling Technique – **техника объектного моделирования***) и *Booch*.

Эти методы были впоследствии объединены в один, из которого и вырос унифицированный язык моделирования – *UML (Unified Modeling Language)*.



*Авторы поставили перед собой три главные **цели**:*

– моделировать системы целиком, от концепции до исполняемого артефакта, с помощью объектно-ориентированных методов;

– решить проблему масштабируемости, которая присуща сложным системам, предназначенным для выполнения ответственных задач;

– создать такой язык моделирования, который может использоваться не только людьми, но и компьютерами.

Суть объектного подхода заключается в объектной декомпозиции, т.е. система представляется в виде совокупности объектов, которые в процессе взаимодействия обмениваются сообщениями.

***Объект** – это самостоятельная, самодостаточная сущность, обладающая состоянием, поведением и семантикой.*

***Состояние** объектов представляют данные которые содержащиеся в объекте. В терминологии объектно-ориентированного программирования эти данные называются атрибутами. Атрибуты включают информацию, которая отличается у разных экземпляров объектов.*

***Поведение** объекта представляет то, что он может сделать. В процедурных языках поведение определяется процедурами, функциями и подпрограммами. В терминологии объектно-ориентированного программирования поведения объектов содержатся в методах, а вызов метода осуществляется путем отправки ему сообщения.*

Объект – это самостоятельная, самодостаточная сущность, обладающая состоянием, поведением и семантикой.

Суть объектного подхода заключается в объектной декомпозиции, т.е. система представляется в виде совокупности объектов, которые в процессе взаимодействия обмениваются сообщениями.

***Методы** реализуют требуемое поведение класса. Каждый объект, экземпляр которого окажется создан на основе этого класса, будет содержать методы, определяемые этим же классом. Методы могут реализовывать поведения, вызываемые из других объектов (с помощью сообщений) либо обеспечивать основное, внутреннее поведение класса. Внутренние поведения — это закрытые методы, которые недоступны другим объектам.*

***Сообщения** — это механизм коммуникаций между объектами. Например, когда объект A вызывает метод объекта B, объект A отправляет сообщение объекту B. Ответ объекта B определяется его возвращаемым значением. Только открытые, а не закрытые методы объекта могут вызываться другим объектом.*

3. Базовые принципы объектно-ориентированного подхода

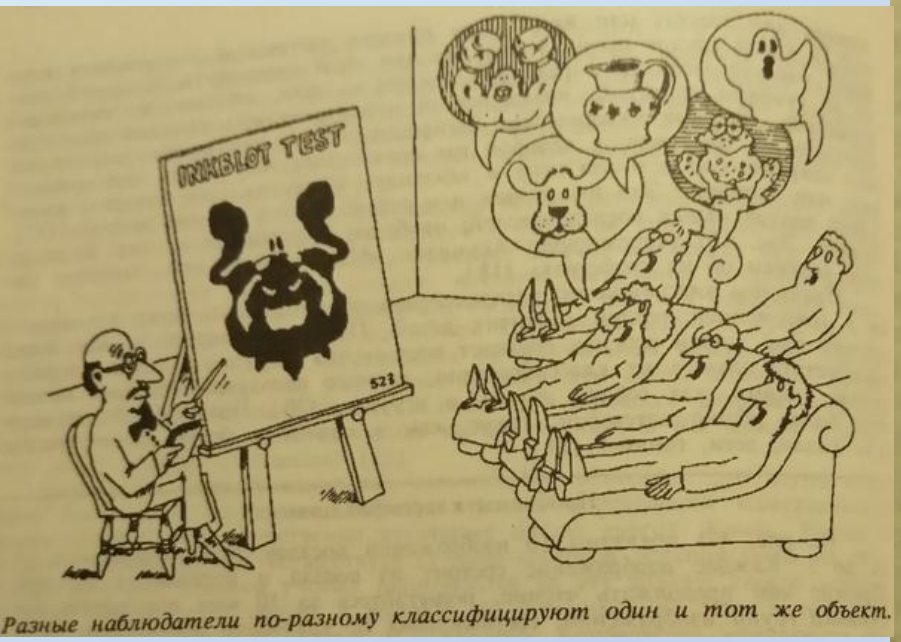
Объектно-ориентированному подходу присуще большинство принципов структурного подхода, а именно: *«разделяй и властвуй», абстрагирование, формализация и непротиворечивость*. Кроме того, выделяются следующие специфичные для него принципы:

1. ***Уникальность.*** *Каждый объект имеет свой адрес в памяти*, и две объектные переменные считаются равными только в том случае, если они указывают на один и тот же адрес.

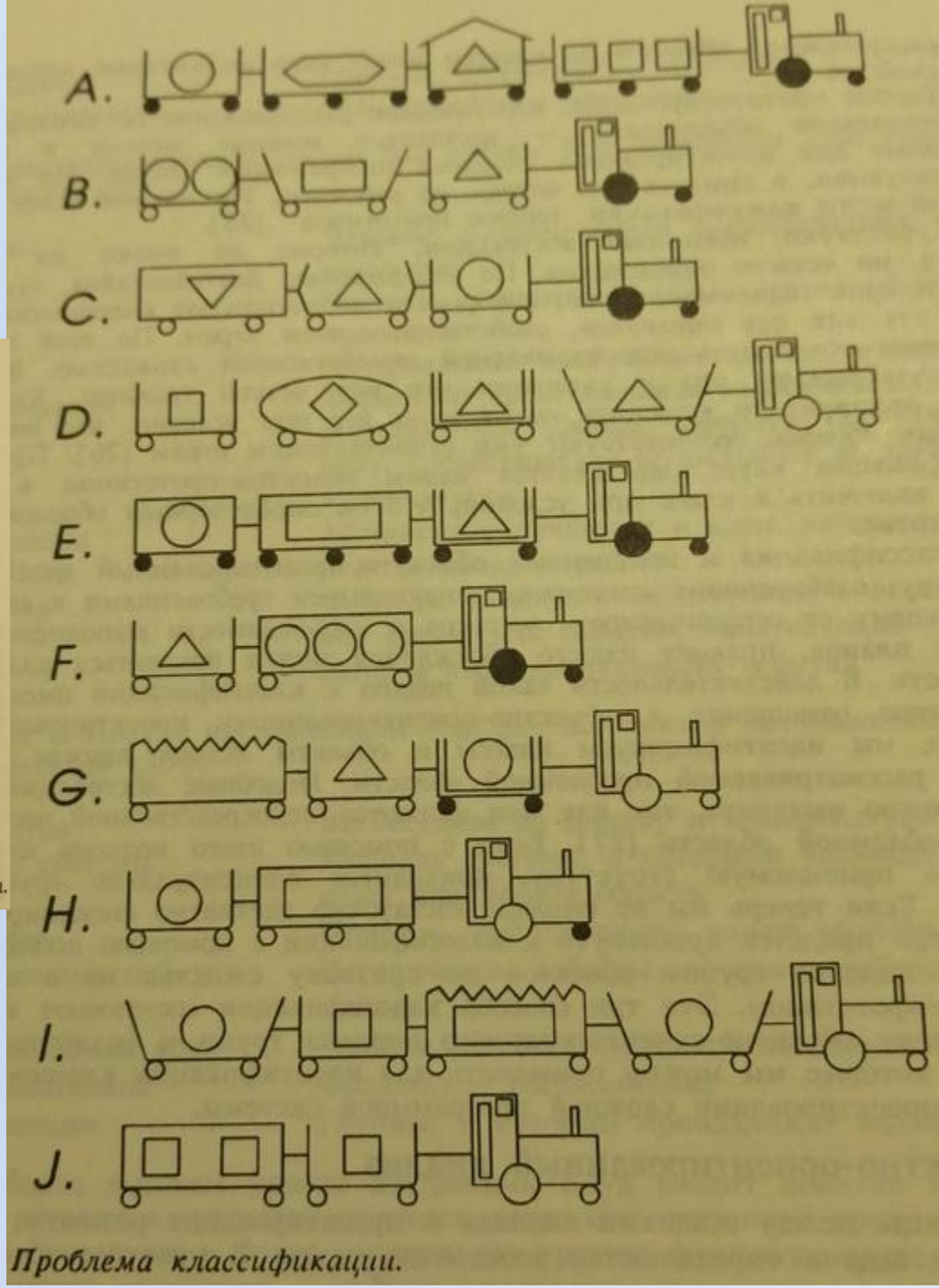
2. ***Классификация.*** Все *объекты объединяются в классы по принципу сходства структуры, поведения и семантики*.

3. ***Инкапсуляция.*** С*овместное «хранение» данных и методов их обработки.* В широком смысле инкапсуляция – это *скрытие реализации за интерфейсом*, т.е. объект обладает внутренней, известной лишь ему структурой и интерфейсом.

Проблемы классификации



Разные наблюдатели по-разному классифицируют один и тот же объект.



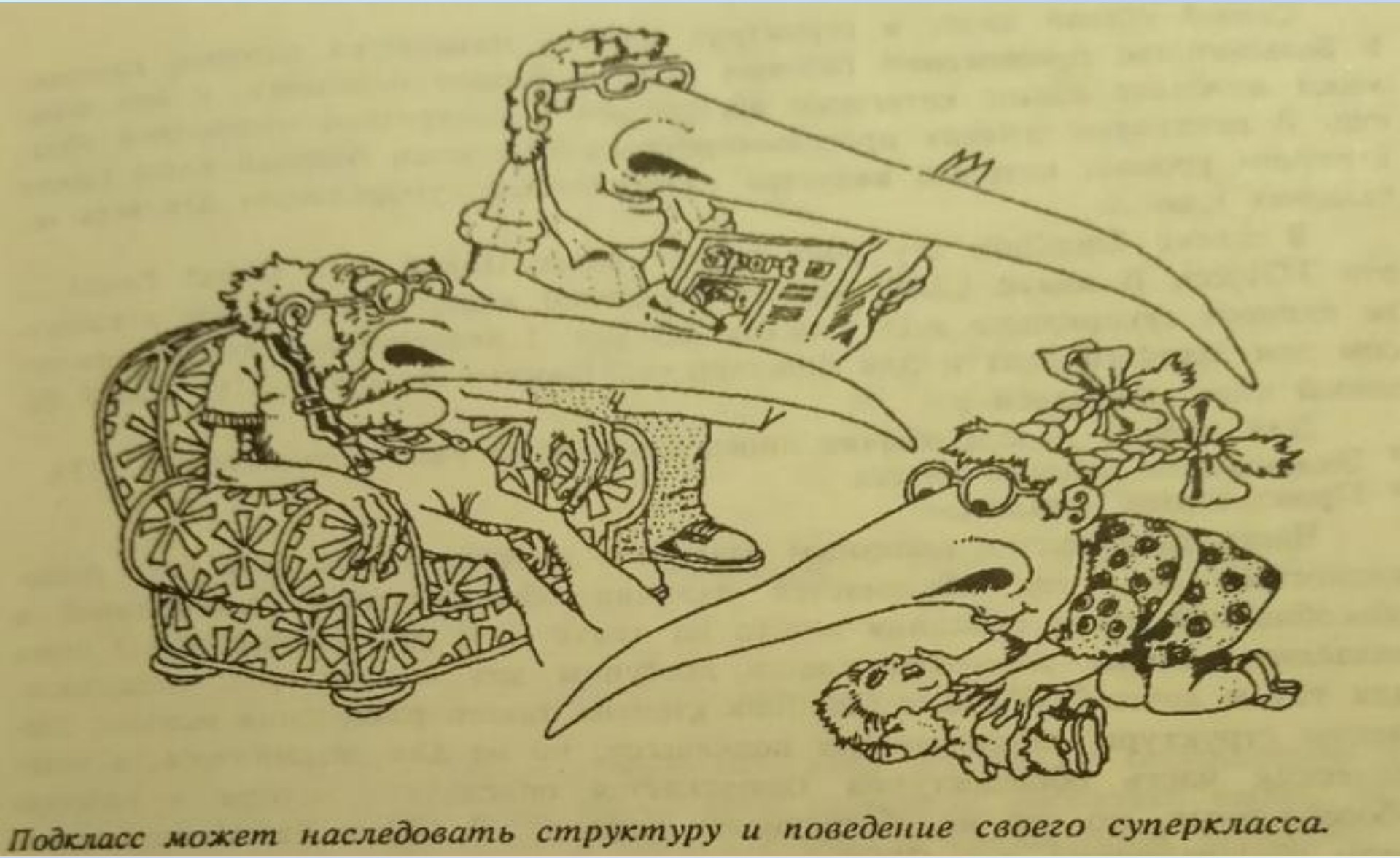
Проблема классификации.

Рисунки: Буч Г. и др. Объектно-ориентированный анализ
М. : «И.Д. Вильямс», 2010

4. Наследование. Классы могут объединяться в иерархии наследования. Структура, поведение и семантика объектов наследуются вниз по иерархии.

5. Полиморфизм. Данный принцип означает возможность доступа к нескольким реализациям через один интерфейс. Способом реализации полиморфизма в объектно-ориентированных языках программирования является механизм переопределения унаследованных методов.

Подклассы могут по иерархии наследовать структуру и поведение своего суперкласса



Подкласс может наследовать структуру и поведение своего суперкласса.

Рисунки: Буч Г. и др. Объектно-ориентированный анализ М. : «И.Д. Вильямс», 2010

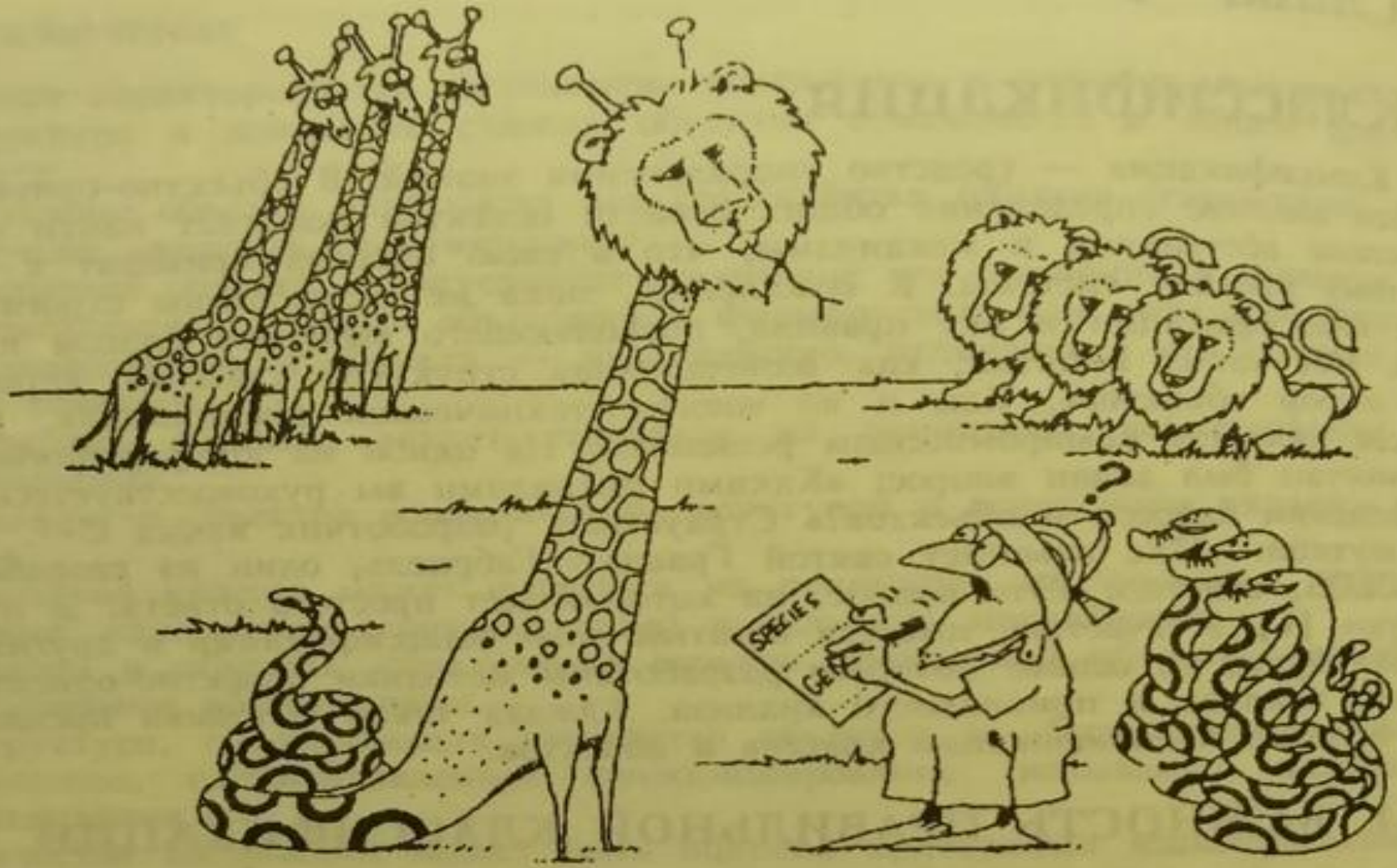
Уникальность

Уникальность - это то, что отличает *объект* от других *объектов*.

В машинном представлении под параметром уникальности объекта чаще всего понимается адрес размещения объекта в памяти. Уникальность объекта состоит в том, что всегда можно определить, указывают две ссылки на один и тот же объект или на разные объекты. При этом два объекта могут во всем быть похожими, их образ в памяти может представляться одинаковыми последовательностями байтов, но, тем не менее, их Identity может быть различна.

*распространенной **ошибкой** является понимание уникальности как имени ссылки на объект. Это неверно, т.к. на один объект может указывать несколько ссылок, и ссылки могут менять свои значения (ссылаться на другие объекты).*

Проблемы классификации



Классификация упорядочивает знания.

Рисунки: Буч Г. и др. Объектно-ориентированный анализ М. : «И.Д. Вильямс», 2010

Классы

Все объекты одного и того же класса описываются одинаковыми наборами атрибутов.

Объединение объектов в классы позволяет рассмотреть задачу в более общей постановке.

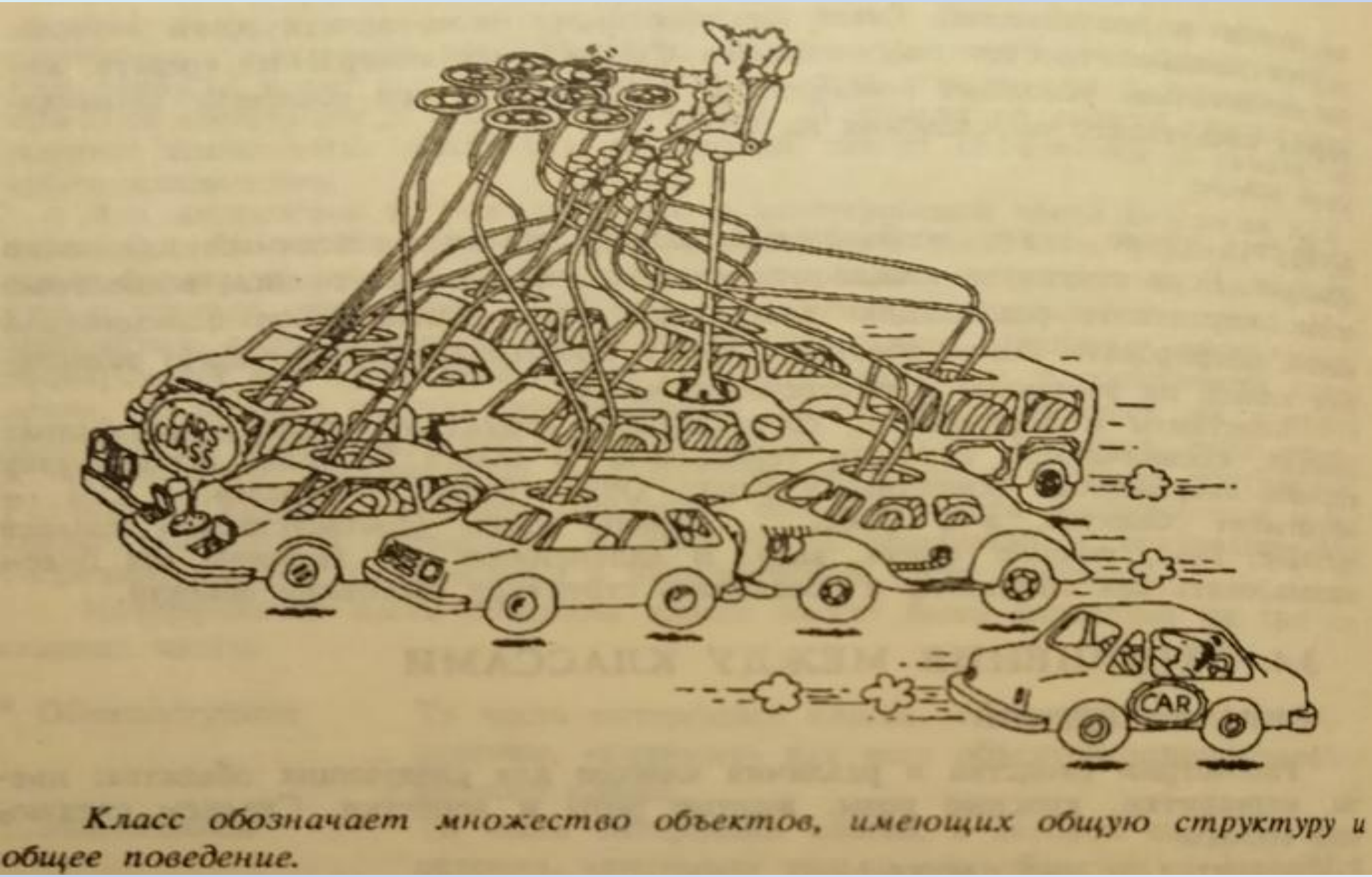
это шаблон поведения объектов определенного типа с заданными параметрами, определяющими состояние. Все экземпляры одного класса (объекты, порожденные от одного класса) имеют один и тот же набор свойств и общее поведение, то есть одинаково реагируют на одинаковые сообщения.



стоимость

металл

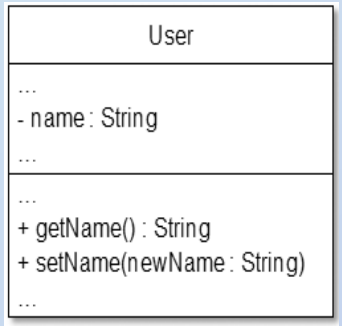
Класс обозначает множество объектов, имеющих общую структуру и общее поведение

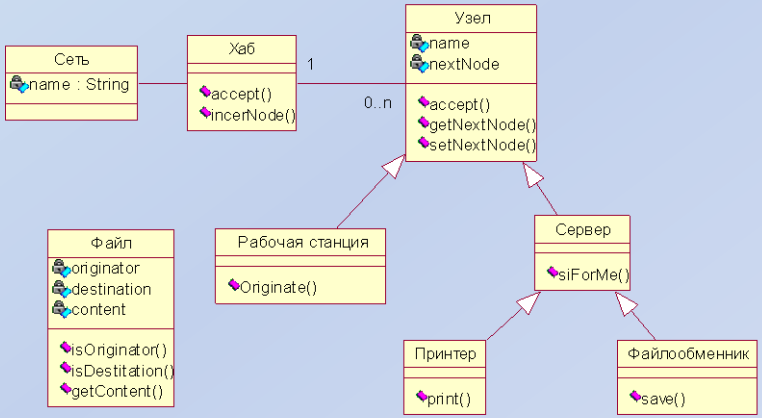


Класс обозначает множество объектов, имеющих общую структуру и общее поведение.

Рисунки: Буч Г. и др. Объектно-ориентированный анализ М. : «И.Д. Вильямс», 2010



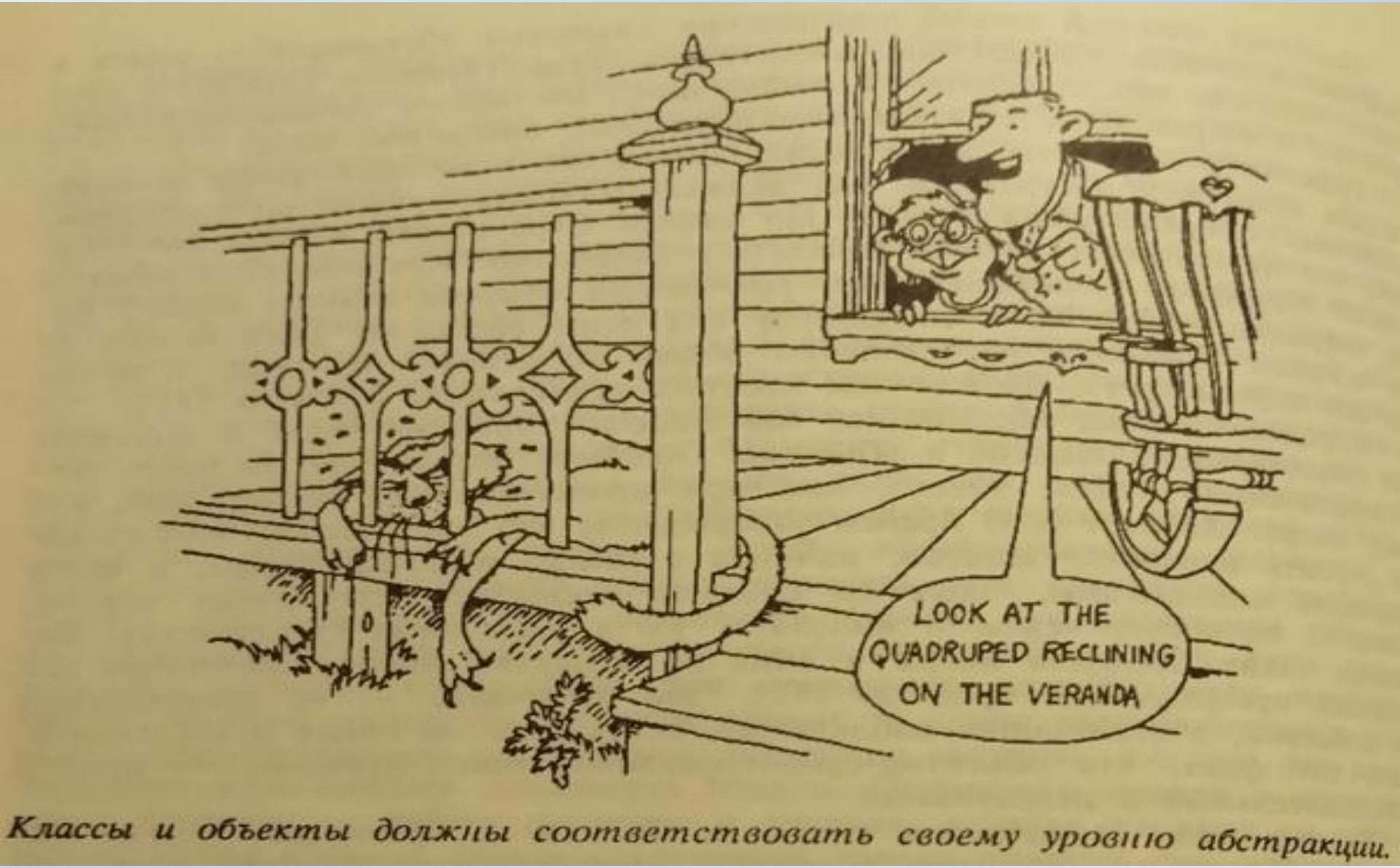




Каждый класс также может иметь специальные методы, которые автоматически вызываются при создании и уничтожении объектов этого класса:

- ***конструктор*** *(constructor) - выполняется при создании объектов ;*
- ***деструктор*** *(destructor) - выполняется при уничтожении объектов.*

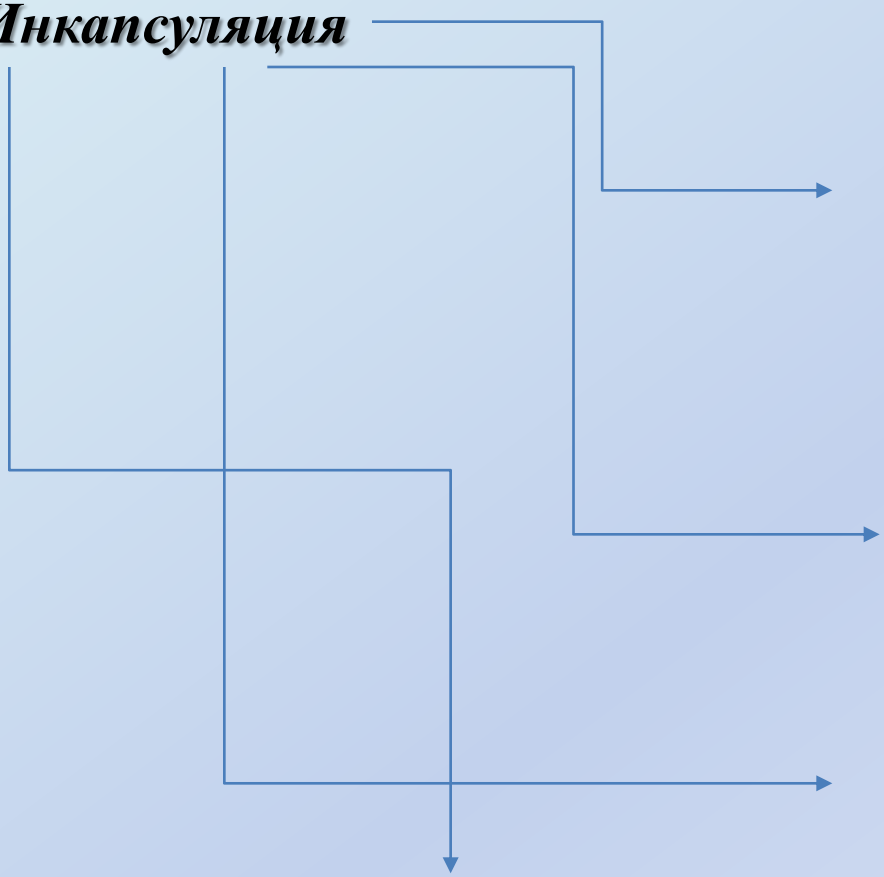
Классы и объекты должны соответствовать своему уровню абстракции



Классы и объекты должны соответствовать своему уровню абстракции.

Рисунки: Буч Г. и др. Объектно-ориентированный анализ М. : «И.Д. Вильямс», 2010

Инкапсуляция



сокрытие реализации класса и отделение его внутреннего представления от внешнего (интерфейса)

При использовании объектно-ориентированного подхода не принято применять прямой доступ к свойствам какого-либо класса из методов других классов. Для доступа к свойствам класса принято задействовать специальные методы этого класса для получения и изменения его свойств.

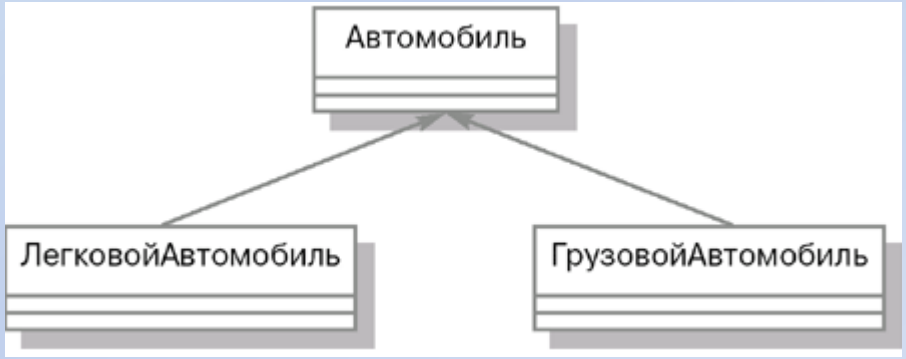
Открытые члены класса составляют внешний интерфейс объекта.

Обеспечение доступа к свойствам класса только через его методы также дает ряд преимуществ. Во-первых, так гораздо проще контролировать корректные значения полей, ведь прямое обращение к свойствам отслеживать невозможно, а значит, им могут присвоить некорректные значения.

Наследование

это отношение между классами, при котором класс использует структуру или поведение другого класса (одиночное наследование), или других (множественное наследование) классов

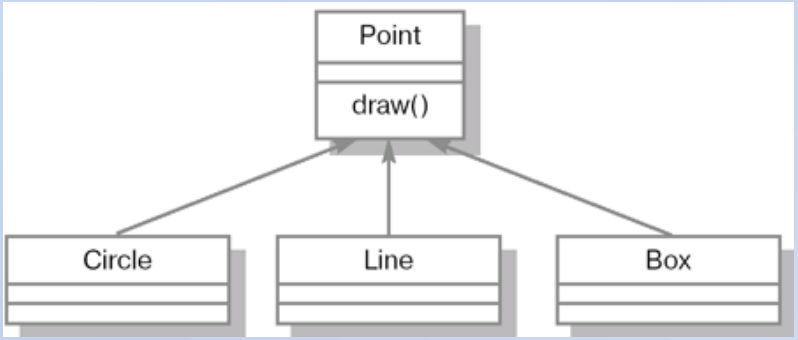
*Наследование вводит иерархию "общее/частное", в которой **подкласс** наследует от одного или нескольких более общих **суперклассов**. Подклассы обычно дополняют или переопределяют унаследованную структуру и поведение.*



Полиморфизм

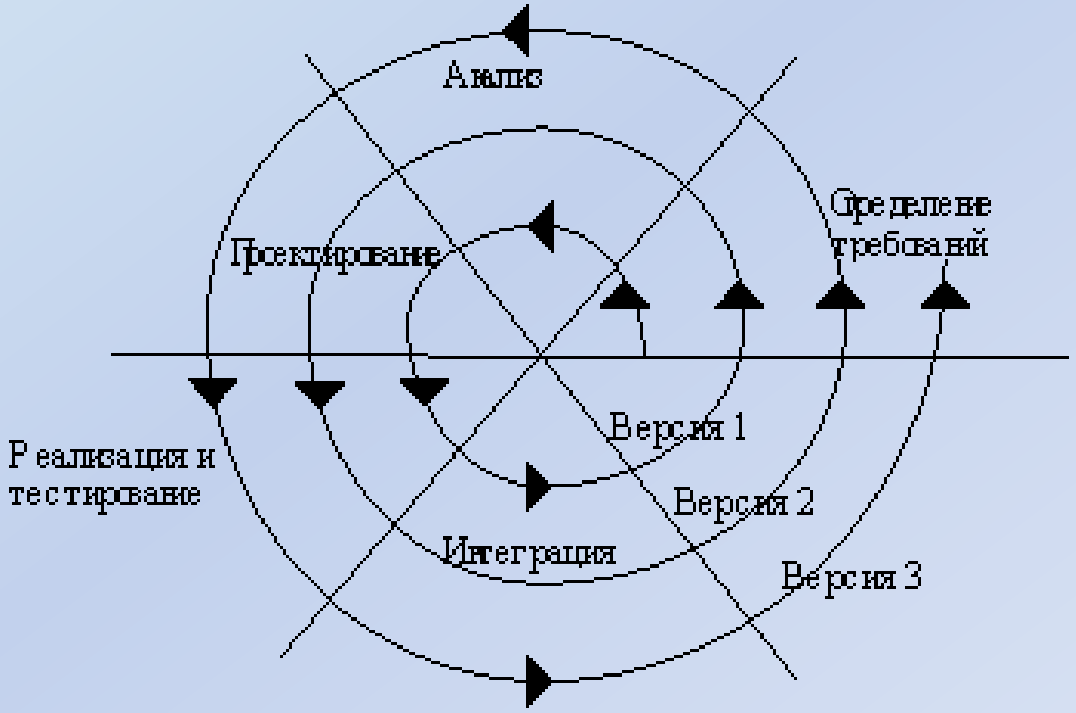
является одним из фундаментальных понятий в объектно-ориентированном программировании наряду с наследованием и инкапсуляцией. Слово " полиморфизм " греческого происхождения и означает "имеющий много форм".

Полиморфизм (polymorphism) - положение теории типов, согласно которому имена (например, переменных) могут обозначать объекты разных (но имеющих общего родителя) классов. Следовательно, любой объект, обозначаемый полиморфным именем, может по-своему реагировать на некий общий набор операций



Модель процесса

Для объектно-ориентированного подхода характерен *спиральный жизненный цикл* системы. В отличие от структурного подхода, объектный подход не использует иерархических структур, в которых определение родителя не имеет смысла без определения всех его потомков.



Вместо этого применяется система объектов, каждый из которых можно анализировать, проектировать, реализовывать и тестировать практически независимо от других. Таким образом, появляется возможность формирования итераций, на каждой из которых выполняются все основные этапы разработки для относительно небольшой группы объектов.

Объектно-ориентированный анализ

В процессе объектно-ориентированного анализа основное внимание уделяется выделению и описанию понятий предметной области, их структуры и связей между ними. Например, в задаче автоматизации библиотеки среди понятий могут присутствовать «библиотека», «издание», «читатель» и др.

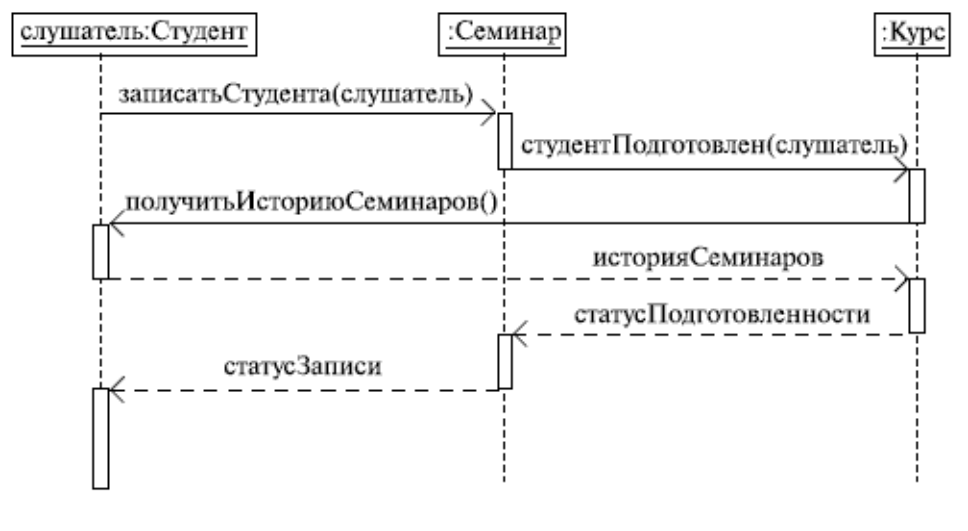
Объектно-ориентированное проектирование

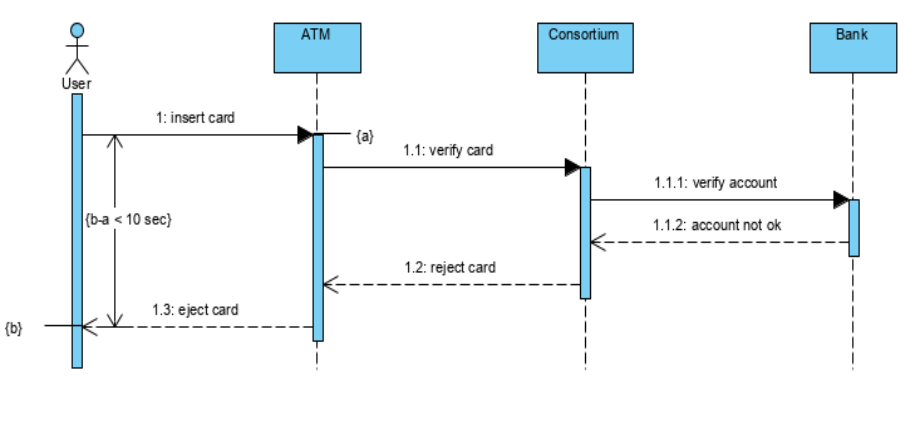
В процессе объектно-ориентированного проектирования определяются логические программные объекты, а также их взаимосвязи и взаимодействие между ними. Эти объекты характеризуются атрибутами и операциями. Например, класс «издание» может содержать атрибут «название» и операцию «выдать».

Диаграмма последовательности *отображает взаимодействие объектов в динамике.*

Диаграмма последовательности относится к диаграммам взаимодействия UML, описывающим поведенческие аспекты системы, но *рассматривает взаимодействие объектов во времени.*

Диаграмма последовательности отображает временные особенности передачи и приема сообщений объектами.





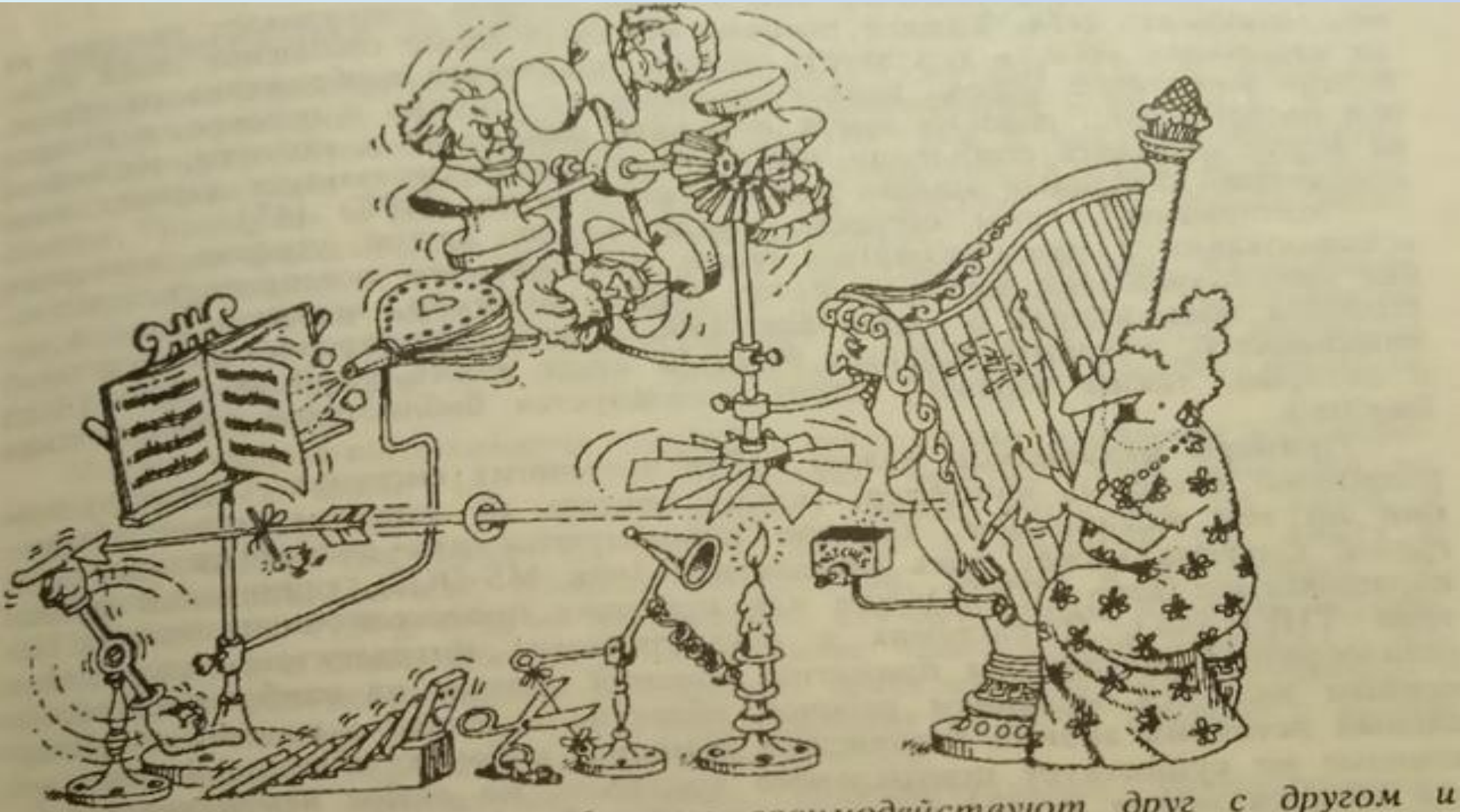
Кооперация (collaboration) служит для обозначения множества взаимодействующих с определенной целью объектов в общем контексте моделируемой системы. Цель самой кооперации состоит в том, чтобы специфицировать особенности реализации отдельных наиболее значимых операций в системе. Кооперация определяет структуру поведения системы в терминах взаимодействия участников этой кооперации.

Кооперация может быть представлена на двух уровнях:

• На уровне спецификации – показывает роли классификаторов и роли ассоциаций в рассматриваемом взаимодействии.

• На уровне примеров – указывает экземпляры и связи, образующие отдельные роли в кооперации.

Механизмы – средство взаимодействия объектов



С помощью механизмов объекты взаимодействуют друг с другом и обеспечивают таким образом более высокий уровень поведения системы.

Рисунки: Буч Г. и др. Объектно-ориентированный анализ М. : «И.Д. Вильямс», 2010

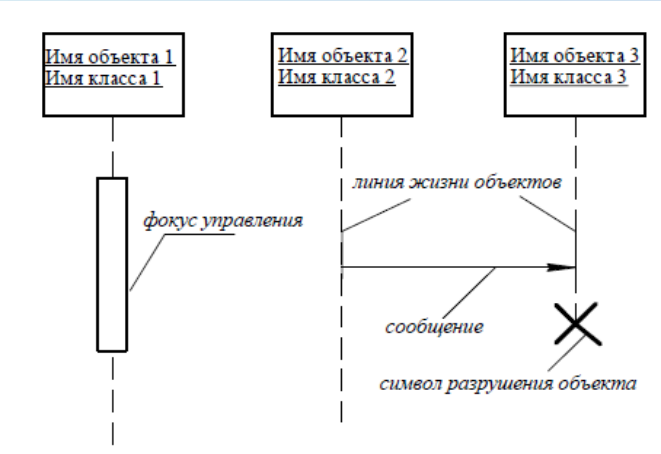
Сообщения могут быть следующих видов:

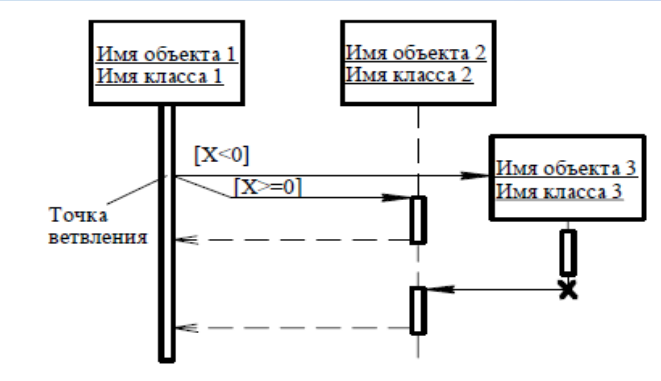
– синхронное сообщение (англ. synchronous message). Клиент посылает сообщение серверу и ждет, пока тот примет и обработает сообщение. Как правило, один объект передает синхронное сообщение второму, второй – третьему и т.д., образуя вложенный поток сообщений. В любом случае клиент, инициирующий поток сообщений, должен дождаться его завершения, т.е. возврата управления. Это самый распространенный тип сообщений;

– асинхронное сообщение (англ. asynchronous message). Клиент посылает сообщение серверу и, не дожидаясь ответа, продолжает выполнять следующие операции;

– возвращающее сообщение (англ. reply message), обозначающее возврат значения или управления от сервера обратно клиенту. Стрелки этого вида зачастую отсутствуют на диаграммах, поскольку неявно предполагается их существование после окончания процесса выполнения операции.

В отдельных случаях объект может посылать сообщения самому себе (вызывать собственные методы), инициируя так называемые рефлексивные сообщения.





При разработке диаграмм следует придерживаться следующих правил:

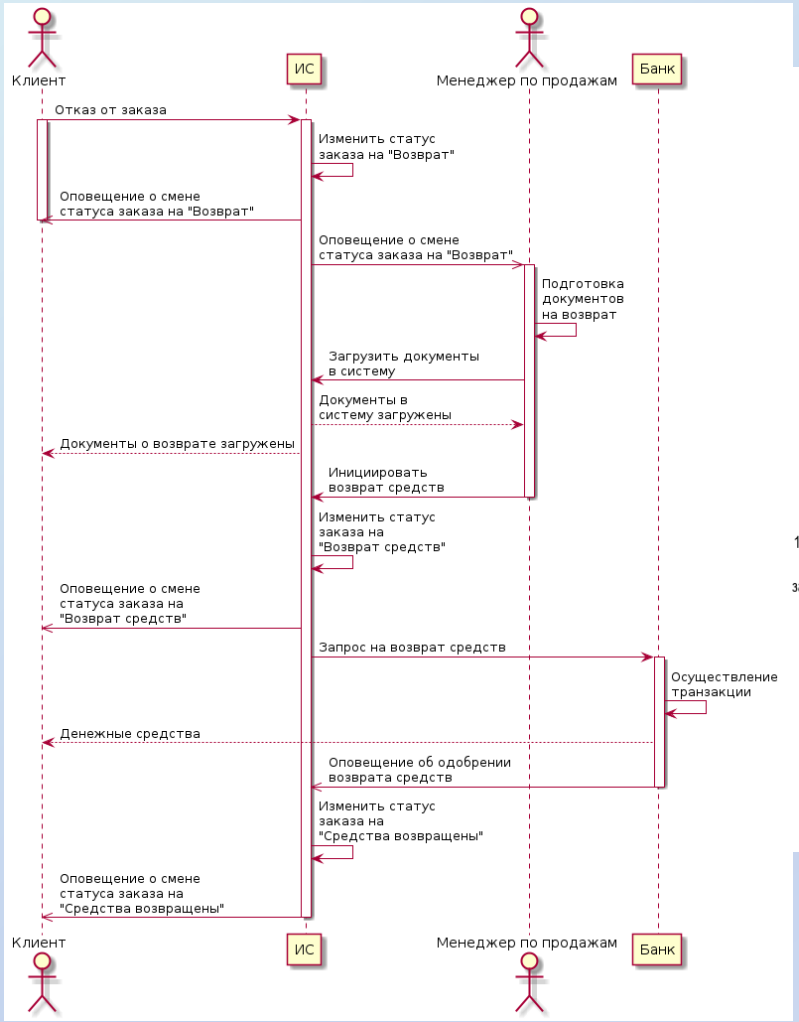
1. Для выбранного варианта использования необходимо перенести с диаграммы классов анализа все участвующие в нем классы, а с диаграммы вариантов использования – актеров.

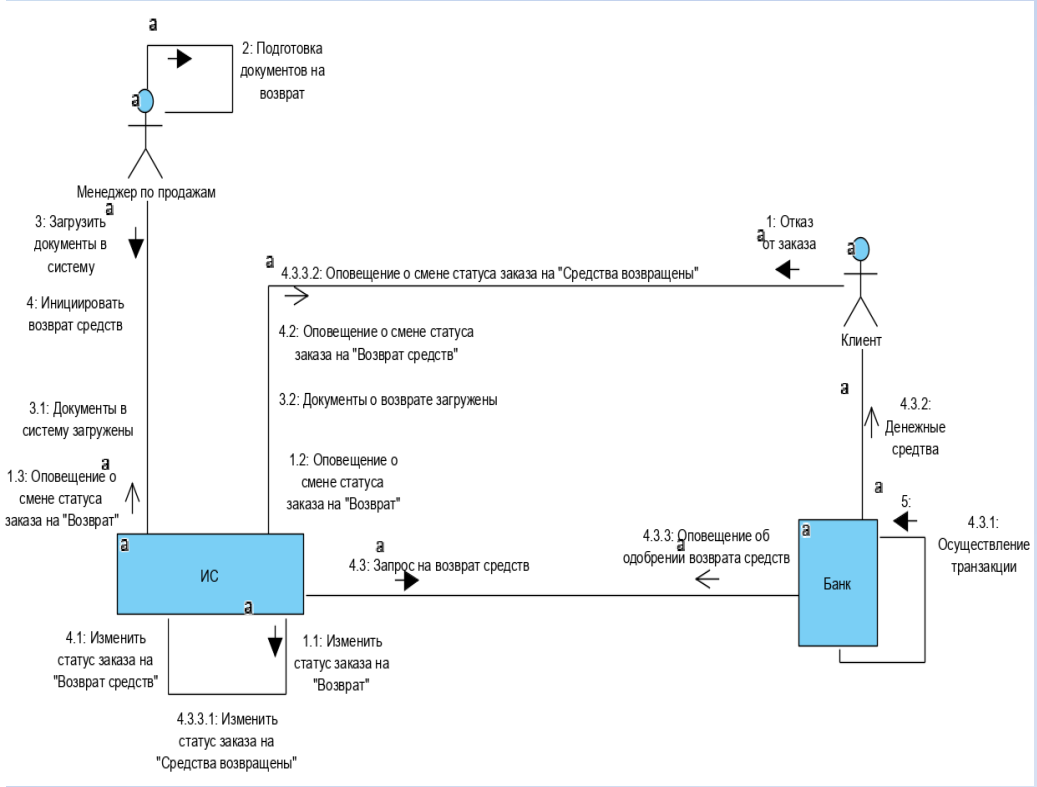
2. На диаграмме коммуникации между классами следует отобразить ассоциации, перенесенные с диаграммы классов анализа, а также добавить ассоциации, связывающие актеров с граничными классами.

3. На стадии анализа имена сообщениям можно давать произвольно или в виде стереотипов. В дальнейшем (в модели проектирования) имена сообщений должны соответствовать методам классов.

4. Имена сущностей на диаграммах (экземпляры актеров и объекты) должны быть подчеркнуты и обозначены соответствующим образом.

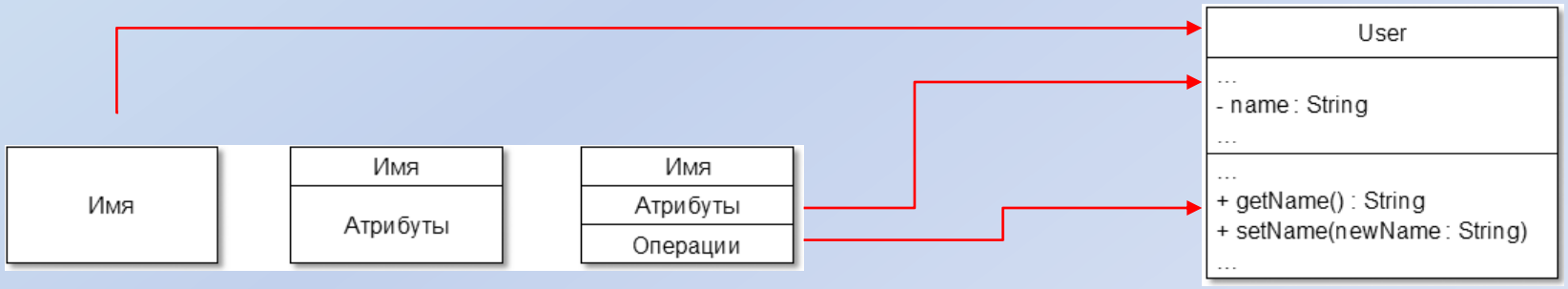
5. На диаграммах последовательности символ уничтожения объектов следует задавать только для тех объектов, которые во время взаимодействия действительно уничтожаются.


Клиент
ИС
Менеджер по продажам
Банк
Отказ от заказа
Изменить статус заказа на "Возврат"
Оповещение о смене статуса заказа на "Возврат"
Оповещение о смене статуса заказа на "Возврат"
Подготовка документов на возврат
Загрузить документы в систему
Документы в систему загружены
Документы о возврате загружены
Инициировать возврат средств
Изменить статус заказа на "Возврат средств"
Оповещение о смене статуса заказа на "Возврат средств"
Запрос на возврат средств
Осуществление транзакции
Денежные средства
Оповещение об одобрении возврата средств
Изменить статус заказа на "Средства возвращены"
Оповещение о смене статуса заказа на "Средства возвращены"
Клиент
ИС
Менеджер по продажам
Банк


2: Подготовка документов на возврат
Менеджер по продажам
3: Загрузить документы в систему
4: Инициировать возврат средств
1: Отказ от заказа
Клиент
4.3.3.2: Оповещение о смене статуса заказа на "Средства возвращены"
4.2: Оповещение о смене статуса заказа на "Возврат средств"
3.2: Документы о возврате загружены
4.3.2: Денежные средства
3.1: Документы в систему загружены
1.2: Оповещение о смене статуса заказа на "Возврат"
1.3: Оповещение о смене статуса заказа на "Возврат"
5:
4.3.1: Осуществление транзакции
ИС
4.3: Запрос на возврат средств
4.3.3: Оповещение об одобрении возврата средств
Банк
4.1: Изменить статус заказа на "Возврат средств"
1.1: Изменить статус заказа на "Возврат"
4.3.3.1: Изменить статус заказа на "Средства возвращены"

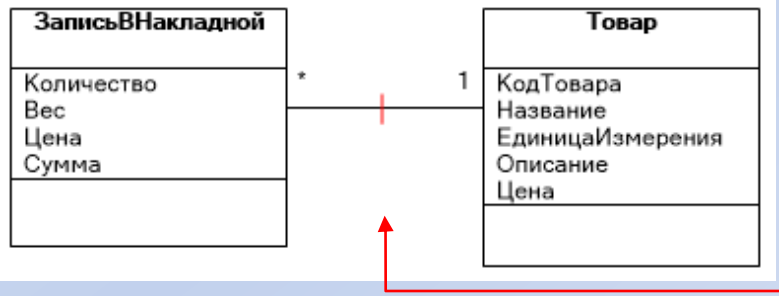
Назначение и состав диаграммы классов

Диаграмма классов определяет типы классов системы и различного рода статические связи, которые существуют между ними. На диаграммах классов изображаются также атрибуты классов, операции классов и ограничения, которые накладываются на связи между классами. Вид и интерпретация диаграммы классов существенно зависит от точки зрения (уровня абстракции): классы могут представлять ***сущности предметной области*** (в процессе анализа) или ***элементы программной системы*** (в процессах проектирования и реализации).



Видимость отображается с помощью следующих символов:
- "+" – общедоступный атрибут (англ. public) – доступен для чтения и модификации из объектов любого класса;
- "#" – защищенный атрибут (англ. protected) – доступен только объектам описываемого класса и его потомкам при наследовании;
- "–" – закрытый атрибут (англ. private) – доступен только объектам описываемого класса;
- "~" – пакетный атрибут (англ. package) – доступен только объектам классов, входящих в тот же пакет.

Видимость *(англ. visibility) характеризует возможность чтения и модификации значения атрибута объекта описываемого класса, из объектов других классов. Модификация значения возможна лишь при условии, что атрибут не является константой.*



Стереотип классов *– это механизм, позволяющий разделять классы на категории. В языке UML определены три основных стереотипа классов: entity, boundary, control.*

В примере на рисунке каждый ***Товар*** *имеет сколь угодно* ***Записей*** *в накладной, но каждая* ***Запись*** *в накладной обязательно один* ***Товар****.*

При разработке диаграммы следует придерживаться следующих правил:

1. За основу диаграммы классов при ее разработке берется диаграмма классов анализа.

2. Для классов должны быть определены и специфицированы все атрибуты и методы. Их спецификация, как правило, выполняется с учетом выбранного языка программирования.

3. При определении методов рекомендуется использовать сообщения с ранее разработанных диаграмм последовательности и коммуникации.

4. Детальное проектирование граничных классов, как правило, не требуется. Большинство современных средств разработки поддерживает визуальную разработку интерфейса системы – меню, диалоговых форм, элементов диалоговых окон, панелей инструментов и т. д. В качестве исходных данных для их проектирования служат прототипы пользовательских интерфейсов. В связи с этим при проектировании таких классов основное внимание следует уделять особенностям отображения информации и специфичным операциям, которые возникают при диалоге пользователя с системой. Граничные классы, определяющие интерфейс взаимодействия с другими системами, требуют детального проектирования.

5. Для проектирования классов-сущностей можно применять подходы, используемые при проектировании БД, особенно в том случае, если данные будут храниться в таблицах БД. Если представление данных в БД и классах отличается друг от друга и в качестве хранилища информации будет применяться реляционная база данных, то рекомендуется разработать отдельную диаграмму классов, описывающую состав и структуру БД.

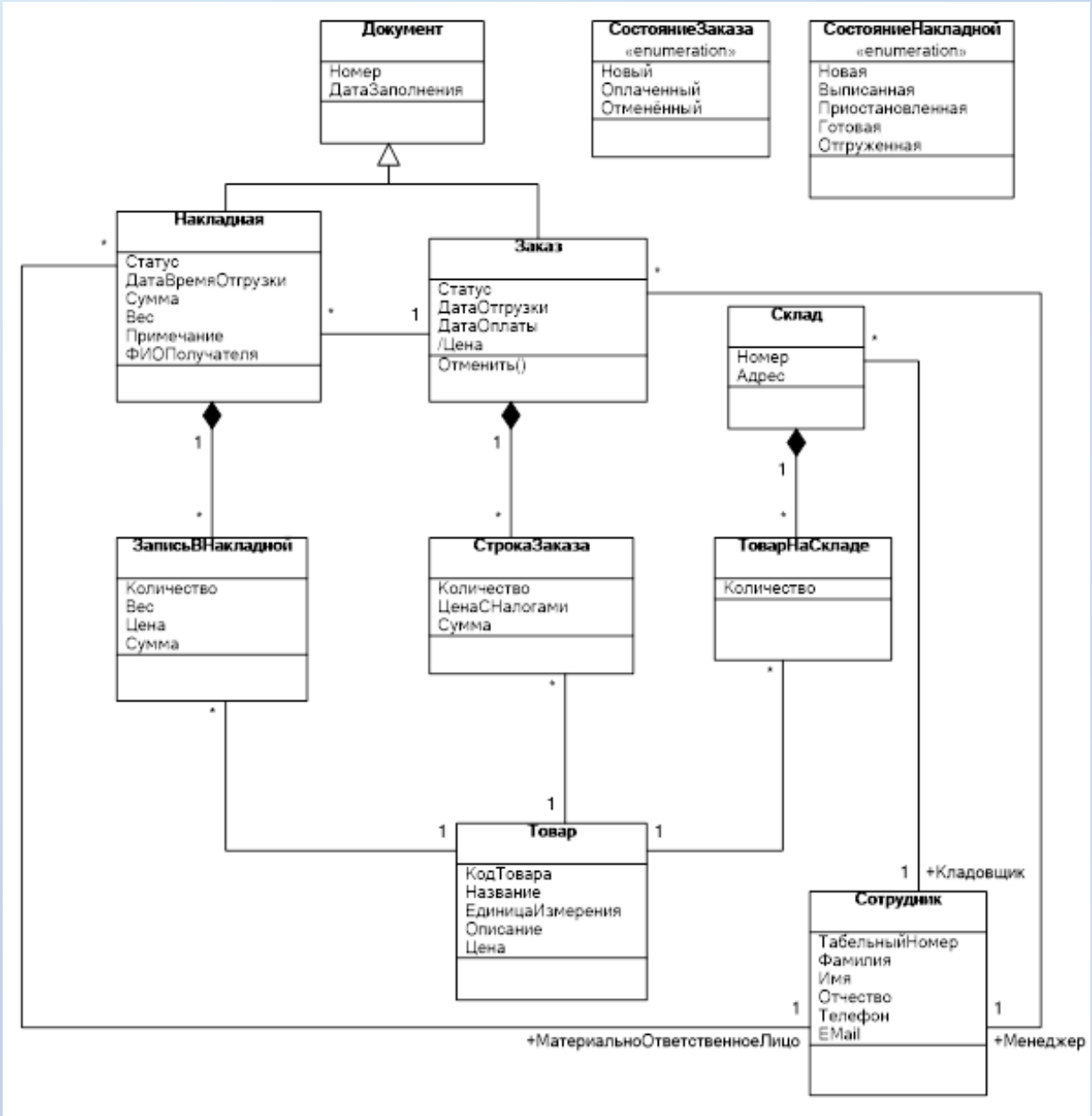
6. Несмотря на то, что каждому объекту при выполнении программы автоматически назначается уникальный идентификатор, рекомендуется для классов-сущностей явно определять атрибуты, хранящие значения первичного ключа.

7. В отличие от реляционных БД поощряется использование в классах многозначных атрибутов в виде массивов, множеств, списков и т. д.

8. Управляющие классы следует проектировать только в случаях крайней необходимости – управления сложным взаимодействием объектов, реализации сложной бизнес-логики и вычислений, контроля целостности объектов и т. п. В противном случае функциональность этого класса лучше распределить между соответствующими граничными классами и классами-сущностями.

В третьей секции указывается перечень операций класса. Можно выделить шесть основных типов операций:

1. ***конструктор*** *– метод, создающий и инициализирующий объект. В Java имя конструктора совпадает с именем класса;*
2. ***деструктор*** *– метод, уничтожающий объект. В некоторых языках программирования (в частности в Java) определение деструкторов не требуется, так как очистка памяти от неиспользуемых объектов (сборка мусора) выполняется автоматически;*
3. ***модификатор*** *– метод, который изменяет состояние объекта (значения атрибутов). Имена модификаторов начинаются, как правило, со слова set (англ. – установить). Например, установить атрибуту Name новое значение setName (newName : String);*
4. ***селектор*** *– метод, который может только считывать значения атрибутов объекта, но не изменяет их. Имена селекторов начинаются, как правило, со слов get (англ. – получить) или is при возврате логического результата. Например, считать значение атрибута Name – getName() или определить видимость на экране элемента графического интерфейса – isVisible();*
5. ***итератор –*** *метод, позволяющий организовать доступ к элементам объекта. Например, для объекта, представляющего собой множество Set или список List, это могут быть методы перехода к первому элементу first(), следующему next(), предыдущему previous() и т. п.;*
6. ***событие*** *– метод, запускаемый на выполнение автоматически при соблюдении определенных условий.*


Документ
Номер
ДатаЗаполнения
СостояниеЗаказа
«enumeration»
Новый
Оплаченный
Отменённый
СостояниеНакладной
«enumeration»
Новая
Выписанная
Приостановленная
Готовая
Отгруженная
Накладная
Статус
ДатаВремяОтгрузки
Сумма
Вес
Примечание
ФИОПолучателя
Заказ
Статус
ДатаОтгрузки
ДатаОплаты
/Цена
Отменить()
Склад
Номер
Адрес
ЗаписьВНакладной
Количество
Вес
Цена
Сумма
СтрокаЗаказа
Количество
ЦенаСНалогами
Сумма
ТоварНаСкладе
Количество
Товар
КодТовара
Название
ЕдиницаИзмерения
Описание
Цена
Сотрудник
ТабельныйНомер
Фамилия
Имя
Отчество
Телефон
EMail
+Кладовщик
+МатериальноОтветственноеЛицо
+Менеджер

Лекция 4
Основы унифицированного языка моделирования

UML (Unified Model Language)

ОСНОВНЫЕ ДИАГРАММЫ

Унифицированный язык объектно-ориентированного моделирования, синтаксис и семантика основных объектов UML, понятия: модель, диаграмма. Диаграмма прецедентов

UML (англ. *Unified Modeling Language* — унифицированный язык моделирования)

- **Язык UML** представляет собой общецелевой язык визуального моделирования, который разработан для спецификации, визуализации, проектирования и документирования компонентов программного обеспечения, бизнес-процессов и других систем. Язык UML может быть эффективно использован для построения концептуальных, логических и графических моделей сложных систем самого различного целевого назначения. Язык UML предлагает набор инструментальных средств, позволяющих проводить всесторонний анализ сложных ИС как с технической точки зрения, так и с точки зрения потребностей бизнеса.

UML упрощает процесс проектирования ИС, снижает его стоимость и повышает эффективность.

UML (Unified Model Language) – семейство графических нотаций в основе которого лежит единая метамодель.

Создан, чтобы помогать в описании и проектировании информационных систем любой сложности.

Считается графическим языком моделирования.

Используется в качестве инструмента обратного и прямого инжиниринга в CASE-средствах разработки информационных систем.

Группа OMG и стандартизация UML

OMG – организация, занимающаяся созданием методологии на основе существующих стандартов.

С момента появления первой версии UML, методологии UML создавались в OMG.



OMG Консорциум, занимающийся разработкой и продвижением объектно-ориентированных технологий и стандартов.

Это некоммерческое объединение, разрабатывающее стандарты для создания интероперабельных, то есть платформо - независимых, приложений на уровне предприятия.

С консорциумом сотрудничает около 800 организаций - крупнейших производителей программного обеспечения.

Существует три основных ***режима*** использования UML диаграмм:

- режим эскиза
- режим проектирования
- режим языка программирования

У режимов есть две опции:

прямой (диаграммы до кода)

и обратный (диаграммы на основании кода) ***инжиниринг***.

МОДЕЛЬ UML И ЕЕ ЭЛЕМЕНТЫ

Модель UML (UML model) — это совокупность конечного множества конструкций языка, главные из которых — это сущности и отношения между ними.

Сами сущности и отношения модели являются экземплярами метаклассов метамодели.

Сущности

Для удобства обзора сущности в UML можно подразделить на четыре группы:

- структурные;
- поведенческие;
- группирующие;
- аннотационные.

Основные понятия языка UML.

- **Объект**— сущность реального мира или концептуальная сущность, либо программная единица, состоящая из атрибутов (полей) и методов для их обработки. Характеризуется состоянием, поведением, и уникальностью. Состояние определяется атрибутами, а поведение – методами.
- **Класс** – описание множества объектов с общими атрибутами, определяющими состояние, и операциями, определяющими поведение. Класс изображается прямоугольником, разделенный на три секции
  В первом приводится имя класса, во втором – список атрибутов, в третьем – список методов. **Ассоциация** – наиболее общий тип отношений первого рода, связанных с передачей сообщений ассоциированным классам путем делегирования

Структурные сущности UML

Структурные сущности, предназначены для описания структуры, к ним обычно относят:

Объект (object) — сущность, обладающая уникальностью и инкапсулирующая в себе состояние и поведение.

Класс (class) — описание множества объектов с общими атрибутами, определяющими состояние, и операциями, определяющими поведение.

Интерфейс (interface) — именованное множество операций, определяющее набор услуг, которые могут быть запрошены потребителем и предоставлены поставщиком услуг.

Кооперация (collaboration) — совокупность объектов, которые взаимодействуют для достижения некоторой цели.

Действующее лицо (actor) — сущность, находящаяся вне моделируемой системы и непосредственно взаимодействующая с ней.

Компонент (component) — модульная часть системы с четко определенным набором требуемых и предоставляемых интерфейсов.

Артефакт (artifact) — элемент информации, который используется или порождается в процессе разработки программного обеспечения. Иначе, артефакт — это физическая единица реализации, получаемая из элемента модели (например, класса или компонента).

Узел (node) — вычислительный ресурс, на котором размещаются и при необходимости выполняются артефакты.

Поведенческие сущности UML

Поведенческие сущности предназначены для описания поведения.

Основных поведенческих сущностей всего две: состояние и действие

(точнее, две с половиной, потому что иногда употребляется еще и деятельность, которую можно рассматривать как особый случай состояния).

Состояние (state) — период в жизненном цикле объекта, находясь в котором объект удовлетворяет некоторому условию и осуществляет собственную деятельность или ожидает наступления некоторого события.

Деятельность (activity) можно считать частным случаем состояния, который характеризуется Продолжительными (по времени) не атомарными вычислениями.

Действие (action) — примитивное атомарное вычисление.

Это только очевидная часть айсберга поведенческих сущностей: состояния бывают разные.

Также, при моделировании поведения используется еще ряд вспомогательных сущностей, которые сосуществуют только вместе с указанными основными.

Несколько особняком стоит сущность — вариант использования (use case), которой присущи Как структурные, так и поведенческие аспекты.

Группирующие и аннотационные сущности UML

Вариант использования (use case) — множество сценариев, объединенных по некоторому Критерию и описывающих последовательности производимых системой действий, доставляющих значимый для некоторого действующего лица результат.

Приведенная классификация не является исчерпывающей.

У каждой из этих сущностей есть различные частные случаи и вариации

Группирующая сущность в UML одна — пакет — универсальная.

Пакет (package) — группа элементов модели (в том числе пакетов).

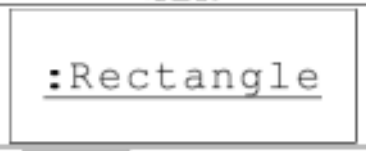
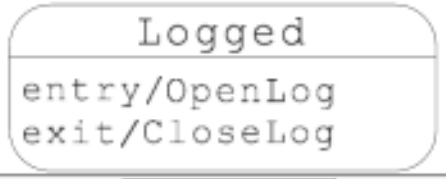
Аннотационная сущность тоже одна — примечание (comment) — зато в нее можно поместить все что угодно, так как содержание примечания UML не ограничивает.

Нотация основных сущностей UML

Нотация основных сущностей

Название	Графическая нотация
Артефакт	«artifact» Requirement Specification; «library» QT
Вариант использования	Make Order; Make Order
Действующее лицо	Customer; Customer
Деятельность и действие	Display main menu; a=15
Интерфейс	IAudio; «interface» IAudio
Класс	Product; Order {abstract}
Компонент	DataBase; «component» DataBase

Нотация основных сущностей UML

Название	Графическая нотация
Кооперация	
Объект	
Пакет	
Примечание	
Состояние	
Узел	

Отношения в UML

В UML используются четыре основных типа отношений:

- зависимость (dependency);

- ассоциация (association);

- обобщение (generalization);

- реализация (realization).

Перечисленные типы отношений являются основными, но возможны также различные их вариации и дополнительные отношения.

Отношение зависимости в UML

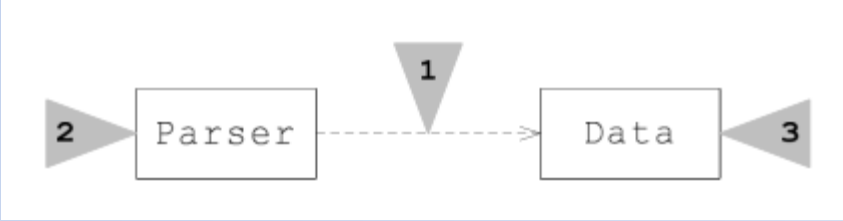
Зависимость — это наиболее общий тип отношения между двумя сущностями.

Отношение зависимости указывает на то, что изменение независимой сущности каким-то образом влияет на зависимую сущность.

Графически отношение зависимости изображается в виде **пунктирной линии** со стрелкой (1), направленной от зависимой сущности (2) к независимой (3).

Как правило, семантика Конкретной зависимости уточняется в модели с помощью дополнительной информации.

Например, зависимость со стереотипом «use» означает, что зависимая сущность использует (скажем, вызывает операцию) независимую сущность.



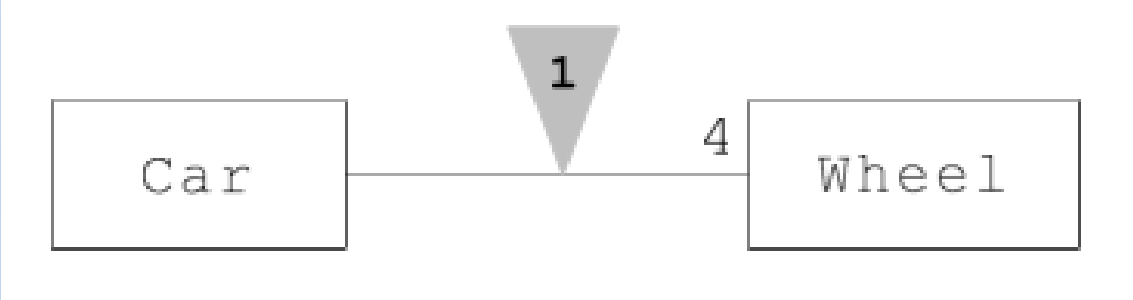
Отношение зависимости в UML

Ассоциация — это наиболее часто используемый тип отношения между сущностями. Отношение ассоциации имеет место, если одна сущность непосредственно связана с другой (или с другими — ассоциация может быть не только бинарной).

Графически ассоциация изображается в виде **сплошной линии** с различными дополнениями, соединяющей связанные сущности.

На программном уровне непосредственная связь может быть реализована различным образом, главное, что ассоциированные сущности знают друг о друге.

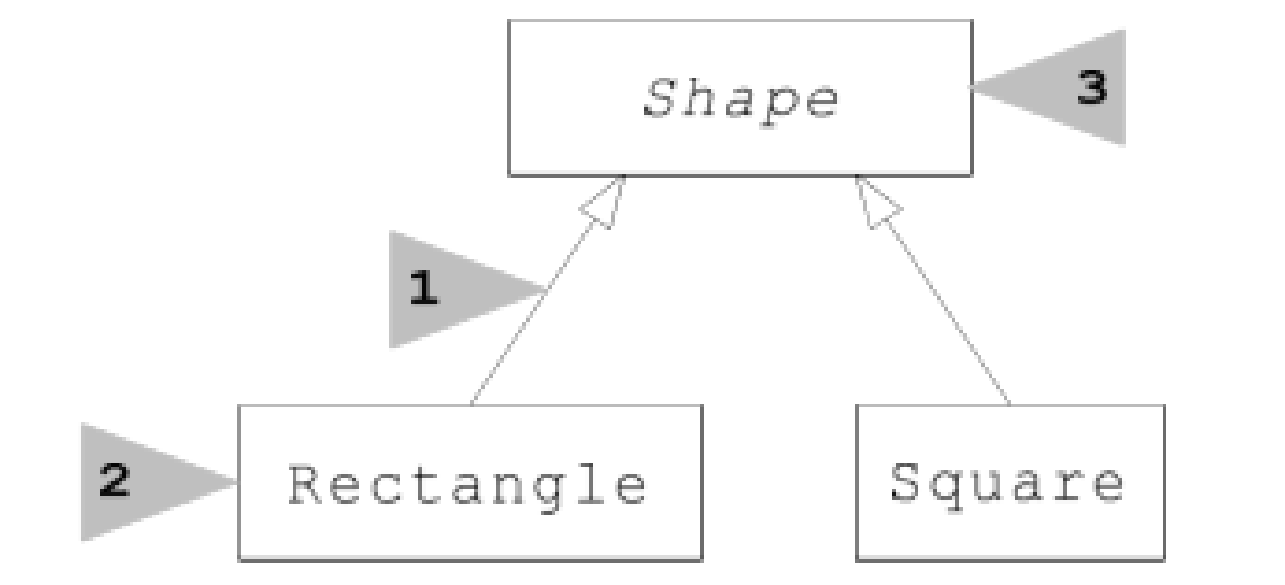
Например, отношение часть–целое является частным случаем ассоциации и называется отношением агрегации.



Отношение обобщения в UML

Обобщение — это отношение между двумя сущностями, одна их которых является частным (специализированным) случаем другой.

Графически обобщение изображается в виде **линии с треугольной незакрашенной стрелкой на конце** (1), направленной от частного (2) (подкласса) к общему (3) (суперклассу)

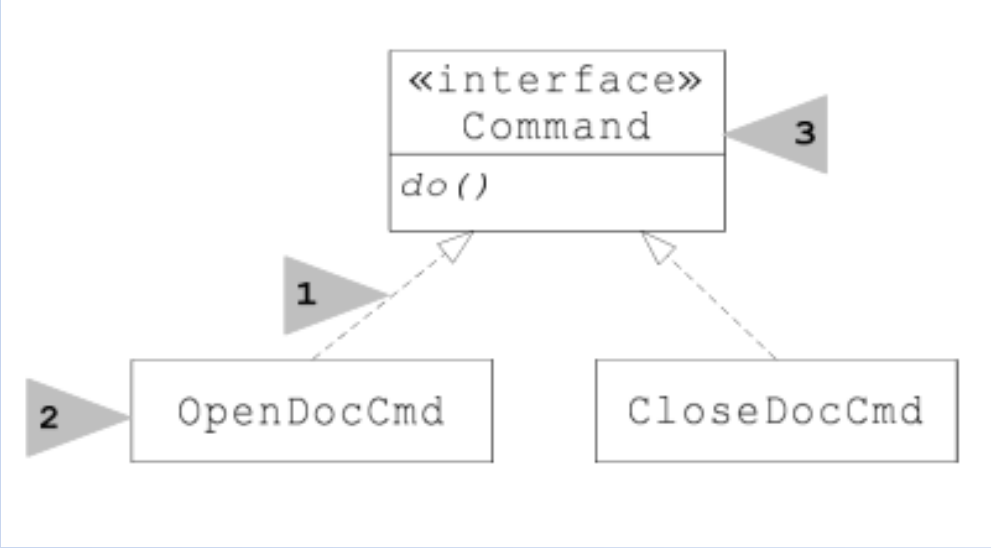


Отношение реализации в UML

Отношение **реализации** используется несколько реже, чем предыдущие три типа отношений, поскольку часто подразумеваются по умолчанию.

Отношение реализации указывает, что одна сущность является реализацией другой.

Графически реализация изображается в виде **пунктирной линии с треугольной незакрашенной стрелкой на конце** (1), направленной от реализующей сущности (2) к реализуемой (3)



Диаграммы в UML

Диаграммы UML есть та основная накладываемая на модель структура, которая облегчает создание и использование модели.

Диаграмм (diagram) — это графическое представление некоторой части графа модели.

Вообще говоря, в диаграмму можно было бы включить любые (допустимые) комбинации сущностей и отношений, но произвол в этом вопросе затруднил бы понимание моделей.

Поэтому авторы UML определили набор рекомендуемых к использованию типов диаграмм, которые получили название канонических типов диаграмм.

Заметим, что помимо сущностей и отношений на диаграмме присутствует другие элементы модели, которые мы также будем называть конструкциями языка.

Это тексты, которые могут быть написаны внутри фигур сущностей или рядом с линиями отношений, рамки диаграмм и их фрагментов, значки, присоединяемые к линиям или помещаемые внутрь фигур.

Эти элементы не только помогают представить модель в более наглядной форме, но подчас несут значительную смысловую нагрузку.

Классификация диаграмм в UML

В UML 1 всего определено 9 канонических типов диаграмм.

Ниже перечислены их названия, принятые в данном учебном пособии

(в других источниках есть отличия).

- Диаграмма использования (Use Case diagram).
- Диаграмма классов (Class diagram).
- Диаграмма объектов (Object diagram).
- Диаграмма состояний (State chart diagram).
- Диаграмма деятельности (Activity diagram).
- Диаграмма последовательности (Sequence diagram).
- Диаграмма кооперации (Collaboration diagram).
- Диаграмма компонентов (Component diagram).
- Диаграмма размещения (Deployment diagram).

Классификация диаграмм в UML

В UML 2 внесены значительные коррективы как в список канонических диаграмм, а именно их число увеличилось до 13, так и в список доступных конструкций языка, что значительно расширило область его применения.

Кроме этого две диаграммы были переименованы: диаграмма кооперации была переименована в диаграмму коммуникации, а диаграмма состояний в диаграмму автомата.

Список новых диаграмм и их названий, приведен ниже. На рисунках ниже приведены соответствующие диаграммы классов.

- Диаграмма внутренней структуры (Composite Structure diagram).

- Диаграмма пакетов (Package diagram).

- Диаграмма автомата (State machine diagram).

- Диаграмма коммуникации (Communication diagram).

- Обзорная диаграмма взаимодействия (Interaction Overview diagram).

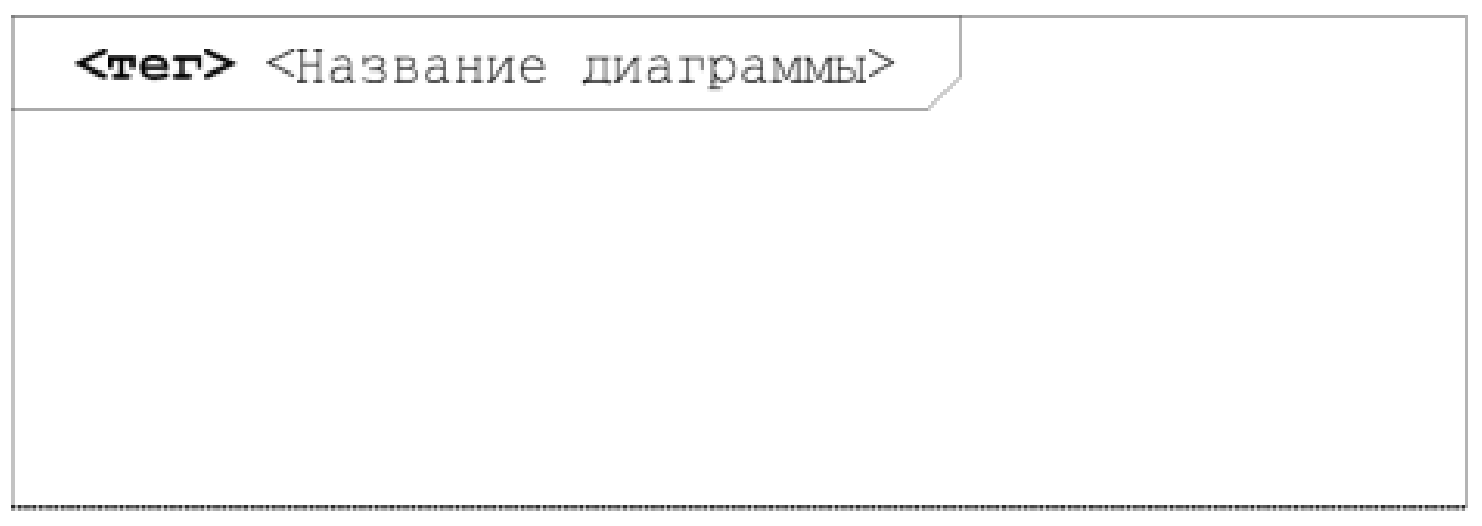
- Диаграмма синхронизации (Timing diagram).

Рассмотрим как стандарт требует оформлять диаграммы.

Оформление диаграмм в UML

Основных элементов оформления два: наружная рамка и ярлычок с названием диаграммы. Если с рамкой все просто – это прямоугольник, ограничивающий область в котором должны находиться элементы диаграммы, то название Диаграммы записывается в специальном формате, приведенном на рисунке Возможные теги (типы) для диаграмм приведены в таблице Теги, предлагаемые стандартом, записаны во второй столбец.

Однако, как показала практика, предлагаемые стандартом правила не всегда удобны и логически обоснованы, поэтому третий столбец таблицы содержит взятую из сложившейся практики альтернативу.



Оформление диаграмм в UML

Типы диаграмм для заголовка

Название диаграммы	Тег (стандартный)	Тег (предлагаемый)
Диаграмма использования	use case или uc	use case
Диаграмма классов	class	class
Диаграмма автомата	state machine или stm	state machine
Диаграмма деятельности	activity или act	activity
Диаграмма последовательности	interaction или sd	sd
Диаграмма коммуникации	interaction или sd	comm
Диаграмма компонентов	component или cmp	component
Диаграмма размещения	не определен	deployment
Диаграмма объектов	не определен	object
Диаграмма внутренней структуры	class	class или component
Обзорная диаграмма взаимодействия	interaction или sd	interaction
Диаграмма синхронизации	interaction или sd	timing
Диаграмма пакетов	package или pkg	package

Классификация диаграмм в UML

Все диаграммы UML можно условно разбить на две группы.

Первая из которых — общие диаграммы, вторая специальные диаграммы

Общие диаграммы

практически не зависят от предмета моделирования и могут применяться в любом программном проекте без оглядки на предметную область, область решений и т. д.

Специальные диаграммы

Специальные диаграммы характеризуются тем, что чаще всего служат для дополнения какой-либо общей диаграммы, например, являются ее частным случаем или же просто играют вспомогательную роль, уточняя некоторые детали.

Типы диаграмм UML

- Структурные
  - Классов(StaticStructurediagram, Classdiagram)
  - Объектов (Objectdiagram)
  - Компонентов (Component diagram)
  - Развёртывания (Deployment diagram)
- Поведения
  - Прецедентов (Use case diagram, Вариантов использования)
  - Состояний (State diagram)
  - Активности (Activity diagram, Деятельности)
- Взаимодействия
  - Кооперации (Колаборации, Коммуникации, Collaboration diagram) – подвид диаграммы композитной/составной структуры
  - Последовательности (Sequence diagram)

Основные типы диаграмм нотации UML

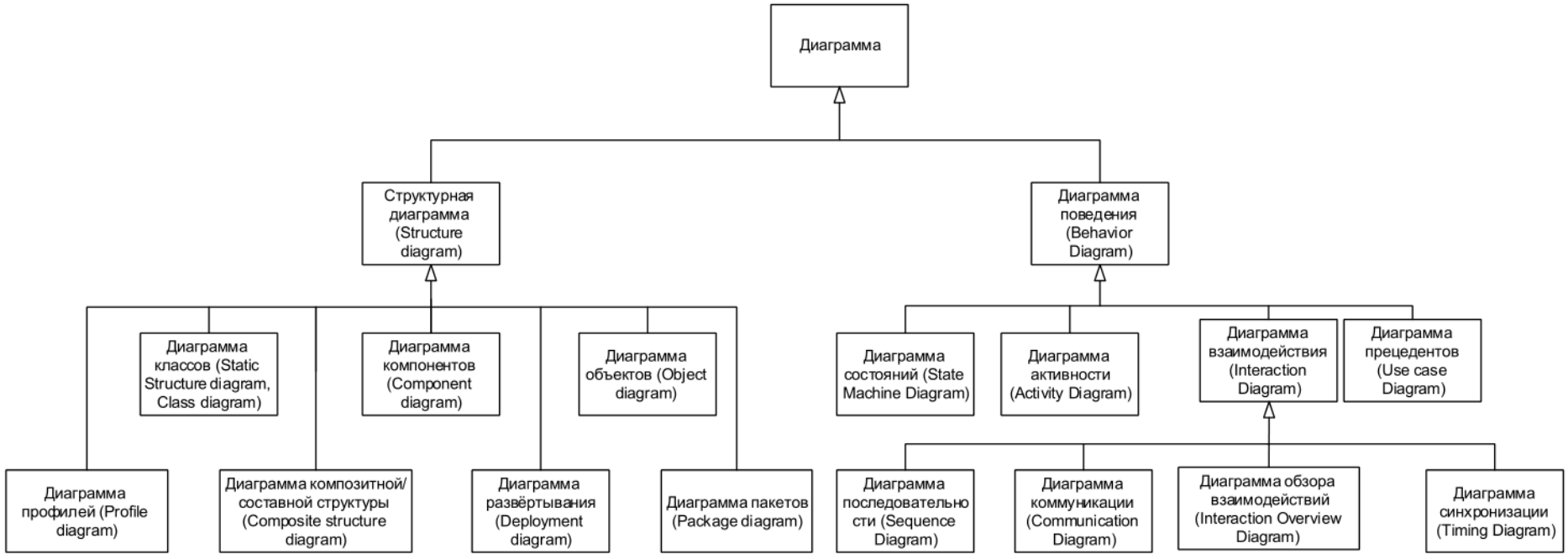


Диаграмма прецедентов

- Диаграммы прецедентов применяются для моделирования вида системы с точки зрения внешнего наблюдателя. На диаграмме прецедентов графически показана совокупность прецедентов и субъектов, а также отношения между ними. **Прецедент** (use case) —это описание множества последовательностей действий (включая их варианты), которые выполняются системой для того, чтобы актер получил результат, имеющий для него определенное значение. Стандартным графическим обозначением прецедента на диаграммах является эллипс, внутри которого содержится краткое название прецедента или имя в форме глагола с пояснительными словами.

Прецеденты – это технология определения функциональных требований к системе

Работа прецедентов заключается в описании типичных взаимодействий между пользователями системы и самой системойи предоставлении описания процесса ее функционирования.

В терминах прецедента пользователи называются актерами. Актер (actor) представляет собой некую роль, которую пользователь играет по отношению к системе.

Примерами актера могут быть пользователь, торговый представитель пользователя, менеджер по продажам и товаровед.

Актеры действуют в рамках прецедентов.

Один актер может выполнять несколько прецедентов; и наоборот, в соответствии с одним прецедентом могут действовать несколько актеров.

Обычно клиентов много, поэтому роль клиента могут играть многие люди.

К тому же один человек может играть несколько ролей, например менеджер по продажам, выполняющий роль торгового представителя клиента.

Актер не обязательно должен быть человеком. -

Если система предоставляет некоторый сервис другой компьютерной системе, то другая система является актером.

Описание прецедента

В каждом прецеденте есть ведущий актер, который посылает системе запрос на обслуживание.

Ведущий актер – это актер, желание которого пытается удовлетворить прецедент и который обычно, но не всегда, является инициатором прецедента.

Одновременно могут быть и другие актеры, с которыми система также взаимодействует во время выполнения прецедента.

Они называются второстепенными актерами.

Каждый шаг в прецеденте – это элемент взаимодействия актера с системой.

Каждый шаг должен быть простым утверждением и должен четко указывать, кто выполняет этот шаг.

Шаг должен показывать намерение актера, а не механику его действий.

Следовательно, в прецеденте интерфейс актера не описывается.

Действительно, составление прецедента обычно предшествует разработке интерфейса пользователя.

Текст прецедента

В тексте описывается главный успешный сценарий прецедента.

После этого приводятся расширения (ответвления или вариации) прецедента.

Главный успешный сценарий:

1. Покупатель просматривает каталог и выбирает товары для покупки.
2. Покупатель оценивает стоимость всех товаров.
3. Покупатель вводит информацию, необходимую для доставки товара (адрес, доставка на следующий день или в течение трех дней).
4. Система предоставляет полную информацию о цене товара и его доставке.
5. Покупатель вводит информацию о кредитной карточке.
6. Система осуществляет авторизацию счета покупателя.
7. Система подтверждает оплату товаров немедленно.
8. Система посылает подтверждение оплаты товаров по адресу электронной почты покупателя.

Расширения:

3а. Клиент является постоянным покупателем.

.1: Система предоставляет информацию о текущей покупке и ее цене, а также информацию о счете.

.2: Покупатель может согласиться или изменить значения по умолчанию, затем возвращаемся к шагу 6 главного успешного сценария.

6а. Система не подтверждает авторизацию счета.

.1: Пользователь может повторить ввод информации о кредитной карте или закончить сеанс.

Расширение прецедента

Расширение внутри прецедента указывает условие, которое приводит к взаимодействиям, отличным от описанных в главном успешном сценарии (main success scenario, MSS), и устанавливает, в чем состоят эти отличия.

Расширение начинается с имени шага, на котором определяется это условие, и предоставляет краткое описание этого условия.

Следуйте этому условию, нумеруя шаги таким же образом, что и в главном успешном сценарии. Заканчивайте эти шаги описанием точки возврата в главный успешный сценарий, если это необходимо.

Структура прецедента – это отличный инструмент для поиска альтернатив главного успешного сценария.

На каждом шаге спрашивайте:

«Что может еще произойти?» и в частности «Что может пойти не так?»

Обычно лучше сначала изучить все возможные условия расширения, чтобы потом не увязнуть в трясине работы над последствиями.

Расширение прецедента

Сложный шаг в прецеденте можно представить другим прецедентом.

В терминах языка UML мы говорим, что первый прецедент включает (includes) второй.

Не существует стандартного способа показать в текст включение прецедента,

Но возможно использовать подчеркивание, которое пред полагает гиперссылку, и во многих инструментах действительно будет гиперссылкой.

Включенные прецеденты могут быть полезными в случае сложных шагов, которые иначе загромождали бы главный сценарий, или когда одни и те же шаги присутствуют в нескольких сценариях.

Не стоит разбивать прецеденты на под прецеденты и использовать их для функциональной декомпозиции.

Расширение прецедента

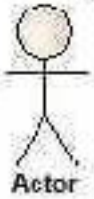
Наряду с шагами сценария можно вставить в прецедент дополнительную общую информацию.

- Предусловие (pre-condition) описывает действия, обязательно выполняемые системой перед тем, как она разрешит начать работу прецедента.
  Это полезная информация, позволяющая разработчикам не проверять некоторые условия в их программе.
- Гарантия (guarantee) описывает обязательные действия системы по окончании работы шаблона ответа. Успешные гарантии выполняются после успешного сценария; минимальные гарантии выполняются после любого сценария.
- Триггер (trigger) определяет событие, инициирующее выполнение прецедента.

Диаграмма прецедентов (Use Case Diagram)

Структурные элементы

Актер



Вариант использования

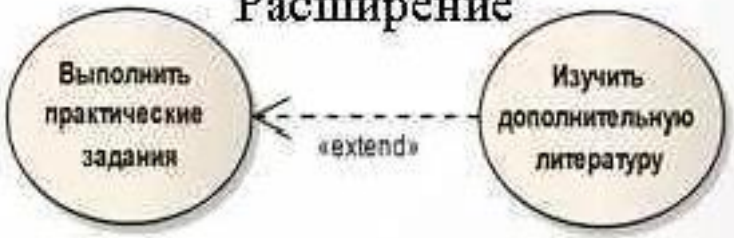


Взаимосвязи

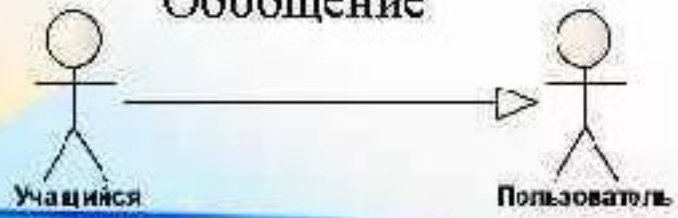
Ассоциация



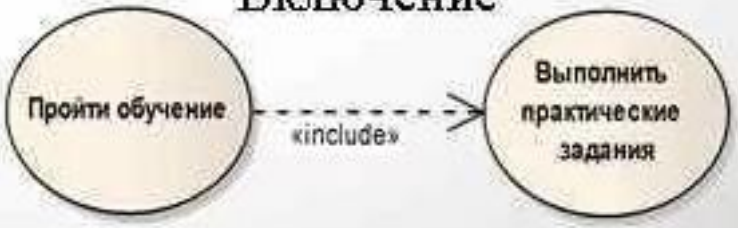
Расширение



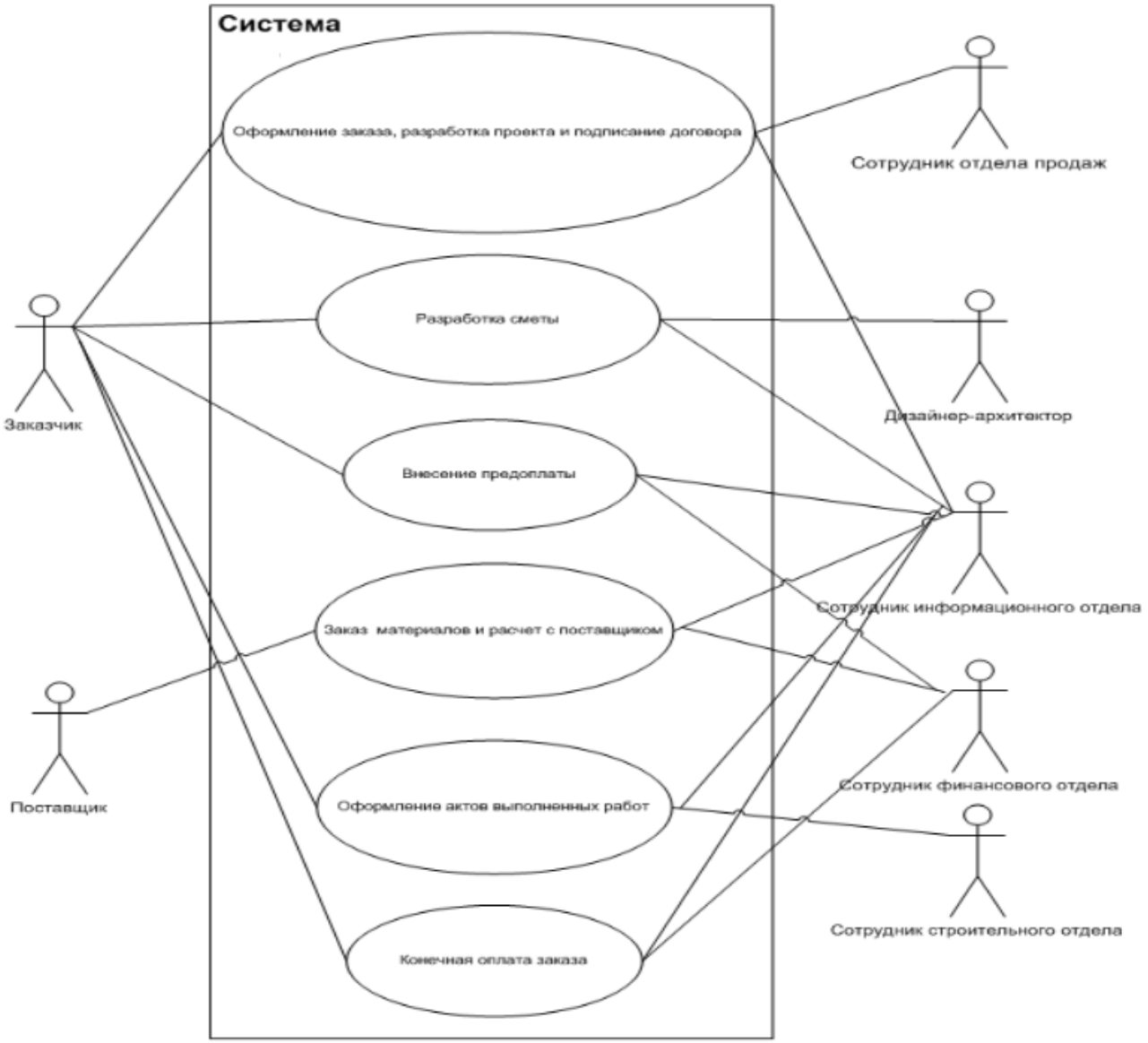
Обобщение



Включение



Пример диаграммы прецедентов



Диаграммы прецедента

Лучше всего обдумывать диаграмму прецедентов с помощью графической таблицы, показывающей их содержимое.

Она напоминает диаграмму контекста, используемую в структурных методах, поскольку она показывает границы системы и ее взаимодействие с внешним миром.

Диаграмма прецедентов показывает актеров, прецеденты и отношения между ними:

- Какие актеры выполняют тот или иной прецедент
- Какие прецеденты включают другие прецеденты

В языке UML помимо отношения «include» (включает) есть и другие типы отношений между прецедентами, например отношение «extend» (расширяет). Не следует слишком злоупотреблять ими лучше описывать сам прецедент

Уровни прецедентов

Общая проблема прецедентов состоит в том, что, увлекшись взаимодействием пользователя с системой, можно не обратить внимание на тот факт, что лучшим способом решения проблемы может быть изменение самого бизнес-процесса.

Часто можно слышать упоминание о прецедентах системы и прецедентах бизнес-процессов.

Эта терминология не является точной, но обычно считается, что прецедент системы (system use case) описывает особенности взаимодействия с программным обеспечением, тогда как прецедент бизнес-процесса (business use case) представляет собой реакцию бизнес-процесса на действие клиента или некоторое событие.

Уровни прецедентов

В книге Кокборна предлагается схема уровней прецедентов.

Базовый прецедент находится на «уровне моря».

Прецеденты уровня моря (sea level) обычно представляют отдельное взаимодействие ведущего актера и системы.

Такие прецеденты предоставляют ведущему актеру какой-либо полезный результат и обычно занимают от пары минут до получаса.

Прецеденты, которые существуют в системе, только если они включены в прецеденты уровня моря, называются прецедентами уровня рыб (fish level).

Прецеденты высшего уровня, уровня воздушного змея (kite-level), показывают, как прецеденты уровня моря на страиваются на более широкое взаимодействие с бизнес-процессами.

Обычно прецеденты уровня воздушного змея являются прецедентами Бизнес-процессов, а на уровне моря и на уровне рыб находятся прецеденты системы. Большинство ваших прецедентов должно принадлежать уровню моря.

Уровни прецедентов

В экстремальном программировании (Extreme Programming) возможности системы называются пожеланиями пользователя.

Общим является вопрос о том, как установить соответствие между возможностями и прецедентами.

Использование возможностей – это хороший способ разделения системы на блоки при планировании итеративного процесса, в результате чего каждая итерация предоставляет определенное количество возможностей.

Таким оразом оба приема описывают требования, их цели различны.

Описать возможности можно сразу, но многие специалисты считают более удобным в первую очередь разработать прецеденты, а уже затем сгенерировать список возможностей.

Возможность может быть представлена целым прецедентом, сценарием в прецеденте, шагом в прецеденте или каким-либо вариантом поведения

Обычно возможности получаются более четко определенными, чем прецеденты.

Применение прецедентов

Прецеденты представляют собой ценный инструмент для понимания функциональных требований к системе.

Первый вариант прецедентов должен составляться на ранней стадии выполнения проекта.

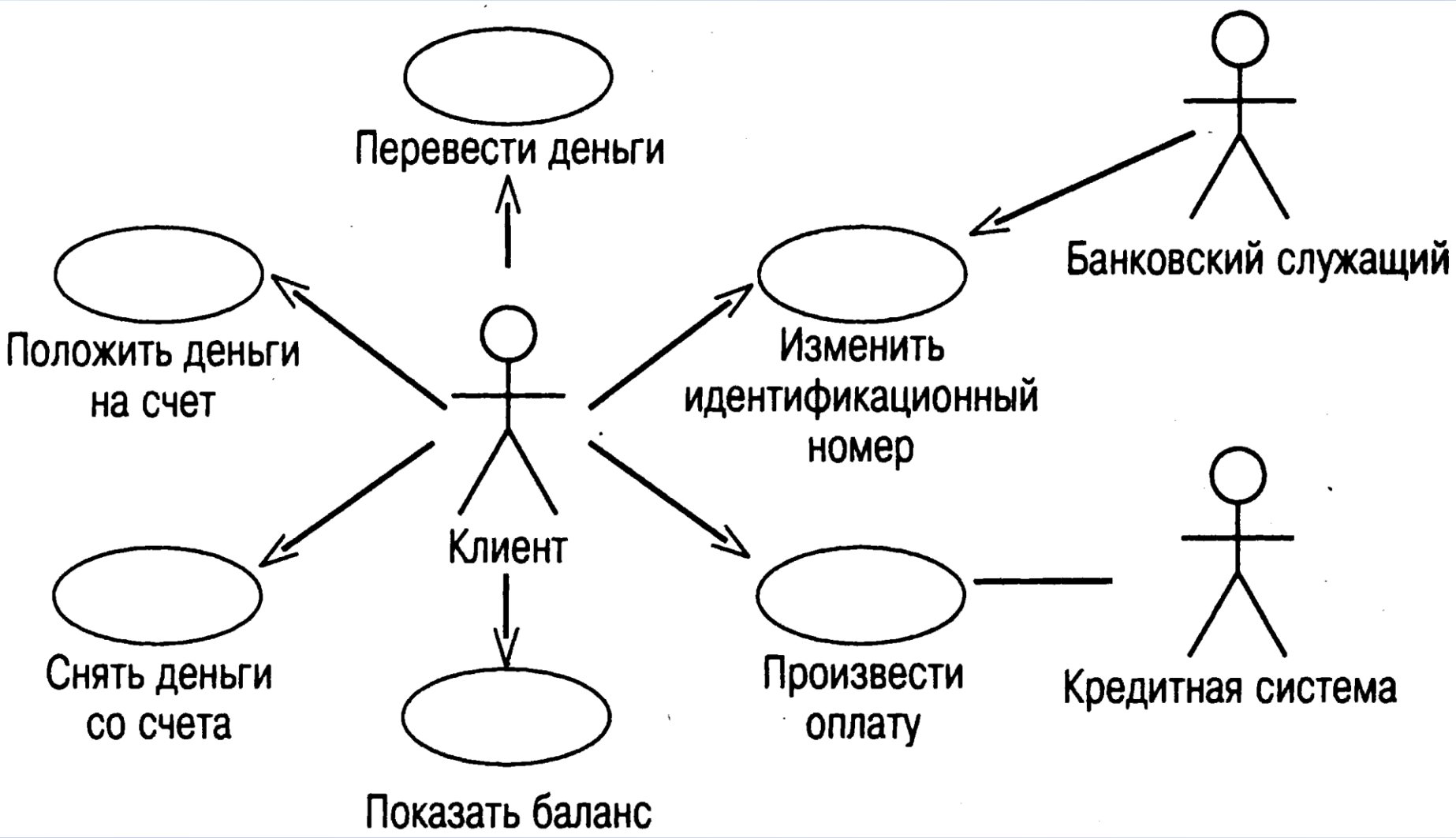
Более подробные версии прецедентов должны появляться непосредственно перед реализацией данного прецедента.

Важно помнить, что прецеденты представляют взгляд на систему со стороны.

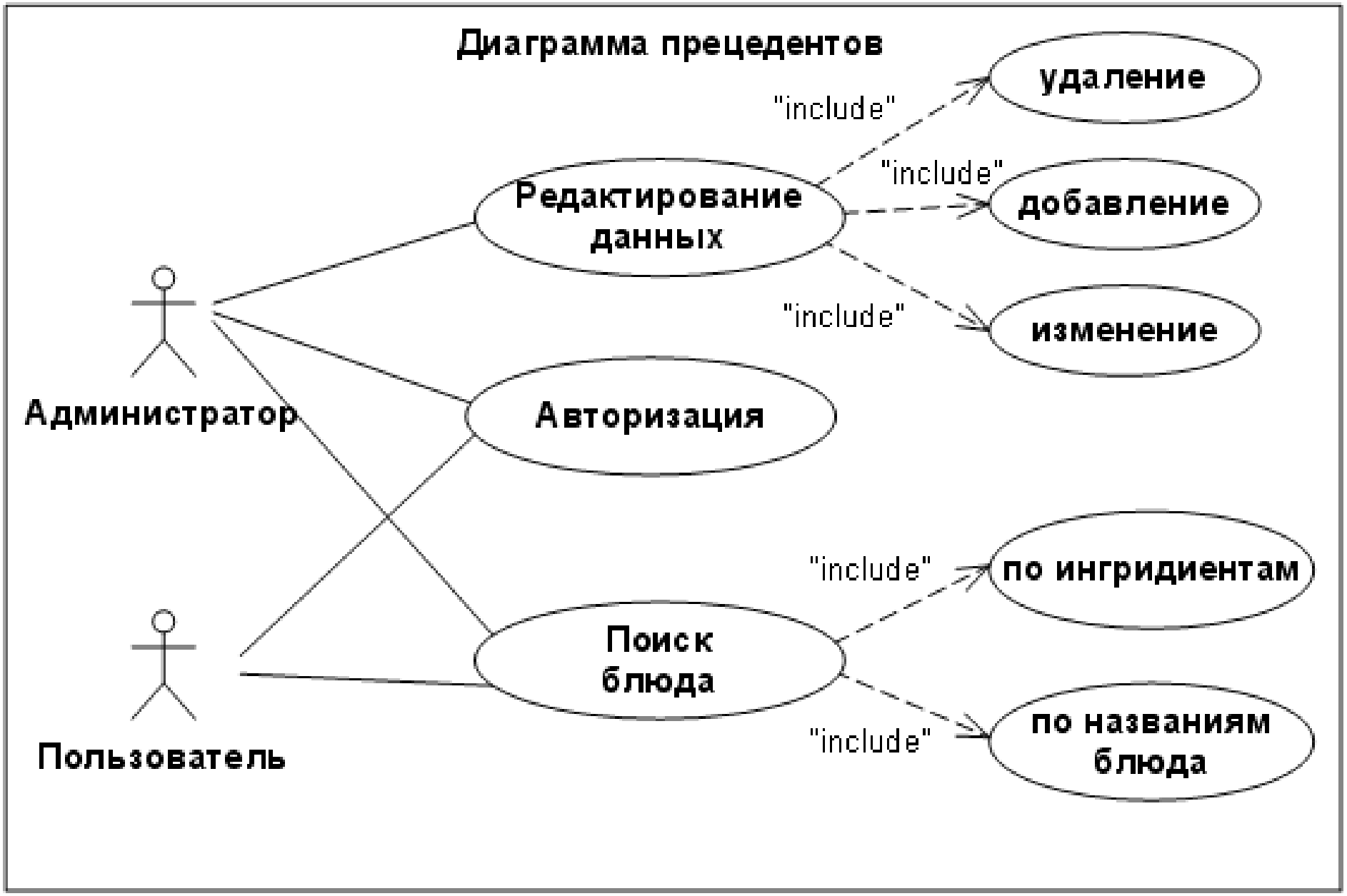
Следовательно не нужно ожидать соответствия между прецедентами и классами внутри системы.

Несмотря на то что в языке UML ничего не говорится о тексте прецедентов, именно текстовое содержание прецедентов является основной ценностью этой технологии.

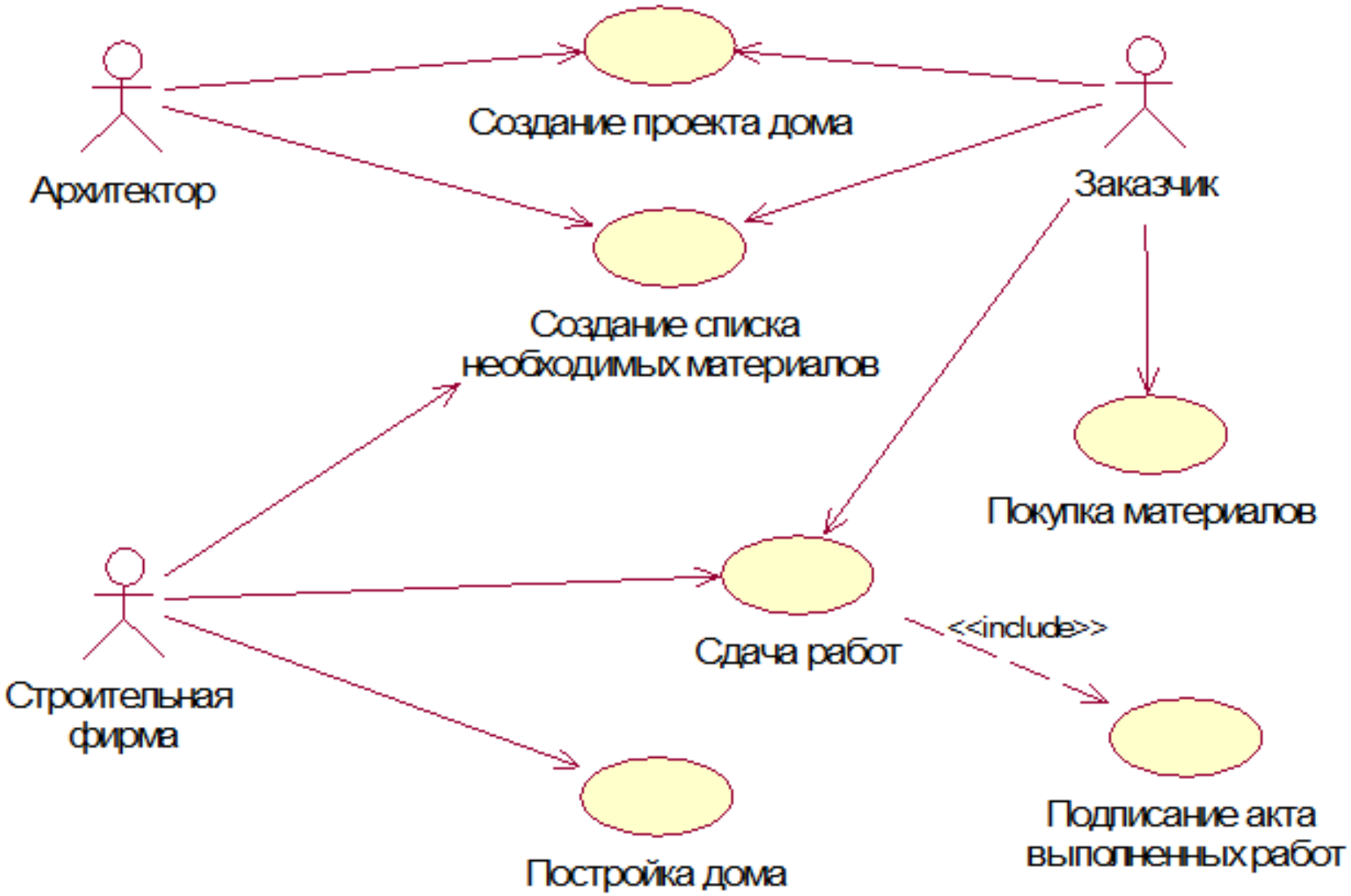
Пример диаграммы прецедентов



Пример диаграммы прецедентов



Пример диаграммы прецедентов



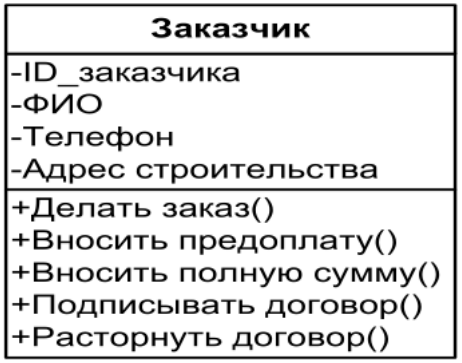
Лекция 5

UML (Unified Model Language)
Модели анализа, проектирования

ОСНОВНЫЕ ДИАГРАММЫ

диаграммы классов анализа, диаграммы взаимодействия (последовательности и коммуникации), диаграммы пакетов, модель проектирования, назначение и состав модели проектирования, диаграммы классов.

Диаграмма классов



- **Диаграмма классов** – основной способ описания структуры системы. Поскольку UML в первую очередь объектно—ориентированный язык, классы в нем являются основным, так называемым, «строительным материалом». Для построения диаграммы классов необходимо отобразить все выбранные классы. После этого необходимо составить список атрибутов и операций для выбранных классов.

Диаграммы классов

Наиболее распространенный тип диаграмм.

Описывает типы объектов системы и различного рода **статические** отношения, которые существуют между ними.

Помимо классов на диаграмме отображаются ***свойства классов***, ***операции классов*** и ***ограничения классов***.

Диаграмма классов

Диаграмма классов описывает типы объектов системы и различного рода статические отношения, которые существуют между ними.

На диаграмме прямоугольники представляют классы и разделены на три части:

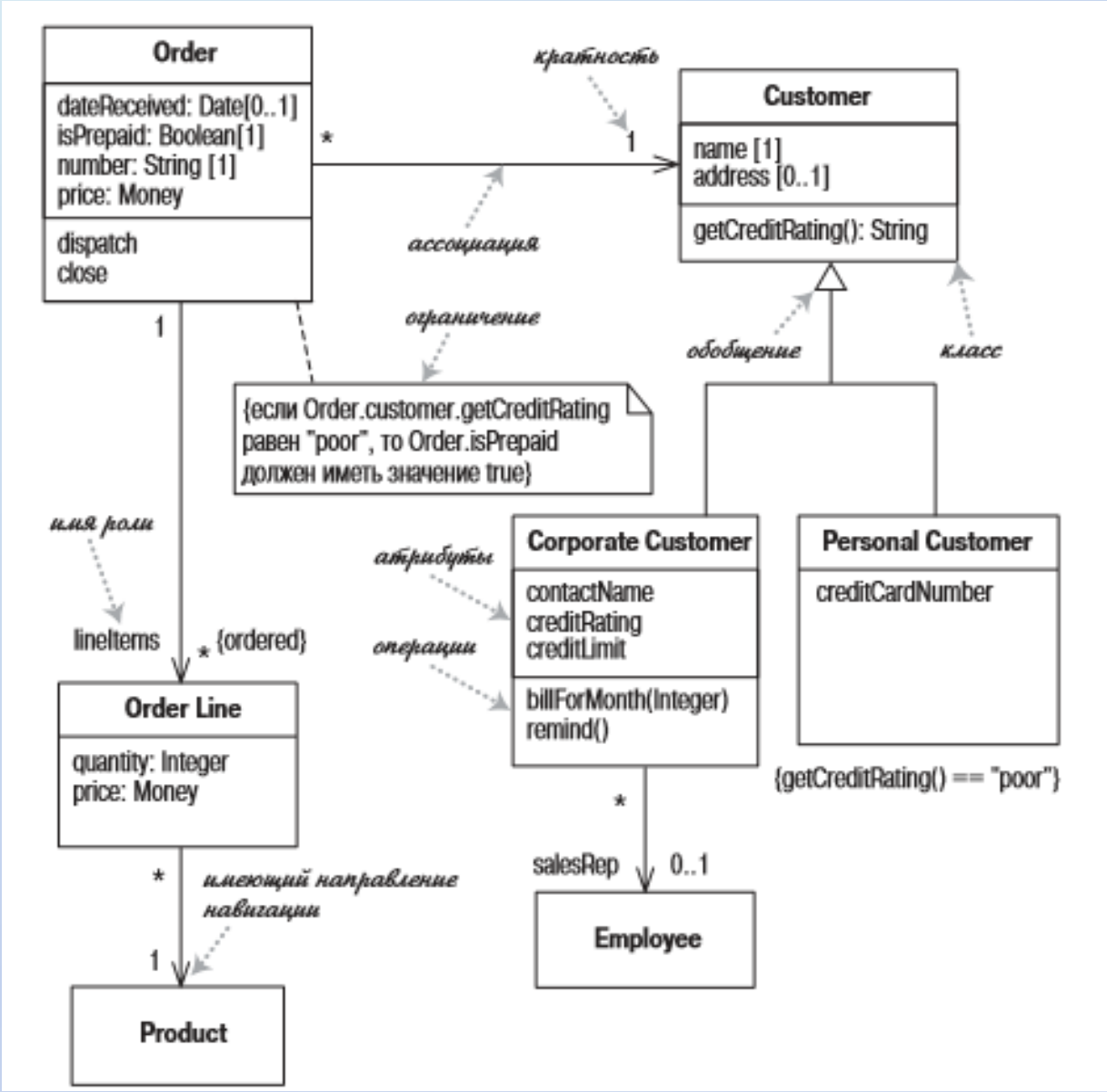
- имя класса(жирный шрифт),
- его атрибуты
- его операции.

На диаграммах классов отображаются также свойства классов, операции классов и ограничения, которые накладываются на связи между объектами.

В UML термин функциональность (feature) используется в качестве основного термина, описывающего и свойства, и операции класса.

Ниже изображена типичная модель класса, связанная с обработкой заказов клиентов.

Простая диаграмма классов



Выше на рисунке показаны два вида связей между классами: ассоциации и обобщения.

Свойства на диаграмме классов.

Свойства представляют структурную функциональность класса.

В первом приближении можно рассматривать свойства как поля класса.

Как мы увидим позднее, в действительности это не так просто, но вполне приемлемо для начала.

Свойства представляют единое понятие, воплощающееся в двух совершенно различных сущностях: в атрибутах и в ассоциациях.

Хотя на диаграмме они выглядят совершенно по-разному, в действительности это одно и то же.

Атрибуты

Атрибут описывает свойство в виде строки текста внутри прямоугольника класса.

• Метка видимость обозначает, относится ли атрибут к открытым (+) (public) или к закрытым (?) (private).

• Имя атрибута – способ ссылки класса на атрибут – приблизительно соответствует имени поля в языке программирования.

• Тип атрибута накладывает ограничение на вид объекта, который может быть размещен в атрибуте. Можно считать его аналогом типа поля в языке программирования.

• Значение по умолчанию представляет собой значение для вновь создаваемых объектов, если атрибут не определен в процессе создания.

• Элемент {строка свойств} позволяет указывать дополнительные свойства атрибута. В примере он равен readOnly, то есть клиенты не могут изменять атрибут. Если он пропущен, то обычно, атрибут можно модифицировать

• Кратность рассмотрена ниже.

Кратность

Кратность свойства обозначает количество объектов, которые могут заполнять данное свойство. Чаще всего встречаются следующие кратности: 0, 0…1, *.

Термины, имеющие отношение к кратности:

- Optional (необязательный) предполагает нулевую нижнюю границу.
- Mandatory (обязательный) подразумевает, что нижняя граница = или > 1.
- Singlevalued (однозначный) – для такого атрибута верхняя граница равна 1.
- Multivalued (многозначный) имеет в виду, что верхняя граница > 1; обычно *.

В большинстве случаев кратности определяются своими нижней и верхней границами. Нижняя граница может быть нулем или положительным числом, верхняя граница представляет собой положительное число или * (без ограничений). Если нижняя и верхняя границы совпадают, то можно указать одно число;

Ассоциации

Другая ипостась свойства – это ассоциация. Значительная часть информации, которую можно указать в атрибуте, появляется в ассоциации.

Ассоциация – это непрерывная линия между двумя классами, направленная от исходного класса к целевому классу. Имя свойства (вместе с кратностью) располагается на целевом конце ассоциации.

Целевой конец ассоциации указывает на класс, который является типом свойства. Большая часть информации в обоих представлениях одинакова, но некоторые элементы отличаются друг от друга. В частности, ассоциация может показывать кратность на обоих концах линии.

К другому распространенному типу ассоциаций относится двунаправленная ассоциация. Двунаправленная ассоциация – это пара свойств, связанных в противоположных направлениях.

(Пример Владелец ⟷ Имущество)

Операции

Операции (operations) представляют собой действия, реализуемые некоторым классом. Существует очевидное соответствие между операциями и методами класса. Обычно можно не показывать такие операции, которые просто манипулируют свойствами, поскольку они и такподразумеваются.

Полный синтаксис операций в языке UML выглядит следующим образом:

видимость имя (список параметров) : возвращаемый тип {строка свойств}

- Метка видимости обозначает, относится ли операция к открытым (+) (public) или к закрытым (?) (private);
- Имя – это строка.
- Список параметров – список параметров операции.
- Возвращаемый тип – тип возвращаемого значения, если таковое есть.
- Строка свойств – значения свойств, которые применяются к данной операции.

Операции

Операции (operations) представляют собой действия, реализуемые некоторым классом. Существует очевидное соответствие между операциями и методами класса. Обычно можно не показывать такие операции, которые просто манипулируют свойствами, поскольку они и такподразумеваются.

Полный синтаксис операций в языке UML выглядит следующим образом:

видимость имя (список параметров) : возвращаемый тип {строка свойств}

- Метка видимости обозначает, относится ли операция к открытым (+) (public) или к закрытым (?) (private);
- Имя – это строка.
- Список параметров – список параметров операции.
- Возвращаемый тип – тип возвращаемого значения, если таковое есть.
- Строка свойств – значения свойств, которые применяются к данной операции.

Зависимость

Считается, что между двумя элементами существует зависимость (dependency),

если изменения в определении одного элемента (сервера)

могут вызвать изменения в другом элементе (клиенте).

В случае классов зависимости появляются по разным причинам:

- один класс посылает сообщение другому классу;

- один класс владеет другим классом как частью своих данных;

- один класс использует другой класс в качестве параметра операции.

UML позволяет изобразить зависимости между элементами всех типов.

Зависимости можно использовать всякий раз, когда надо показать,

как изменения в одном элементе могут повлиять на другие элементы.

Правила ограничений

При построении диаграмм классов большая часть времени уходит на представление различных ограничений. Например на рисунке показано, что Заказ (Order) может быть сделан только одним единственным Клиентом (Customer).

Из этой диаграммы классов также следует, что каждая Line Item (Позиция заказа) рассматривается отдельно. Также диаграмма показывает, что Корпоративные клиенты располагают кредитами, а Индивидуальные клиенты – нет.

С помощью базовых конструкций ассоциации, атрибута и обобщения можно сделать многое, специфицируя наиболее важные ограничения, но этими средствами невозможно записать каждое ограничение.

Эти ограничения еще нужно каким-то образом отобразить, и диаграмма классов является вполне подходящим местом для этого.

Правила ограничений

Язык UML разрешает использовать для описания ограничений все что угодно.

При этом необходимо лишь придерживаться правила: ограничения следует помещать в фигурные скобки ({}).

Можно употреблять разговорный язык, язык программирования или формальный объектный язык ограничений UML (Object Constraint Language, OCL)

Формальное написание позволяет избежать риска неоднозначного толкования конструкций разговорного языка.

Можно предварять ограничение именем с двоеточием

Пример готовой диаграммы классов

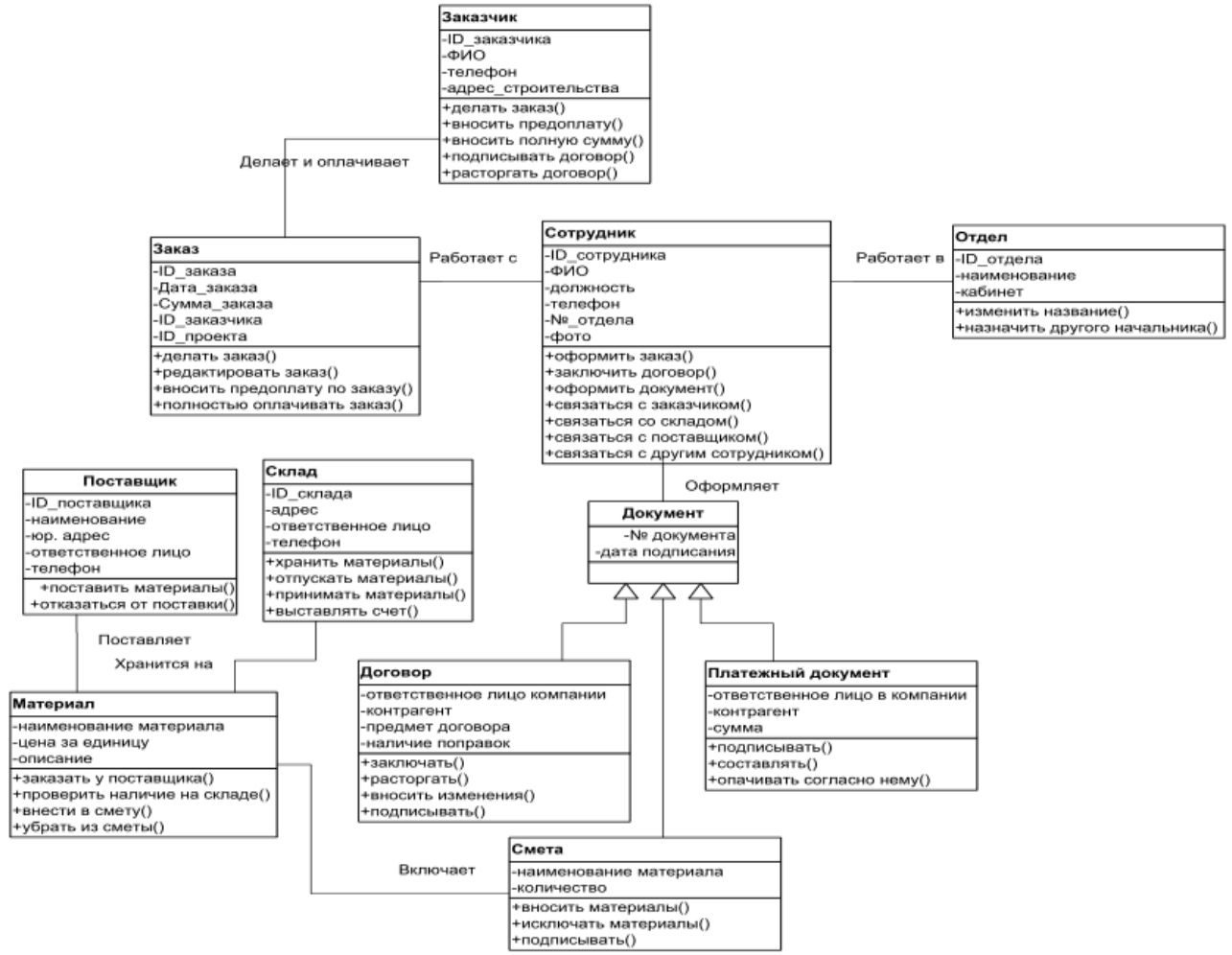


Диаграмма анализа классов

Фундаментальными понятиями объектно-ориентированного подхода являются понятия *объекта* и *класса* которые представляются абстракциями реальной или воображаемой сущности (набора сущностей).

Класс анализа – еще более абстрактная сущность, чем просто класс, представляет собой набор из одного или более классов.

Таким образом, ***класс анализа*** – это укрупненная абстракция, которая на концептуальном уровне (без точного определения атрибутов и операций) описывает некоторый фрагмент системы.

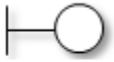
Существует три вида классов анализа:

- граничный;
- управляющий;
- сущности.

Диаграмма анализа классов

Диаграмма классов анализа по существу является прообразом классической диаграммы классов

Элементами, отображаемыми на диаграмме, являются классы и отношения между ними.

Варианты отображения	Граничный класс	Управляющий класс	Класс сущности
Графический стереотип	 Диалоговое окно «Нормативы»	 Расчет Vдоп	 План
Стандартное обозначение со строкой-стереотипом	«boundary» Диалоговое окно «Нормативы»	«control» Расчет Vдоп	«entity» План

При этом для отображения классов можно воспользоваться стандартным обозначением класса (прямоугольник) с указанием внутри него соответствующего стереотипа или значком-стереотипом.

Диаграмма анализа классов

Рассмотренное разбиение классов анализа на три группы согласуется с популярным шаблоном проектирования **«Модель-Представление-Контроллер»** (англ. MVC, Model-View-Controller).

При этом под классом сущности - понимается модель, граничным классом - представление и управляющим классом - контроллер.

Связи между классами анализа отображаются с использованием ***отношений*** пяти видов:

- ассоциаций;
- агрегаций;
- композиций;
- обобщения;
- зависимостей.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

1. При выделении классов анализа следует учитывать тот факт, что они являются обобщенными (укрупненными) сущностями, которые в дальнейшем подлежат уточнению и возможному разбиению на несколько более мелких классов. Можно провести аналогию между ними и накопителями данных на DFD, которые, как правило, являются крупными концептуальными объектами (совокупностями тесно взаимосвязанных таблиц), подлежащими уточнению на стадии информационного проектирования с помощью одной из методологий ERD.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

2. Для выделения классов сущностей необходимо определить все реальные либо воображаемые объекты, имеющие существенное значение для рассматриваемой предметной области, информация о которых подлежит хранению. При этом из спецификаций вариантов использования следует выделить все объекты, которые могут существовать независимо от других. Например, объект «билет» является независимой сущностью, потому что любой билет существует независимо от того, знаем мы его номер, стоимость или нет. Т. е. при выделении классов-сущностей действуют те же правила, что при построении концептуальной модели БД.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

3. Для каждого актера следует предусмотреть, как минимум, один граничный класс в целях организации интерфейса между ним и системой. Аналогично для каждого класса сущности, как правило, должен быть граничный класс – ведь по каждому объекту класса сущности должна быть предусмотрена возможность просмотра, ввода и/или корректировки информации через определенную форму ввода/вывода или чтения/записи через определенный интерфейс.

4. Для управления, обеспечения взаимодействия и координации работы объектов, реализующих одну из функций системы (обычно, вариант использования), необходимо предусмотреть, как минимум, один управляющий класс. Как правило, взаимодействие между граничным классом и классом сущности происходит через управляющий класс.

Диаграмма анализа классов

При разработке диаграммы следует придерживаться следующих правил и рекомендаций.

5. В целях облегчения восприятия специфики связей между классами рекомендуется использовать отношения агрегации, композиции и обобщения.

6. При разработке диаграммы основное внимание должно быть уделено определению и детализации классов сущностей, управляющих и граничных классов, обеспечивающих взаимодействие с внешними системами.

Граничные классы, обеспечивающие взаимодействие с пользователями, не требуют излишней детализации до уровня отдельного поля ввода или ниспадающего списка, так как современные среды программирования обладают богатыми возможностями по быстрому созданию пользовательского интерфейса.

Диаграмма анализа классов. Пример

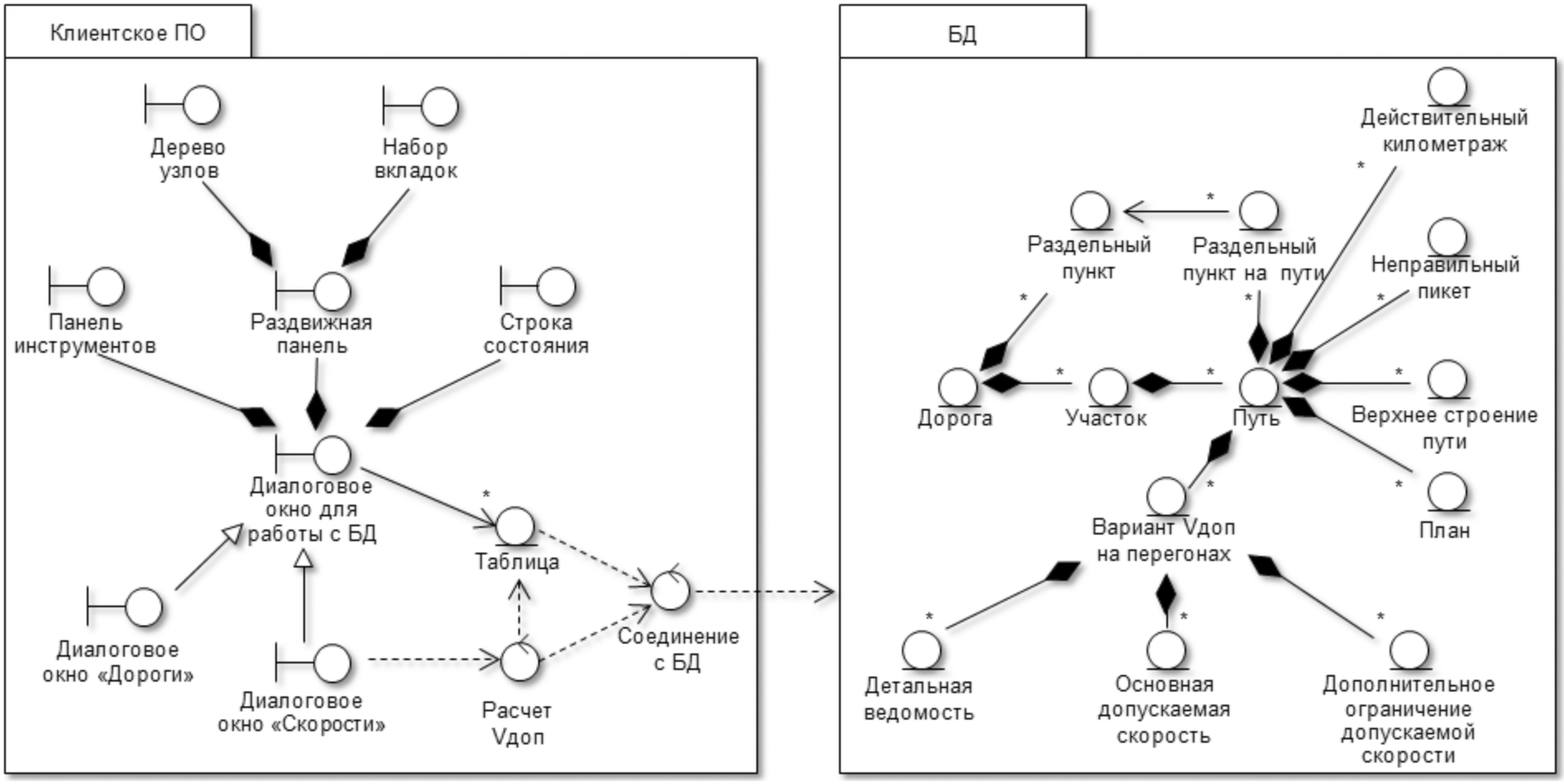


Диаграмма анализа классов

В правой части (начиная с класса «Дорога») расположены классы, описывающие структуру БД на сервере. Хотя исходный программный код не будет содержать этих классов, их детальная разработка не менее важна, чем разработка остальных классов. В данном случае правая часть диаграммы представляет концептуальную модель БД, которую можно описать с помощью одной из методологий ERD.

Во время работы программы информация из БД считывается в объекты класса «Таблица» (кэшируется), обрабатывается объектами других классов и при необходимости обратно записывается в БД. Каждый такой объект, помимо данных конкретной таблицы БД, содержит описание свойств таблицы в целом, полей (имя поля, его тип, значение по умолчанию и т. д.), описание структуры заголовка таблицы, отображаемого на экране, и другую необходимую информацию.

За каждым диалоговым окном, предназначенным для работы с БД, закрепляется определенный набор объектов класса «Таблица».

Диаграмма анализа классов

Аналогично классу «Диалоговое окно "Дороги"» в левую часть диаграммы должны быть нанесены классы «Диалоговое окно "Справочники"» и «Диалоговое окно "Нормативы"», а аналогично классу «Диалоговое окно "Скорости"» – класс «Диалоговое окно "Возвышения"», связанный с управляющим классом «Расчет возвышений».

Концептуальная модель БД должна быть дополнена таблицами, описывающими нормативно-справочную информацию и остальные данные дорожного уровня, необходимые для работы системы.

Диаграммы последовательности

Диаграммы взаимодействия (interaction diagrams) описывают взаимодействие групп объектов в различных условиях их поведения. UML определяет диаграммы взаимодействия нескольких типов, из которых наиболее употребительными являются диаграммы последовательности.

Обычно диаграмма последовательности описывает один сценарий.

На диаграмме показаны экземпляры объектов и сообщения, которыми обмениваются объекты в рамках одного прецедента (use case).

Предположим, что у нас есть заказ, и мы собираемся вызвать команду для определения его стоимости. При этом объекту заказа (Order) необходимо просмотреть все позиции заказа (Line Items) и определить их цены, основанные на правилах построения цены продукции в строке заказа (Order Line).

Проделав это для всех позиций заказа, объект заказа должен вычислить общую скидку, которая определяется индивидуально для каждого клиента.

Диаграммы последовательности

Диаграммы взаимодействия (interaction diagrams) описывают взаимодействие групп объектов в различных условиях их поведения. UML определяет диаграммы взаимодействия нескольких типов, из которых наиболее употребительными являются диаграммы последовательности.

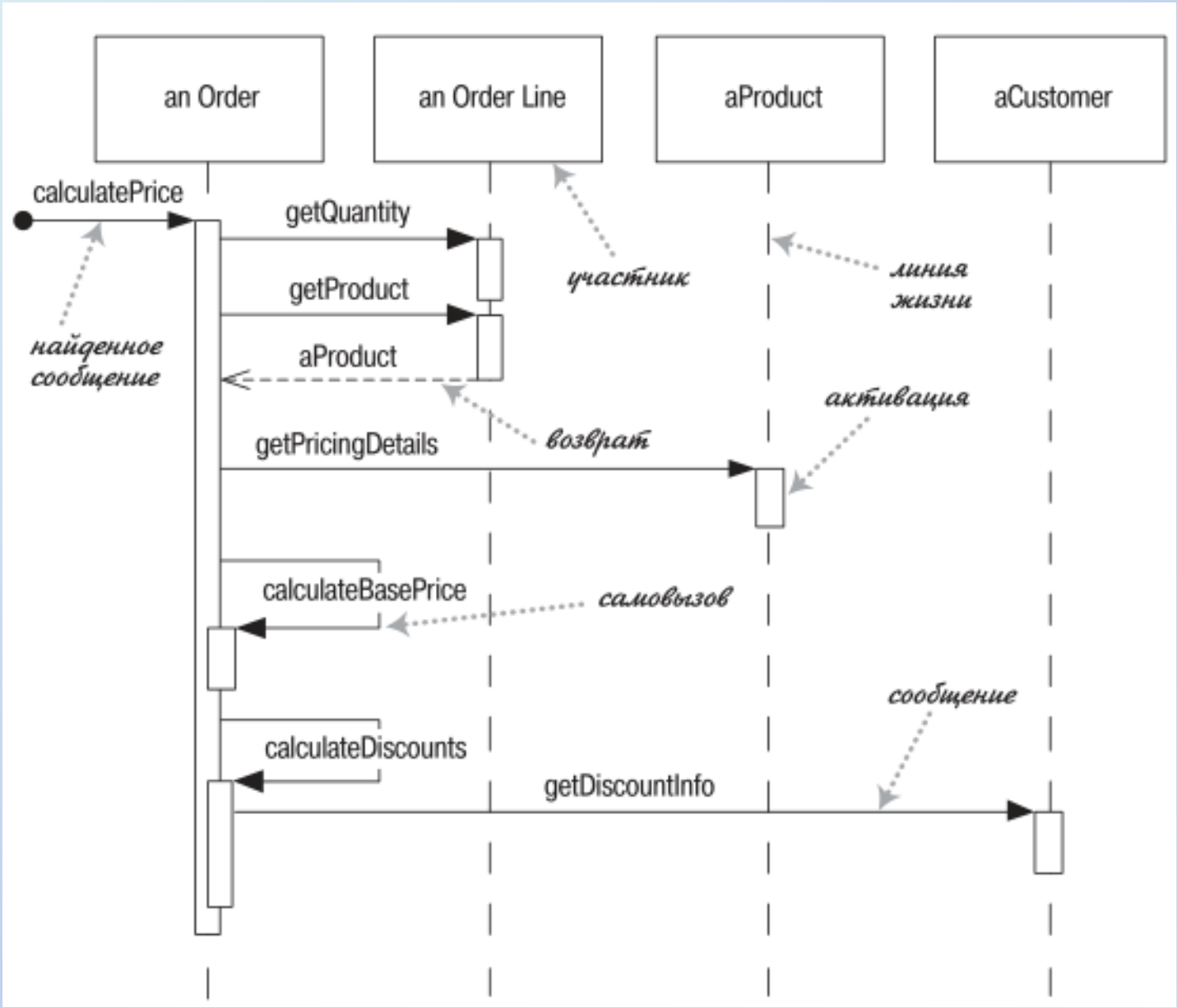
Обычно диаграмма последовательности описывает один сценарий.

На диаграмме показаны экземпляры объектов и сообщения, которыми обмениваются объекты в рамках одного прецедента (use case).

Предположим, что у нас есть заказ, и мы собираемся вызвать команду для определения его стоимости. При этом объекту заказа (Order) необходимо просмотреть все позиции заказа (Line Items) и определить их цены, основанные на правилах построения цены продукции в строке заказа (Order Line).

Проделав это для всех позиций заказа, объект заказа должен вычислить общую скидку, которая определяется индивидуально для каждого клиента.

Диаграммы последовательности. Пример



Диаграммы последовательности

Диаграммы последовательности показывают взаимодействие, представляя каждого участника вместе с его линией жизни (lifeline), которая идет вертикально вниз и упорядочивает сообщения на странице;

Сообщения также следует читать сверху вниз, что понятно на диаграмме

Видно, что экземпляр заказа посылает строке заказа сообщения getQuantity и getProduct. Можно увидеть также, как заказ применяет метод к самому себе и как этот метод посылает сообщение getDiscountInfo экземпляру клиента.

Однако диаграмма не все показывает так хорошо. Последовательность сообщений getQuantity, getProduct, getPricingDetails и calculateBasePrice должна быть реализована для каждой строки заказа, тогда как метод calculateDiscounts вызывается лишь однажды.

Диаграммы последовательности

В большинстве случаев можно считать участников диаграммы взаимодействия объектами, как это и было в действительности в UML 1.

Однако в UML 2 их роль значительно сложнее, и полное ее объяснение выходит за рамки этой книги. Поэтому я употребляю термин участники (participants), который формально не входит в спецификацию UML.

В UML версии 1 участники были объектами, и поэтому их имена подчеркивались, но в UML 2 их надо показывать без подчеркивания.

На приведенной диаграмме именуют участников, используя стиль anOrder.

В большинстве случаев это вполне приемлемо.

Вот более полный синтаксис: имя : Класс, где и имя, и класс не обязательны, но если класс используется, то двоеточие должно присутствовать.

У первого сообщения нет участника, пославшего его, так как оно приходит из неизвестного источника. Оно называется найденным сообщением (found message).

Диаграммы последовательности

Каждая линия жизни имеет полосу активности, которая показывает интервал активности участника при взаимодействии. Она соответствует времени нахождения в стеке одного из методов участника. В языке UML полосы активности не обязательны Именование бывает часто полезным для установления связей между участниками на диаграмме. Как видно на диаграмме, вызов метода getProduct возвращает aProduct, имеющего то же самое имя и, следовательно, означающего того же самого участника, aProduct, которому по сылается вызов getPricingDetails. **Обратной стрелкой обозначен только этот вызов с целью показать соответствие.**

Многие разработчики используют возвраты для всех вызовов, но лучше применять их, только когда это дает дополнительную информацию; иначе они просто вносят неразбериху.

Возможно, что даже в данном случае можно было опустить возврат.

Диаграммы последовательности

Основная задача остается той же самой, но способ взаимодействия участников для ее решения совершенно другой.

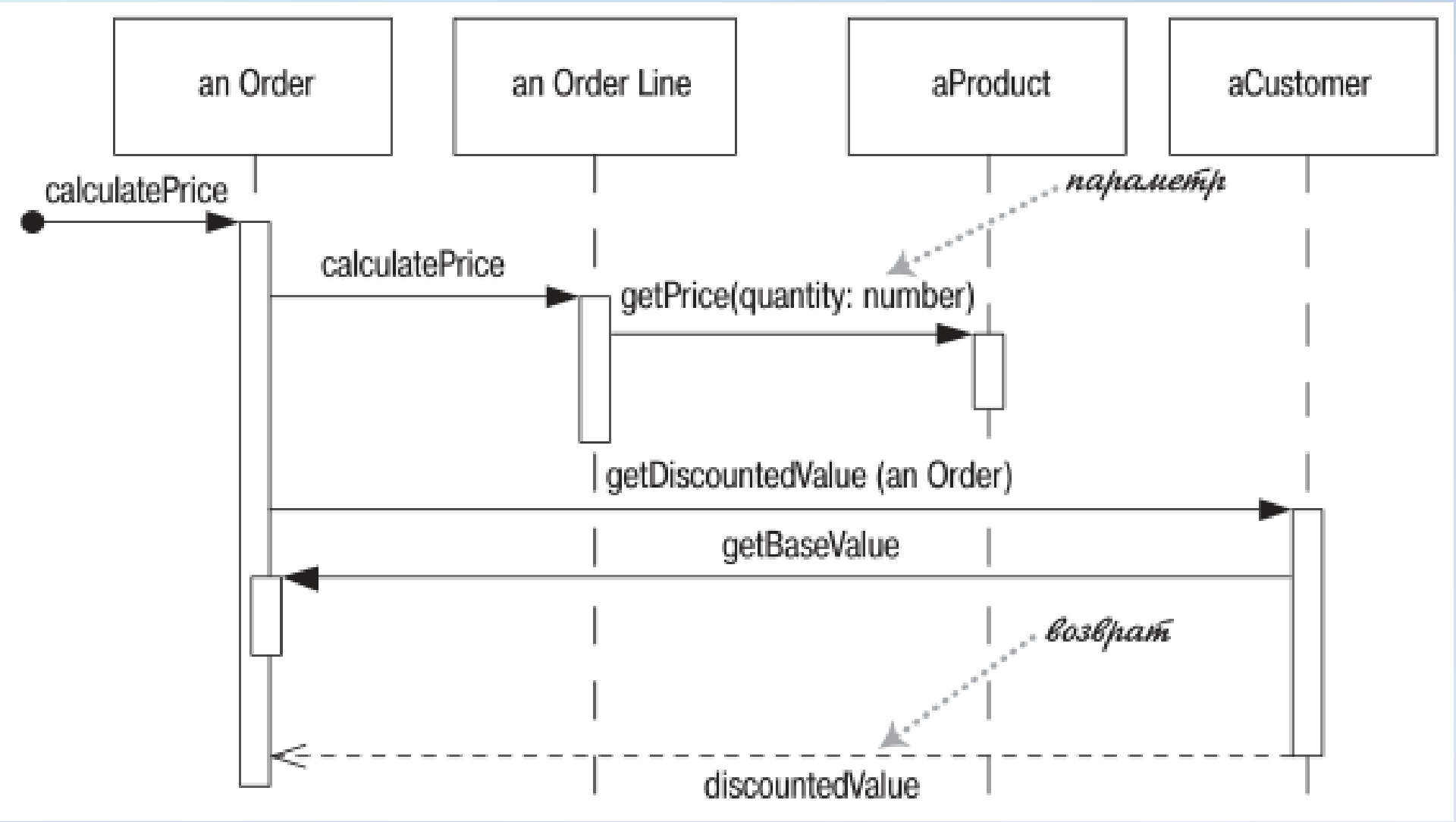
Заказ спрашивает каждую строку заказа о его собственной цене (Price).

Сама строка заказа передает вычисление дальше – объекту продукта (Product); обратите внимание, как мы показываем передачу параметра.

Подобным же образом для вычисления скидки объект заказа вызывает метод для клиента (Customer).

Поскольку для выполнения этой задачи клиенту требуется информация от объекта заказа, то он делает повторный вызов в отношении заказа для получения этих данных.

Диаграммы последовательности



Диаграммы последовательности

На этих двух диаграммах надо обратить внимание, на сколько ясно диаграмма последовательности показывает различия во взаимодействии участников. В этом проявляется мощь диаграмм взаимодействий. Они не очень хорошо представляют детали алгоритмов, такие как циклы или условное поведение, но делают абсолютно прозрачными вызовы между участниками и дают действительно ясную картину того, какую обработку выполняют конкретные участники.

Во-вторых, разница в стиле между двумя взаимодействиями заключается также в том, что на первом рисунке представлено централизованное управление (centralized control), когда один из участников в значительной степени выполняет всю обработку, а другие предоставляют данные. На втором рисунке изображено распределенное управление (distributed control), при котором обработка распределяется между многими участниками, каждый их которых выполняет небольшую часть алгоритма.

Диаграммы последовательности

Большинство разработчиков, особенно новички в объектно-ориентированном программировании, чаще всего применяют централизованное управление.

Во многих случаях это проще, так как вся обработка сосредоточена в одном месте; напротив, в случае распределенного управления при попытке понять программу создается ощущение погони за объектами.

Кроме того, распределенное управление позволяет создать больше возможностей для применения полиморфизма, чем в случае применения условной логики.

Если алгоритмы определения цены отличаются для различных типов продуктов, то механизм распределенного управления позволяет нам использовать подклассы класса продукта (Product) для обработки этих вариантов.

Диаграммы последовательности

Объектно-ориентированный стиль предназначен для работы с большим количеством небольших объектов, обладающих множеством небольших методов, что дает широкие возможности для переопределения и изменения.

Этот стиль сбивает с толку людей, применяющих длинные процедуры; действительно это изменение является сердцем смены парадигмы (paradigm shift) при объектной ориентации.

Научить этому трудно, возможно, что единственный способ действительно понять это заключается в использовании распределенного управления при работе в объектно-ориентированном окружении.

Диаграммы последовательности

В диаграммах последовательности для создания и удаления участников применяются некоторые дополнительные обозначения.

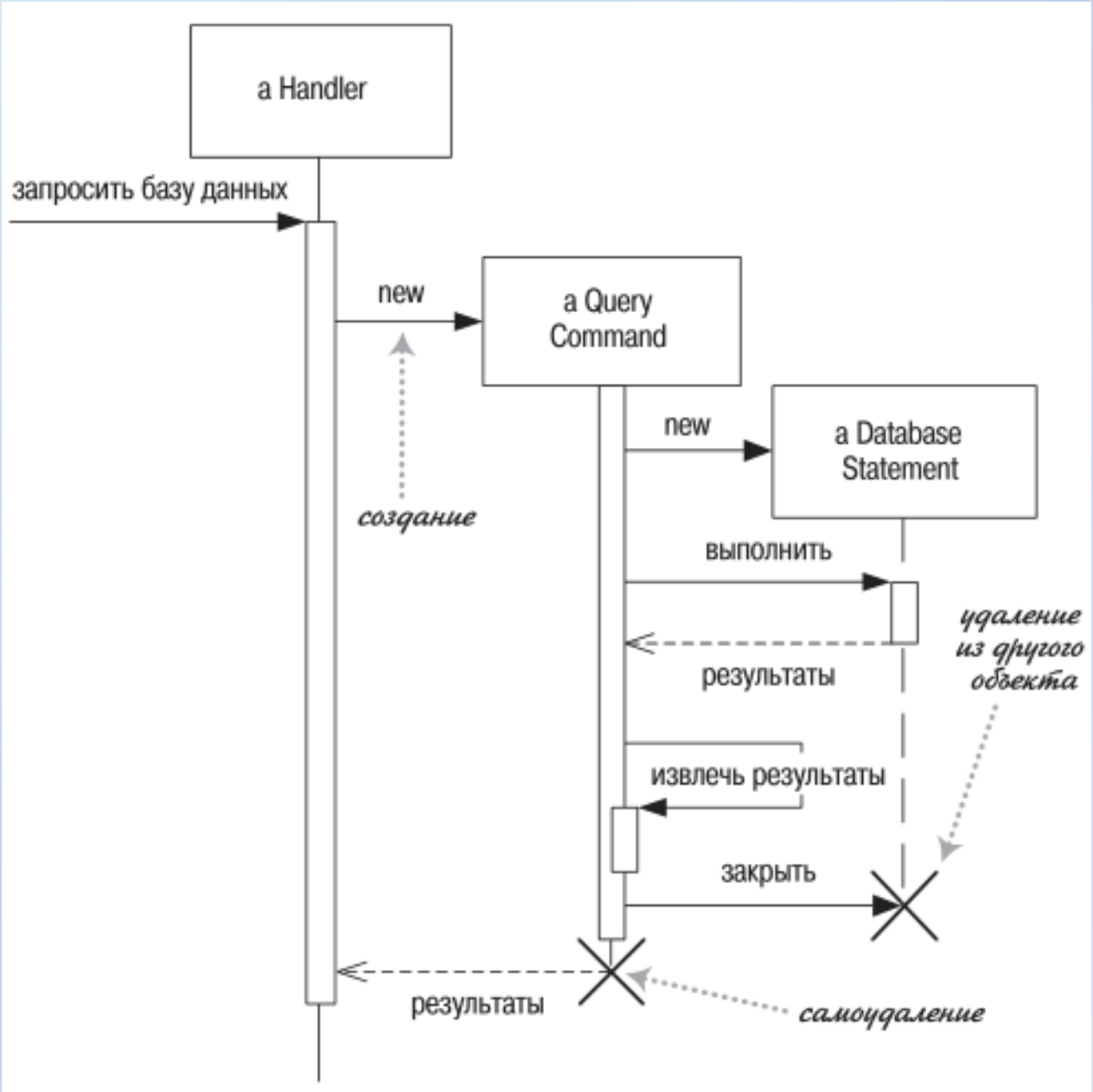
В случае создания участника надо нарисовать стрелку сообщения, направленную к прямоугольнику участника.

Удаление участника обозначается большим крестом (X). Стрелка сообщения, идущая в X, означает, что один участник явным образом удаляет другого; X в конце линии жизни показывает, что участник удаляет сам себя.

Если в системе работает сборщик мусора, то объекты не удаляются вручную, тем не менее следует при помощи X показать, что объект больше не нужен и готов к удалению.

Так следует поступать и в случае операций закрытия, показывая, что объект больше не используется.

Диаграммы последовательности



Диаграммы последовательности

Общая проблема диаграмм последовательности заключается в том,

как отображать циклы и условные конструкции.

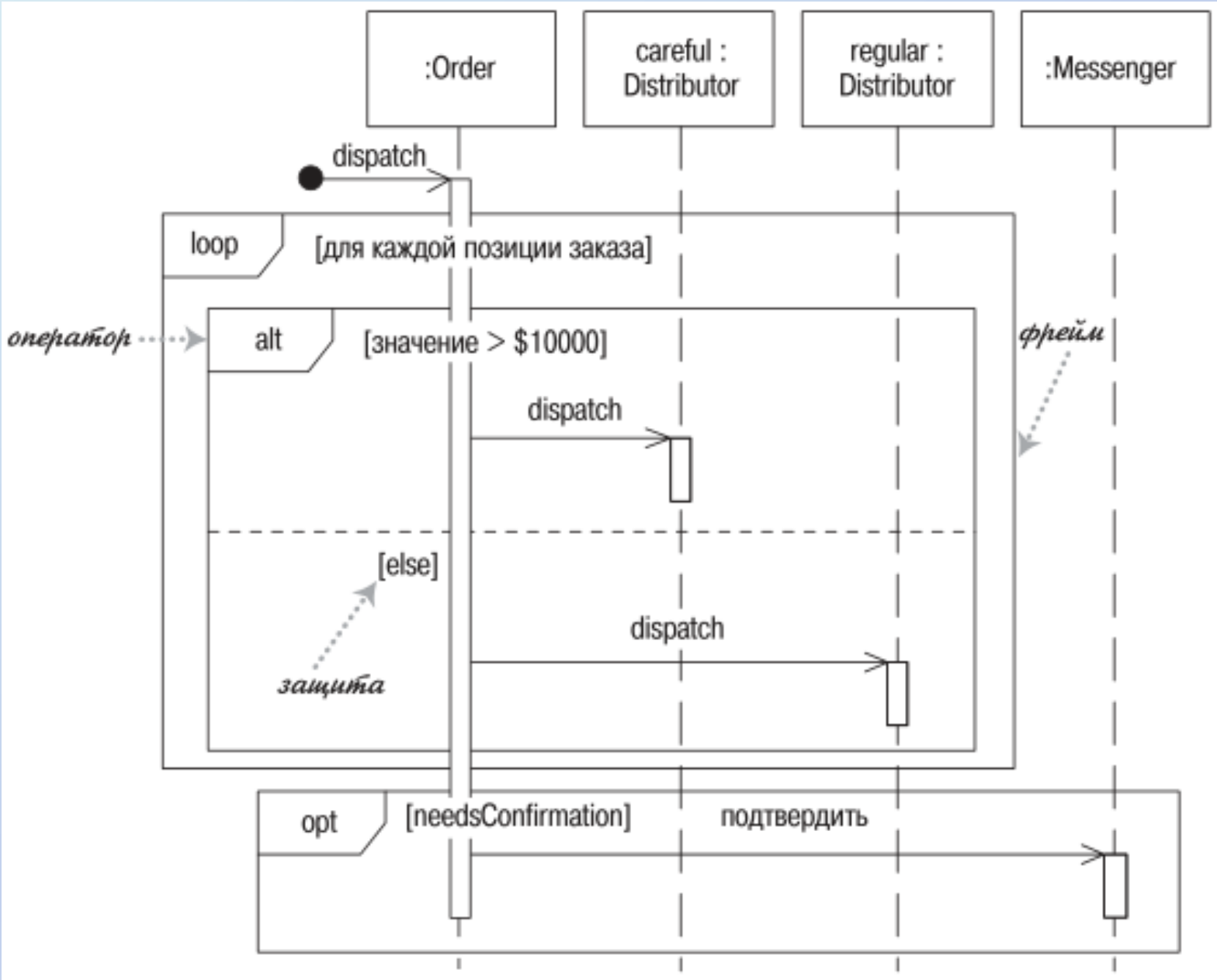
Прежде всего надо усвоить, что диаграммы последовательности для этого не предназначены.

Подобные управляющие структуры лучше показывать с помощью диаграммы деятельности или собственно кода.

Диаграммы последовательности применяются для визуализации процесса взаимодействия объектов, а не как средство моделирования алгоритма управления.

Как было сказано, существуют дополнительные обозначения. И для циклов, и для условий используются фреймы взаимодействий (interaction frames), представляющие собой средство разметки диаграммы взаимодействия.

Диаграммы последовательности



Диаграммы последовательности

В основном фреймы состоят из некоторой области диаграммы последовательности, разделенной на несколько фрагментов.

Каждый фрейм имеет оператор, а каждый фрагмент может иметь защиту.

Ниже в таблице перечислены общепринятые операторы для фреймов взаимодействия.

Для отображения цикла применяется оператор loop с единственным фрагментом, а тело итерации помещается в защиту.

Для условной логики можно использовать оператор alt и помещать условие в каждый фрагмент. Будет выполнен только тот фрагмент, защита которого имеет истинное значение.

Для единственной области существует оператор opt.

Диаграммы последовательности

Оператор	Значение
`alt`	Несколько альтернативных фрагментов (alternative); выполняется только тот фрагмент, условие которого истинно
`opt`	Необязательный (optional) фрагмент; выполняется, только если условие истинно. Эквивалентно `alt` с одной веткой
`par`	Параллельный (parallel); все фрагменты выполняются параллельно
`loop`	Цикл (loop); фрагмент может выполняться несколько раз, а защита обозначает тело итерации
`region`	Критическая область (critical region); фрагмент может иметь только один поток, выполняющийся за один прием
`neg`	Отрицательный (negative) фрагмент; обозначает неверное взаимодействие
`ref`	Ссылка (reference); ссылается на взаимодействие, определенное на другой диаграмме. Фрейм рисуется, чтобы охватить линии жизни, вовлеченные во взаимодействие. Можно определять параметры и возвращать значение
`sd`	Диаграмма последовательности (sequence diagram); используется для очерчивания всей диаграммы последовательности, если это необходимо

Диаграммы последовательности

Фреймы взаимодействия – новинка UML 2. В диаграммах, разработанных до создания UML 2, применяется другой подход; кроме того, некоторые разработчики не любят фреймы и предпочитают прежние соглашения.

Ниже на рисунке показаны некоторые из этих неофициальных приемов.

В UML 1 использовались маркеры итераций и защиты. В качестве маркера итерации (iteration marker) выступал символ *, добавленный к имени сообщения. Для обозначения тела итерации можно добавить текст в квадратных скобках. Защита (guard) – это условное выражение, размещенное в квадратных скобках и означающее, что сообщение посылается, только когда защита принимает истинное значение.

Эти обозначения исключены из UML 2, но они все еще встречаются в диаграммах взаимодействия.

Диаграммы последовательности

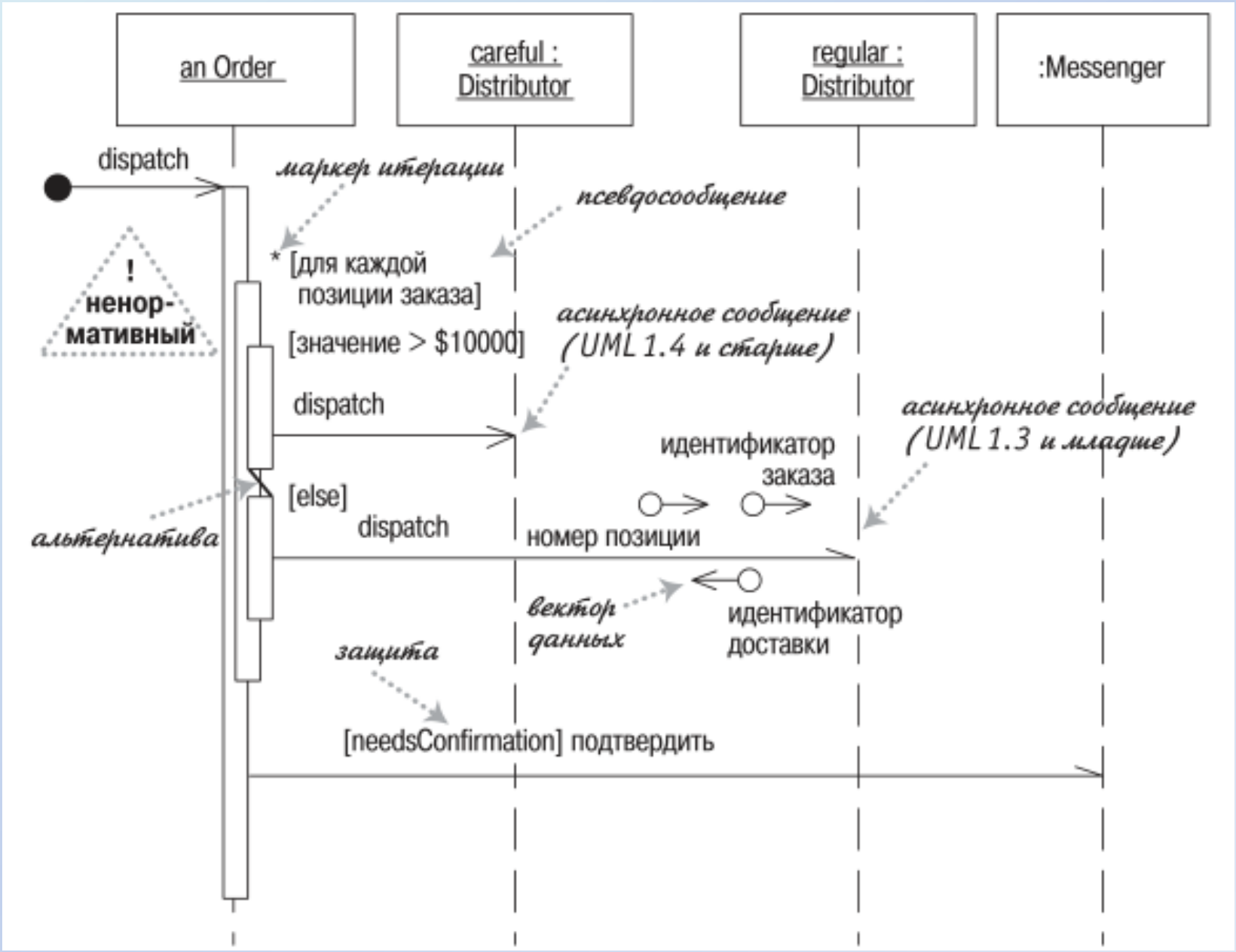
Несмотря на то что маркеры итерации и защиты могут оказаться полезными, они имеют один недостаток.

С помощью защиты нельзя показать, что несколько защит взаимно исключают друг друга.

Оба обозначения работают только в случае отправки одного сообщения и не работают, когда при одной активации посылается несколько сообщений в рамках того же самого цикла или условного блока.

Решением последней проблемы может служить ставшее популярным неофициальное соглашение, заключающееся в применении псевдо-сообщения (pseudomessage) в виде условия цикла или защиты на одном из вариантов обозначения самовызова.

Диаграммы последовательности



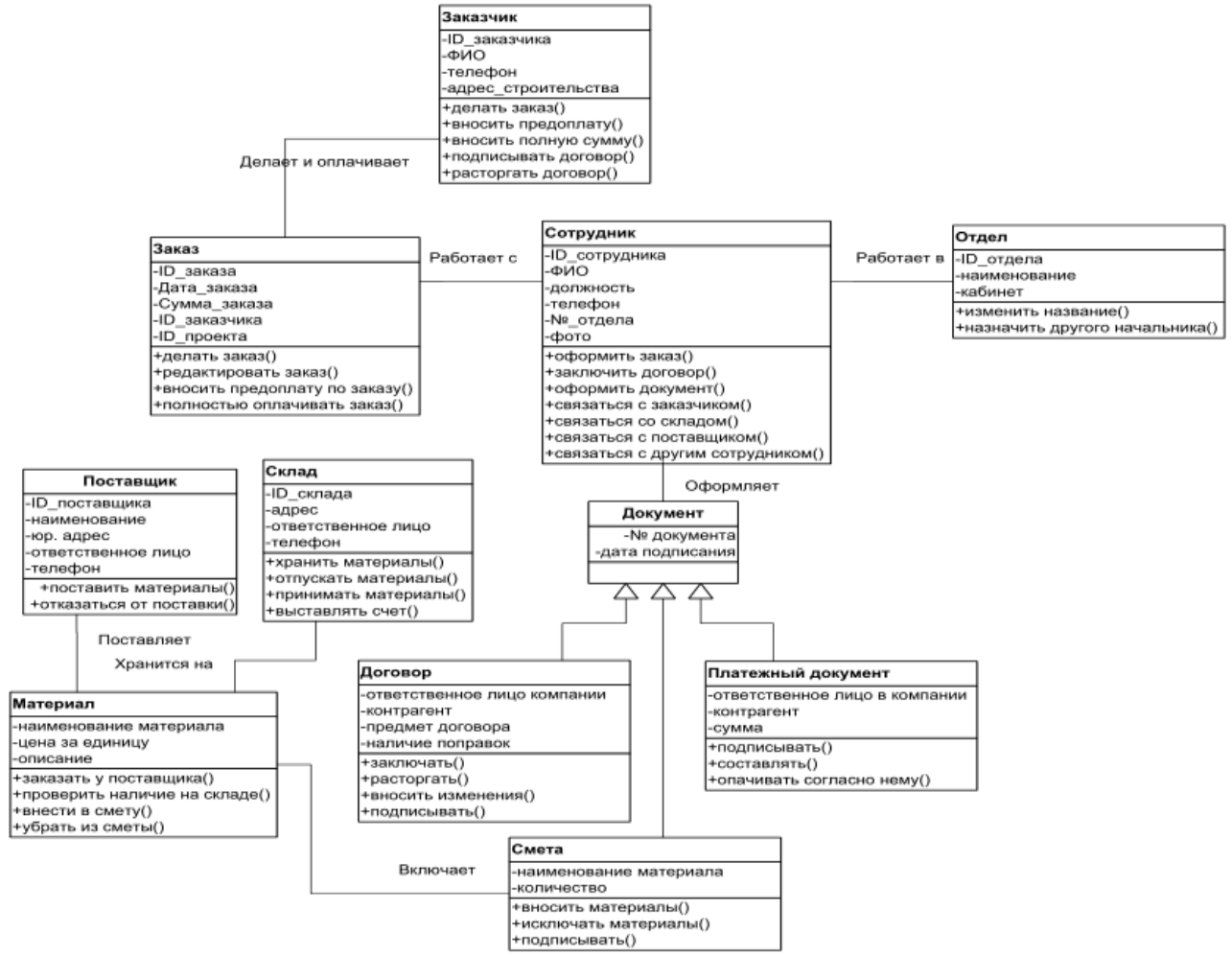
Диаграммы последовательности

Стандарт UML не предоставляет графических средств для обозначения передаваемых данных; вместо этого они показываются с помощью параметров в имени сообщения и на стрелках возврата.

Векторы данных повсеместно использовались во многих областях для обозначения перемещения данных, и многие разработчики все еще с удовольствием применяют их в UML.

В конечном счете, хотя различные системы могут включать в диаграммы последовательности обозначения для условной логики, я не думаю, что они работают сколько-нибудь лучше программного кода или, по крайней мере, псевдокода.

Пример готовой диаграммы классов



Лекция 6

UML (Unified Model Language)
Модели анализа, проектирования

ОСНОВНЫЕ ДИАГРАММЫ

диаграммы деятельности, шаблоны проектирования, диаграммы развертывания, специальные диаграммы.

- Для разработки модели конкретного бизнес-процесса следует использовать диаграмму деятельности (**activity diagram**)

Диаграмма деятельности может быть использована для моделирования

- бизнес-процессов;
- 2. бизнес-правил;
- 3. сценариев работы пользователя с системой
- 5. функций, процессов, алгоритмов системы;
- 4. правил системы;
- 5. сценариев тестирования

Диаграммы деятельности – это технология, позволяющая описывать логику процедур, бизнес-процессы и потоки работ. Во многих случаях они напоминают блок-схемы, но принципиальная разница между диаграммами деятельности и нотацией блок-схем заключается в том, что первые поддерживают параллельное процессы.

Диаграммы деятельности подвергались самым большим изменениям при смене версий языка UML, поэтому неудивительно, что они были снова изменены и существенно расширены в UML 2.

В UML 1 диаграммы деятельности рассматривались как особый случай диаграмм состояний. Это вызвало немало трудностей у специалистов, моделирующих потоки работ, для которых хорошо подходят диаграммы деятельности. В UML 2 это ограничение было ликвидировано.

Узлы на диаграмме деятельности называются операциями (actions), а не активностями (activities). Проще говоря, диаграмма представляет деятельность, состоящую из операций.

Условное поведение схематически обозначается с помощью решений (decisions) и слияний (merges).

Решение, которое в UML 1 называлось ветвью, имеет один входящий поток и несколько защищенных выходных потоков.

Каждый выходной поток имеет защиту – условное выражение, помещенное в квадратные скобки.

Каждый раз при достижении решения выбирается только один из выходных потоков, поэтому защиты должны быть взаимно исключающими. Применение [else] в качестве защиты означает, что поток [else] используется в том случае, когда другие защиты данного решения принимают ложное значение.

Диаграмма деятельности

Диаграмма деятельности

(**activity diagram**) должна иметь только одно начальное состояние. Конечных же состояний может существовать множество. Новые начальные состояния могут быть только на диаграммах, которые декомпозируют отдельные виды деятельности

- Для разработки модели конкретного бизнес-процесса следует использовать диаграмму деятельности (**activity diagram**)

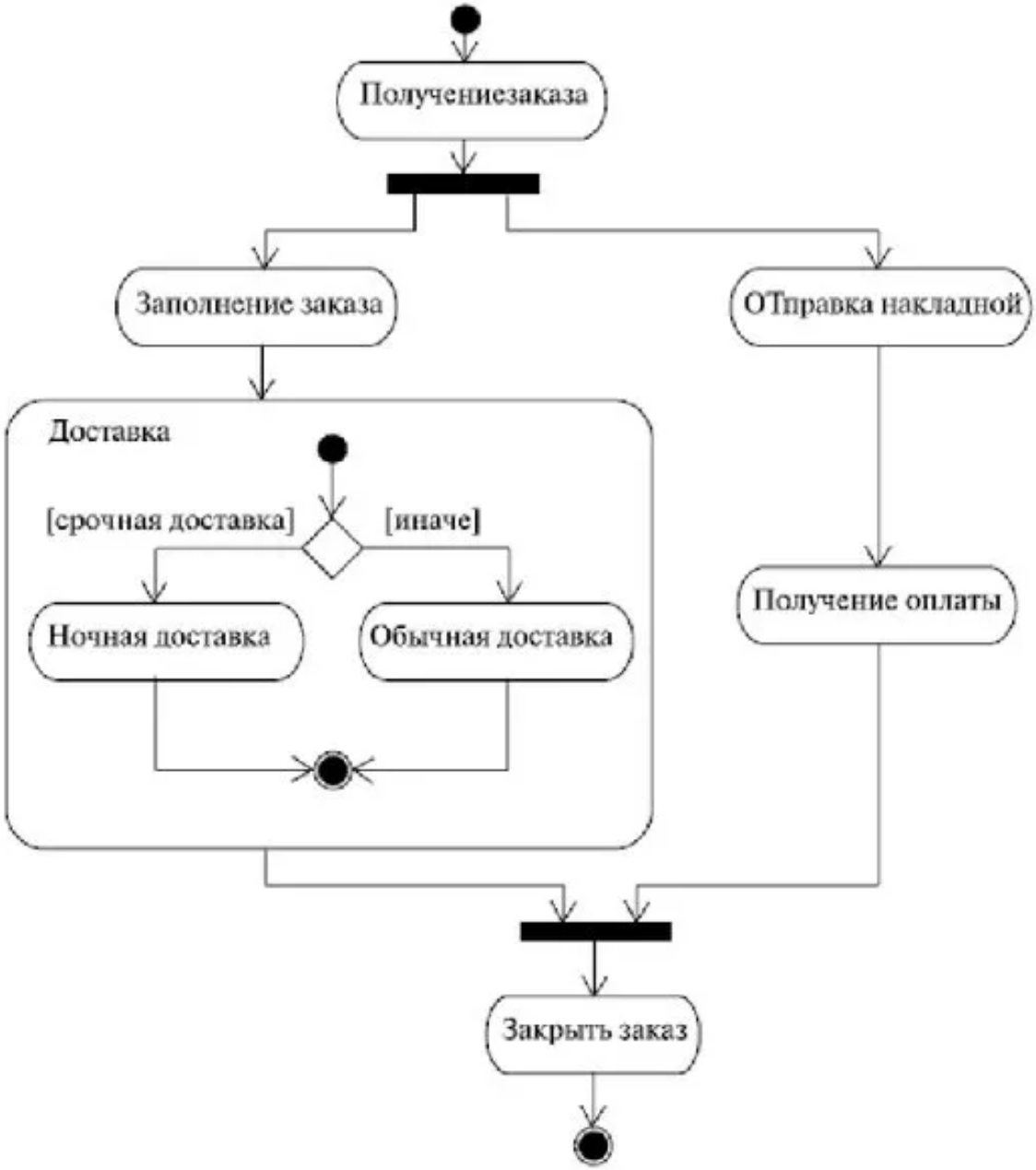
- Начальное состояние (**start state**) должно обозначаться черным маленьким кружком, с которым может быть связано название, например, «начало».



- Конечное состояние (**end state**) должно обозначаться большим черным кружком внутри круга, с которым может быть связано название, например, «конец»



Диаграмма активности (деятельности)



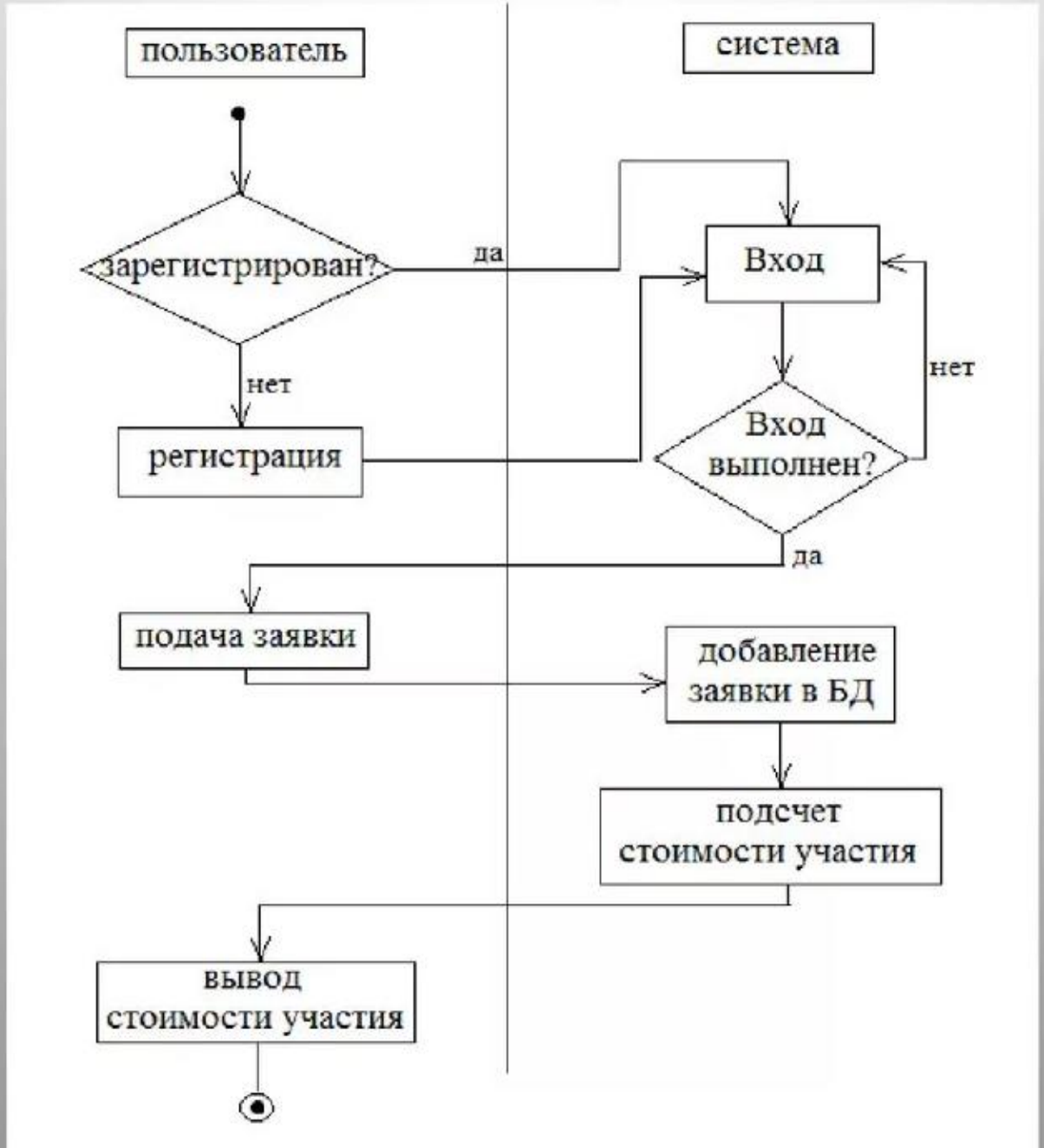
Диаграммы деятельности рассказывают о том, что происходит, но ничего не говорят о том, кто какие действия выполняет. В программировании это означает, что диаграмма не отражает, какой класс является ответственным за ту или иную операцию.

В моделировании бизнес-процессов это означает, что не отражено распределение обязанностей между подразделениями фирмы. Это не всегда представляет собой трудность; часто имеет смысл сконцентрироваться на том, что происходит, а не на том, кто какую роль играет в данном сценарии.

Можно разбить диаграмму деятельности на разделы (partitions), что бы показать, кто что делает, то есть какие операции выполняет тот или иной класс или подразделение предприятия.

На рисунке приведен простой пример, показывающий, как операции при подаче заявки могут быть распределены между пользователем и системой.

Диаграмма деятельности на примере подачи заявки



Пример диаграммы деятельности

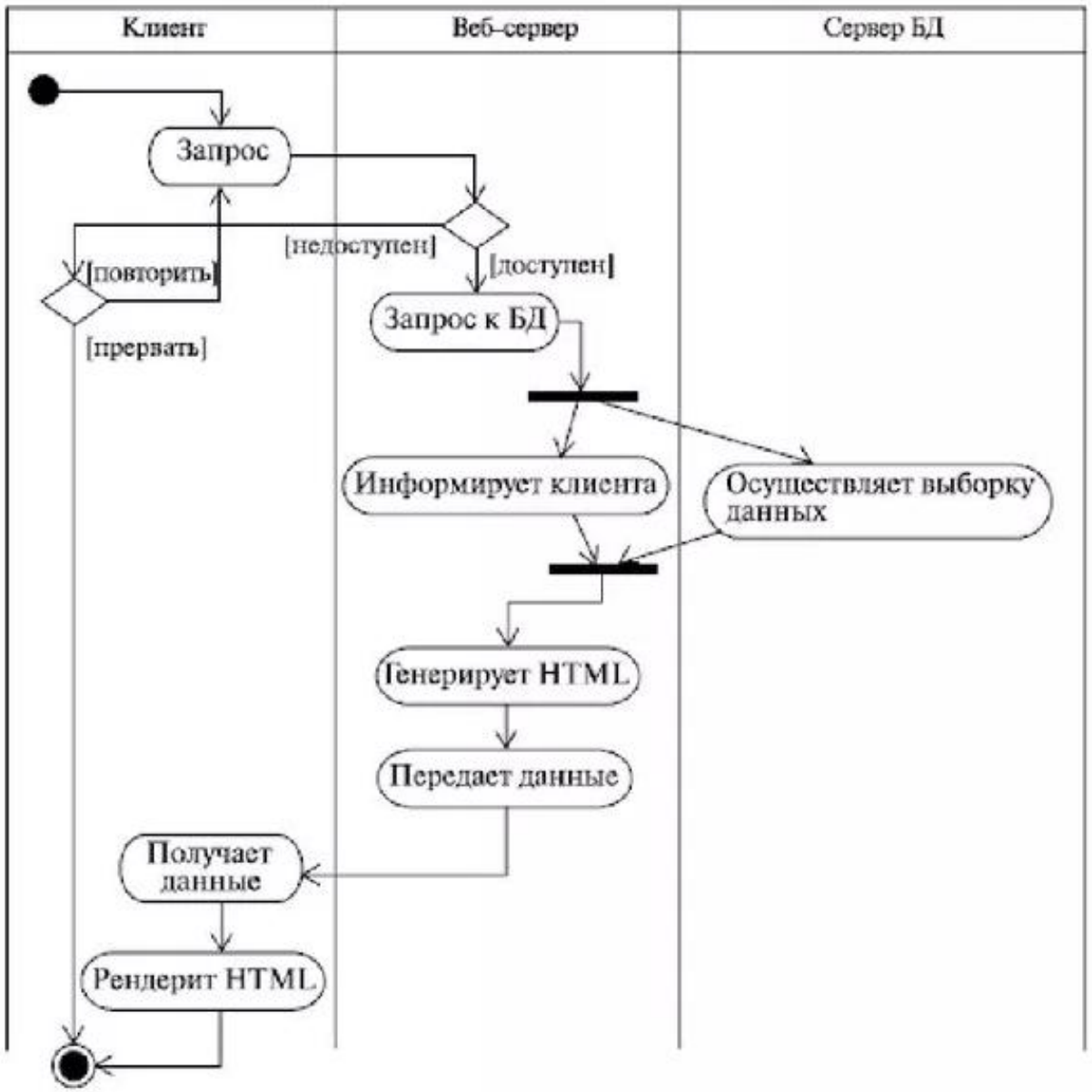


Диаграмма объектов (Objectdiagram)

- – демонстрирует полный или частичный снимок моделируемой системы в заданный момент времени. На диаграмме объектов отображаются экземпляры классов (объекты) системы с указанием текущих значений их атрибутов и связей между объектами.

Диаграмма объектов (Objectdiagram)

- – демонстрирует полный или частичный снимок моделируемой системы в заданный момент времени. На диаграмме объектов отображаются экземпляры классов (объекты) системы с указанием текущих значений их атрибутов и связей между объектами.

Объект (object):

- **Объект (object):**
- конкретная материализация абстракции;
- сущность с хорошо определенными границами, в которой инкапсулированы состояние и поведение;
- экземпляр класса. Объект уникально идентифицируется значениями атрибутов, определяющими его состояние в данный момент времени.

Применение диаграммы объектов

- **Применение диаграммы объектов**
- С помощью диаграмм объектов, как и с помощью диаграмм классов, моделируют статический вид системы с точки зрения проектирования или процессов, но принимая во внимание реальные экземпляры или прототипы. Этот вид описывает главным образом функциональные требования к системе, то есть услуги, которые она должна предоставлять конечным пользователям.
- При моделировании статического вида системы с точки зрения проектирования или процессов объектные диаграммы применяют для того, чтобы моделировать структуру объектов.

Диаграмма состояний

- **Диаграмма состояний** используются для описания поведения, реализуемого в рамках варианта использования, или поведения экземпляров класса (объектов), компонента, узла или системы в целом.
- Диаграммы состояний чаще всего используются для описания поведения отдельных объектов, но также могут быть применены для спецификации функциональности других компонентов моделей, таких как варианты использования, актеры, подсистемы, операции и методы.

Состояние

- **Состояние** – абстрактный класс для моделирования отдельной ситуации. Состояние может быть задано в виде набора конкретных значений атрибутов класса или объекта. Изменение отдельных значений атрибутов будет отражать изменение состояния моделируемого класса или объекта.

Диаграмма компонентов (Componentdiagram

• Диаграмма компонентов (Componentdiagram) — Статическая структурная диаграмма, показывает разбиение программной системы на структурные компоненты и связи (зависимости) между компонентами.

• Компонент (component) —

Реализует некоторый набор интерфейсов и служит для общего обозначения Элементов физического представления модели.

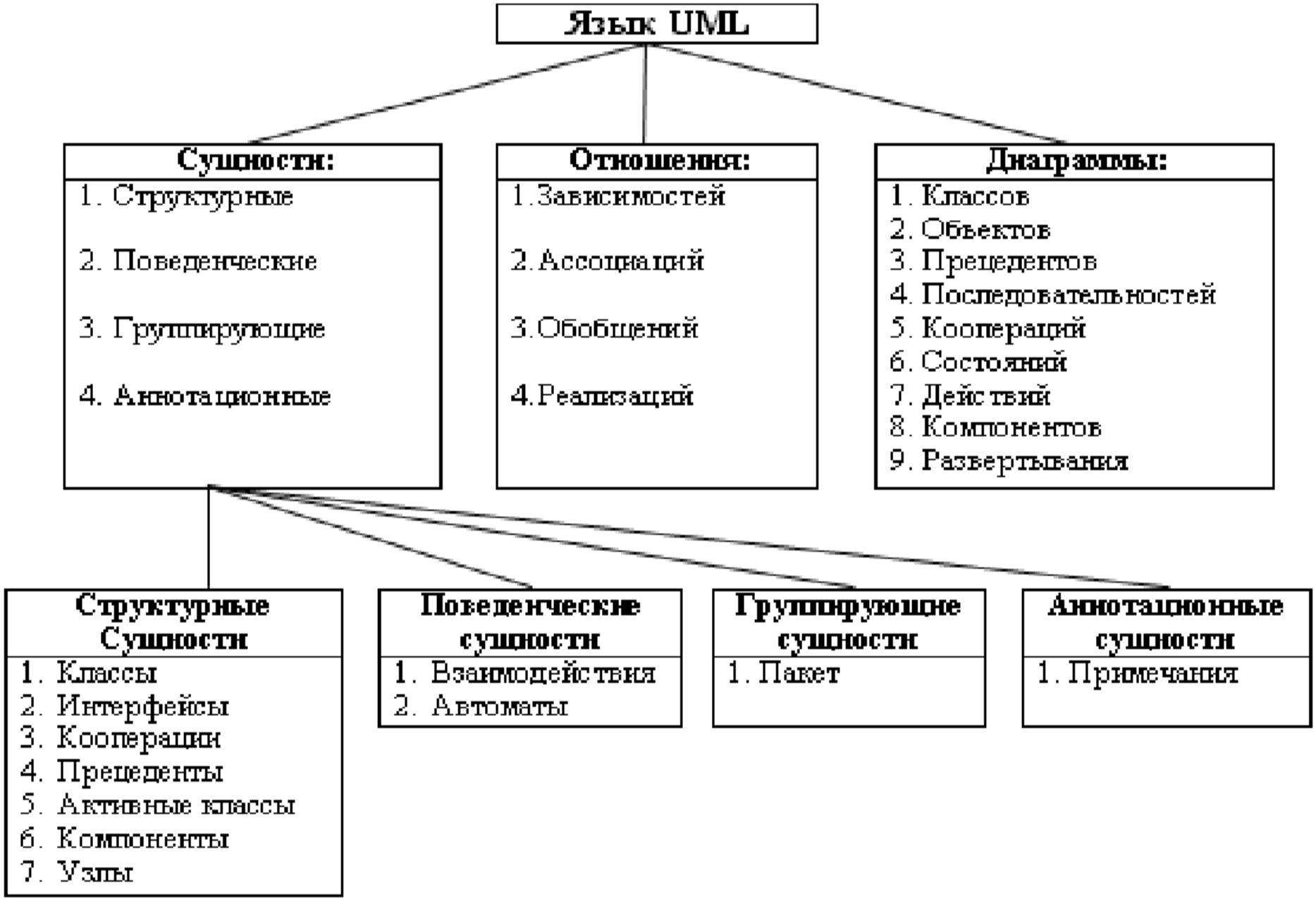
• Для графического представления компонента используется специальный символ — Прямоугольник со вставленными слева двумя более мелкими прямоугольниками.

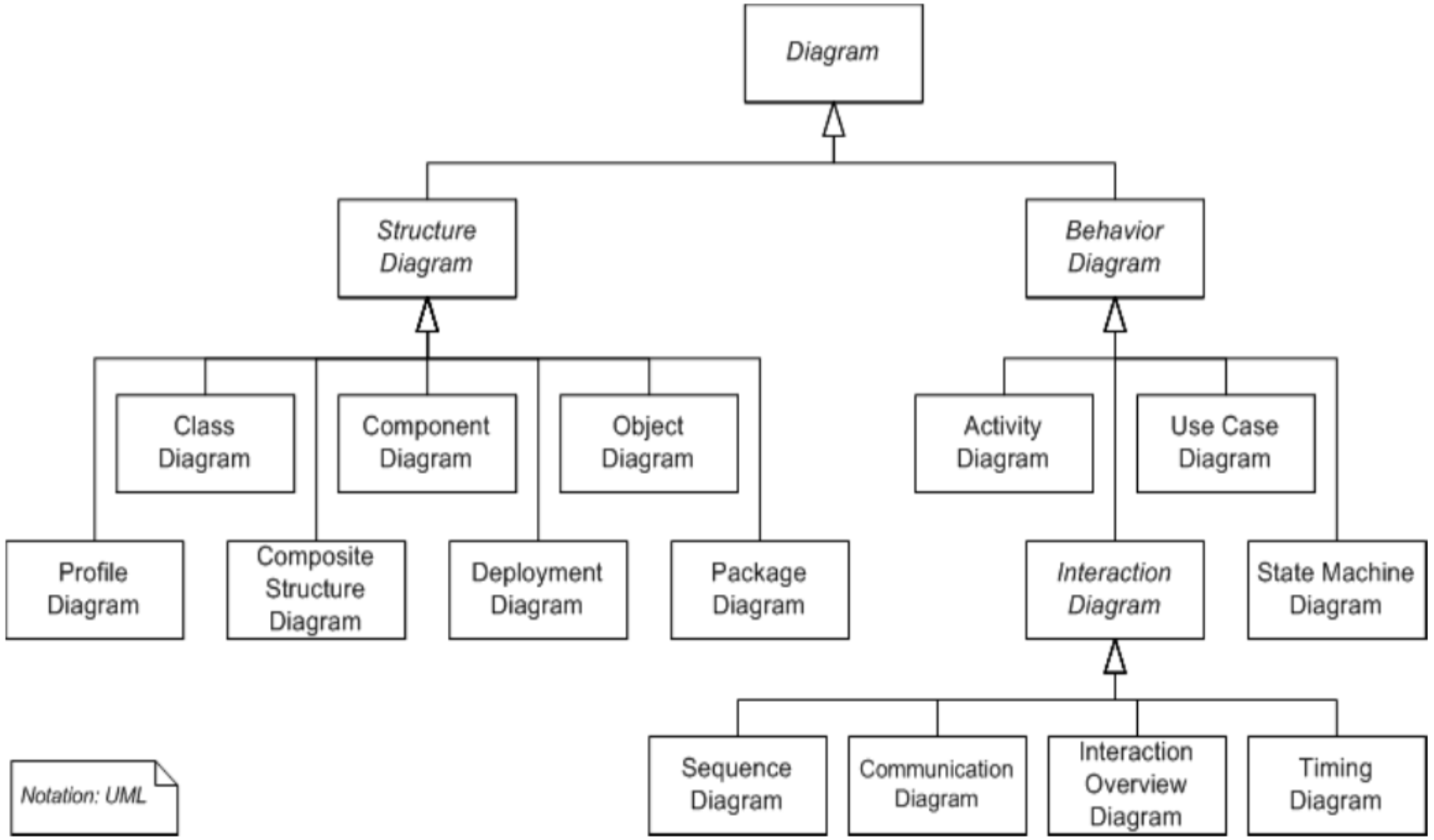
Внутри большого прямоугольника записывается имя компонент а и, при необходимости, некоторая дополнительная информация.

Диаграмма развертывания

- **Диаграмма развертывания - (Deploymentdiagram, диаграмма размещения)** — модель физической архитектуры системы, которая показывает аппаратную конфигурацию ИС.
- **Узел** (node) представляет собой некоторый физически существующий элемент системы, обладающий некоторым вычислительным ресурсом.
- Графически на диаграмме развертывания узел изображается в форме трехмерного куба. Узел имеет собственное имя, которое указывается внутри этого графического символа.

Структура языка UML




Diagram
Structure Diagram
Behavior Diagram
Class Diagram
Component Diagram
Object Diagram
Activity Diagram
Use Case Diagram
Profile Diagram
Composite Structure Diagram
Deployment Diagram
Package Diagram
Interaction Diagram
State Machine Diagram
Sequence Diagram
Communication Diagram
Interaction Overview Diagram
Timing Diagram
Notation: UML

Рекомендуемая последовательность разработки диаграмм:

- диаграмма прецедентов
- диаграмма классов
- диаграмма объектов
- диаграмма последовательностей
- диаграмма кооперацій
- диаграмма состояний
- диаграмма деятельности
- диаграмма компонентов

Выводы по теме:

- UML – унифицированный язык визуального моделирования,который является языком широкого профиля; это открытый стандарт, использующий графические обозначения для создания абстрактной модели системы, называемой UML-моделью. UML был создан для определения, визуализации, проектирования и документирования в основном программных систем. Использование UML не ограничивается моделированием программного обеспечения. Его также используют для моделирования бизнес-процессов, системного проектирования и отображения организационных структур.

Лекция 7. ИССЛЕДОВАНИЕ СИСТЕМ УПРАВЛЕНИЯ

СОДЕРЖАНИЕ ЛЕКЦИИ:

- Аксиомы теории управления.
- Управление как информационный процесс.
- Организационная структура систем с управлением. Виды организационных структур управления.
- Определение понятий качества и эффективности систем.

1. Понятие управления

Управление – процесс взаимодействия субъекта и объекта управления для достижения системой поставленной цели.

Основное содержание понятия управления раскрывают такие его общие функции, как предвидение, организация, контроль, регулирование, координация и активизация (мотивация).

В процессах развития управления возникают новые реальности и новые потребности, которые определенным образом отражаются и на содержании управления.

Сегодня одной из основных функций управления становится функция исследования.

Она является следствием возрастающей динамичности и диверсификации

управления, важным фактором антикризисного управления, повышения роли и значения профессионализма в управлении.

1. Аксиомы теории управления

Аксиома 1. Наличие наблюдаемости объекта управления.

Аксиома 2. Наличие управляемости – способности ОУ переходить в пространстве возможных состояний из текущего состояния в требуемое под воздействием управляющей системы.

Аксиома 3. Наличие цели управления. Под целью управления понимают совокупность значений количественных и качественных показателей, определяющих требуемое состояние ОУ.

Аксиома 4. Свобода выбора – выбор управляющих воздействий из множества возможных альтернативных решений.

Аксиома 5. Наличие критерия эффективности управления. Обобщенным критерием эффективности управления считается степень достижения цели функционирования системы.

Аксиома 6. Наличие ресурсов (материальных, информационных, финансовых, и других), обеспечивающих реализацию принятых решений.

1. Аксиомы теории управления

Аксиома 1. Наличие наблюдаемости объекта управления.

Аксиома 2. Наличие управляемости – способности ОУ переходить в пространстве возможных состояний из текущего состояния в требуемое под воздействием управляющей системы.

Аксиома 3. Наличие цели управления. Под целью управления понимают совокупность значений количественных и качественных показателей, определяющих требуемое состояние ОУ.

Аксиома 4. Свобода выбора – выбор управляющих воздействий из множества возможных альтернативных решений.

Аксиома 5. Наличие критерия эффективности управления. Обобщенным критерием эффективности управления считается степень достижения цели функционирования системы.

Аксиома 6. Наличие ресурсов (материальных, информационных, финансовых, и других), обеспечивающих реализацию принятых решений.

1. Понятие системы управления

Система управления представляет собой совокупность взаимосвязанных управляющих и управляемых элементов, которые находятся во взаимной зависимости и постоянно взаимодействуют друг с другом для достижения единой общей цели. Система управления по своим характеристикам может быть отнесена к различным классам систем:

- **социальным,**
- **сложным,**
- **открытым,**
- **самоорганизующимся,**
- **саморазвивающимся,**
- **адаптирующимся,**
- **многоуровневым,**
- **иерархическим,**
- **стохастически-детерминированным,**
- **активным,**
- **полиструктурным,**
- **многоцелевым,**
- **динамическим.**

1. Понятие системы управления

Следует особо выделить такие **свойства систем управления**, как:

– уникальность,

– слабопредсказуемость,

– целенаправленность,

– способность к самообучению и адаптации.

Для эффективного управления система управления должна быть сложнее управляемого ею объекта

Систему управления и ее структуру можно рассматривать в различных аспектах, каждый из которых имеет свою специфику исследования.

Система управления – организационная система, состоящая из двух подсистем: управляющей (субъект управления) и управляемой (объект управления), прямых и обратных связей между ними.

В системах управления всегда протекают реальные процессы, а значит, существуют и материальные, и информационные потоки, вход и выход.

Общая схема процесса управления



1. Общая схема процесса управления

1 – управляющие воздействия и информация по прямому каналу связи;

2 – информация о состоянии управляемой подсистемы по обратному каналу связи;

3 – ресурсы на входе системы;

4 – выход системы (продукция, услуги, выполненные обязательства;

5 – внешние задающие воздействия;

6 – исходящая информация в системы более высокого иерархического уровня;

7 – исходящая информация и документация во внешнюю среду;

8 – воздействия внешней среды;

9 – информация о внешней среде;

10 – исходящая информация и задающие воздействия в системы более низкого иерархического уровня;

11 – входящая информация из систем более низкого иерархического уровня)

Понятие и классификация систем управления

Воздействие, вызывающее изменение состояния целеустремленной системы, ведущее к достижению ее цели, называется управлением.

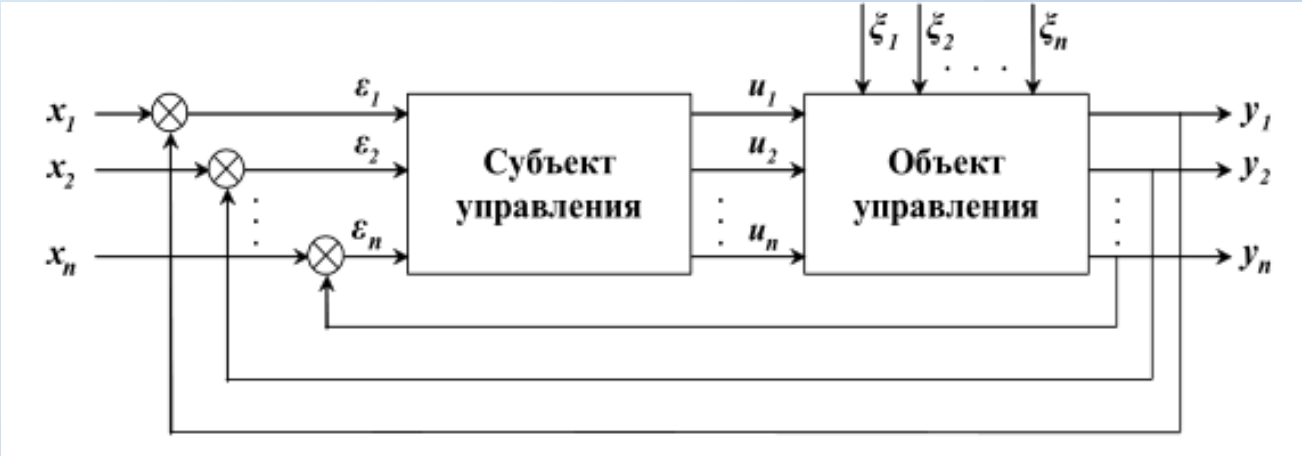
Система управления – совокупность объекта и субъекта управления, действие которой направлено на поддержание или улучшение работы объекта управления.

Объект управления – система, в которой происходят процессы, подлежащие управлению.

Субъект (орган) управления – система, осуществляющая сбор, накопление, обработку и передачу информации и формирование управляющих сигналов.

Технология управления – совокупность методов и процессов, необходимых для достижения цели управления.

Для системы управления характерно единство субъекта и объекта управления – ее управляющей и управляемой частей, которое обеспечивается наличием между ними прямых и обратных связей, образующих в своей совокупности контур управления



Под влиянием управляющего воздействия ($\boldsymbol{u}=(u_1, \ldots, u_n)$), вырабатываемого субъектом управления, в объекте управления происходят изменения, результаты которых отражаются на численных значениях его измеряемых параметров. Среди всего множества параметров, по значениям которых можно судить о состоянии объекта управления, выделяют входные (x_i) и выходные (y_i), управляемые и неуправляемые ("возмущения" – ξ_i).

Управляющее воздействие $\boldsymbol{u}=(u_1, \ldots, u_n)$ является функцией величины $\varepsilon=(\varepsilon_1, \ldots, \varepsilon_n)$, равной разности между задающим воздействием (входным сигналом) $\boldsymbol{x}=(x_1, \ldots, x_n)$ и реакцией системы $\boldsymbol{y}=(y_1, \ldots, y_n)$.

1. Аксиомы теории управления

1 – управляющие воздействия и информация по прямому каналу связи;

2 – информация о состоянии управляемой подсистемы по обратному каналу связи;

3 – ресурсы на входе системы;

4 – выход системы (продукция, услуги, выполненные обязательства;

5 – внешние задающие воздействия;

6 – исходящая информация в системы более высокого иерархического уровня;

7 – исходящая информация и документация во внешнюю среду;

8 – воздействия внешней среды;

9 – информация о внешней среде;

10 – исходящая информация и задающие воздействия в системы более низкого иерархического уровня;

11 – входящая информация из систем более низкого иерархического уровня)

Исследование **– это вид деятельности состоящий в:**

- **распознавании проблемы;**
- **определение ее происхождения, содержания и свойств;**
- **установление ее связей;**
- **нахождение путей и средств разрешения проблемы.**

1. Понятие Исследования

Исследование — это вид деятельности человека, состоящий в:

1) распознавании проблем и ситуаций;

2) определении их происхождения;

3) выявлении свойств, содержания, закономерностей поведения и развития;

4) установлении места этих проблем и ситуаций в системе накопленных знаний;

5) нахождении путей, средств и возможностей использования новых представлений или знаний о данной проблеме в практике ее разрешения (см. схему 2).

В реальной практике эти признаки исследования находятся в определенном соотношении, характеризующем и степень профессионализма, и конкретные цели и задачи деятельности.

Исследование — это более широкий вид деятельности чем анализ, проектирование или диагностика. Исследование проблемы или ситуации включает в себя более широкий набор методов, чем анализ или проектирование. Это и наблюдение, и оценка, и проведение эксперимента, и классификация, и построение показателей, и многое другое. Исследование также включает, в себя и анализ, но не сводится к нему.

Таким образом **исследование представляет собой более высокий уровень творческой деятельности человека.**

Характеристики исследования

Любое исследование имеет комплекс характеристик, которые необходимо учитывать при его проведении и организации. Основными из этих характеристик являются следующие (см. схему):

1. Методология исследования — совокупность целей, подходов, ориентиров, приоритетов, средств и методов исследования.

2. Организация исследования — порядок проведения, основанный на распределении функций и ответственности, закрепленных в регламентах, нормативах и инструкциях.

3. Ресурсы исследования — комплекс средств и возможностей (например, информационных, экономических, людских и пр.), обеспечивающих успешное проведение исследования и достижение его результатов.

4. Объект и предмет исследования. Объектом является система управления, относящаяся к классу социально-экономических систем, предметом — конкретная проблема, разрешение которой требует проведения исследования.

5. Тип исследования — принадлежность его к определенному типу, отражающему своеобразие всех характеристик.

Характеристики исследования

Любое исследование имеет комплекс характеристик, которые необходимо учитывать при его проведении и организации. Основными из этих характеристик являются следующие (см. схему):

6. Потребность исследования — степень остроты проблемы, профессионализма в подходах к ее решению, стиль управления.

7. Результат исследования — рекомендации, модель, формула, методика, способствующие успешному разрешению проблемы, пониманию ее содержания, истоков и последствий. Результаты исследования могут иметь различный вид. Они бывают непосредственными в виде конкретных рецептов, рекомендаций, моделей, инновационных программ, стратегий, методик, систем мотивации и пр. В то же время результаты могут быть и опосредованными. Такими являются, например, имидж фирмы, социально-психологическая атмосфера работы, антикризисное развитие и пр. Результаты исследования могут основными и дополнительными.

8. Эффективность исследования — соразмерность использованных ресурсов на проведение исследования и результатов, полученных от него.

Основные характеристики исследования



Этапы исследования систем управления



Информационное обеспечение исследования

Информация – некоторое сообщение, которое тем или иным способом уменьшает незнание потребителя (приемника) о некотором объекте, факте, явлении, проблеме.

Виды информации различаются:

— формой представления;
— организацией хранения;
— характером обработки.

Формы представления информации:

— символьно-текстовая (с помощью букв, цифр, знаков и т.п.);
— графическая (изображение и т.п.);
— звуковая (фонетическая).

Требования к информации:

— корректность,
— ценность,
— достоверность,
— точность,
— актуальность,
— полнота.

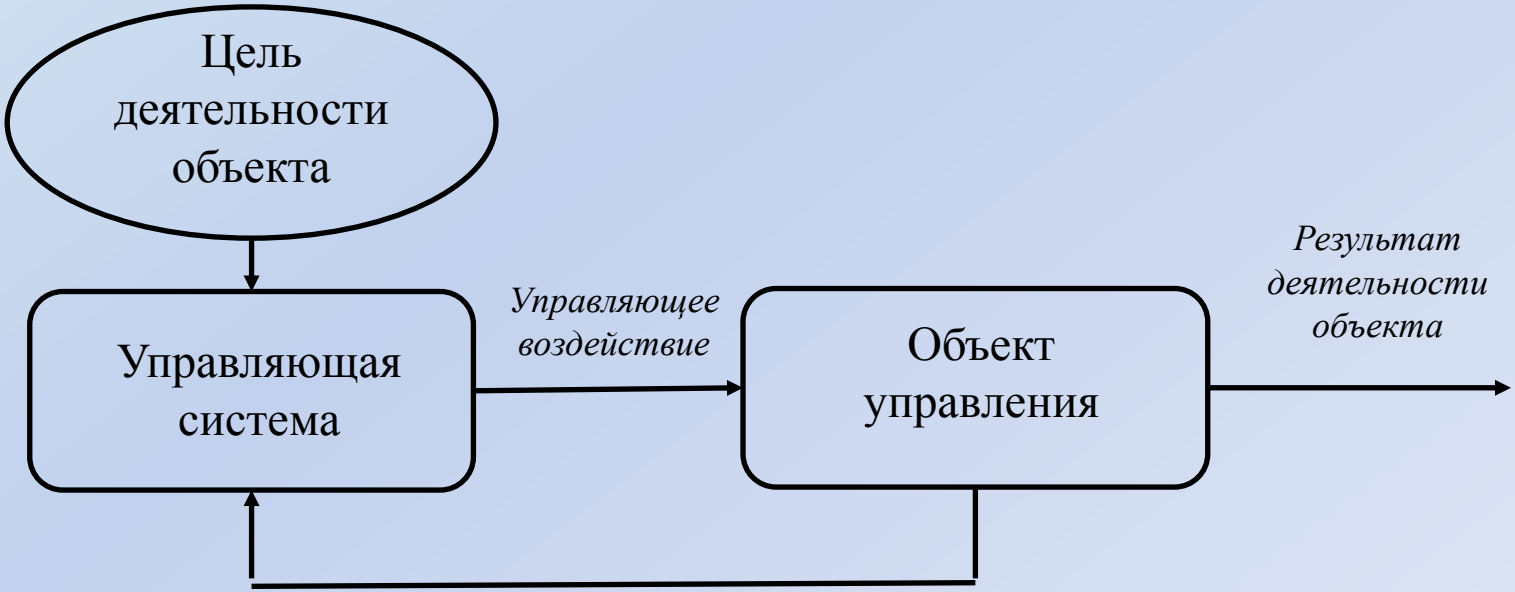
2. Управление как информационный процесс

Управление по своей сущности является информационным процессом

- *сбор информации*
- *Хранение информации*
- *Обработка информации*
- *Выработка управленческого решения*

Схема процессов управления

Функция управления - это обособившийся вид управленческой деятельности.



Информационное обеспечение исследования

Функциональным свойством информации является свойство системности.

Содержательная классификация информации зависит от отраслевой принадлежности и уровня управления.

Элементы передачи информации:

1. Источник (передатчик).
2. Приемник (потребитель, клиент).

Цикл информационного процесса включает:

1. Входная информация (директивная, осведомительная, первичная).
2. Внутренняя информация (плановая, учетная, нормативно-справочная).
3. Выходная информация (управляющая, отчетная, статистическая).

Этапы информационного процесса:

источник,

— сбор,
— преобразование,
— передача,
— обработка,
— хранение,
— тиражирование
— распространение.

Информационное обеспечение исследования

Функциональным свойством информации является свойство системности.

Содержательная классификация информации зависит от отраслевой принадлежности и уровня управления.

Информация фирмы необходимая для ИСУ:

1. Сведения о потребности и наличии ресурсов (кадровый состав, основные и оборотные средства, информация и др.).
2. Техническая характеристика средств производства и оборудования.
3. Описание технологии и условий производства.
4. Состояние рыночной конъюнктуры (поставщики, потребители, конкуренты, цены, объем спроса и др.).
5. Нормативные акты, плановые задания, учетные и отчетные документы, приказы, распоряжения, положения, инструкции, методики и др.
6. Совокупность расчетных показателей (прибыль, рентабельность, себестоимость продукции, ликвидность, финансовая устойчивость, фондоемкость, фондоотдача, оборачиваемость оборотных средств, текучесть кадров, норма управляемости, производительность труда и др.).

Требования к совершенствованию систем управления:

Сокращение длительности цикла управления.

Повышение качества управляющих воздействий (решений).

Пути совершенствования систем с управлением:

•оптимизация численности управленческого персонала;

•новые способы организации работы СУ;

•новые методы решения управленческих задач;

•изменение структуры СУ;

•перераспределение функций и задач в УС;

•механизация управленческого труда;

•автоматизация.

ИССЛЕДОВАНИЕ –
это вид деятельности, состоящий в . . .

- **распознавании проблем и ситуаций,**
- **определении их происхождения,**
- **выявлении свойств, содержания и закономерностей их поведения и развития,**
- **установление их места в системе накопленных знаний**
- **нахождение путей и средств использования полученных знаний для разрешения проблемы**

Классификационный признак исследования	Вид исследования
Направленность на результат	Фундаментальное, прикладное
Решаемая задача	Текущее (оперативное), перспективное (стратегическое)
Продолжительность проведения	Экспресс, краткосрочное (непродолжительное), долгосрочное (продолжительные)
Емкость используемых ресурсов	Малоресурсоемкое, среднересурсоемкое, ресурсоемкое
Используемая информация	Внутреннеинформационное, внешнеинформационное, концептуальное
Число исследователей	Индивидуальное, коллективное
Организованность	Спонтанное, организованное
Полнота изучения и число исследуемых объектов	Тематическое, общее; частное, общее; локальное, глобальное
Регулярность проведения исследований	Регулярное (периодическое), эпизодическое, разовое
Сложность	Простое, сложное
Подход к исследуемому объекту	Локальное, системное
Место проведения	Лабораторное, промышленное
Наличие модели исследования	Натурное (реальное), модельное
Характер объекта	Геополитическое, политическое, социологическое, экономическое, техническое, технологическое и др.
Отношение к периоду функционированию объекта	Ретроспективное (предшествующее), реально-временное, прогнозное (последующее)
Методология проведения	Контрольное, диагностическое, сравнительное, классификационное и др.
Характер исследования	Поисковое, научно-исследовательское, опытно-промышленное
Отношение к методу исследования	**Теоретическое** (позволяющее объяснить ранее открытые факты и эмпирические закономерности на базе имеющейся теории — комплексе взглядов, представлений, идей, и дающее возможность определять и формулировать общие для исследуемого предмета новые законы, закономерности, принципы и т.п.) **Эмпирическое** (позволяющее на базе установленных опытным путем новых фактов и их последующего обобщения сформулировать новые знания и прогнозы)
Форма ожидаемого результата	Количественное, качественное

Для комплексного описания предприятий используют модели:

организационная

Отражает состав и взаимосвязь структурных элементов (подразделений)

функциональная

Отражает процессы, операции, действия, связанные с реализацией отдельных функций предприятия

информационная

Отражает информационные процессы и потоки

3. Организационная структура систем с управлением

Организационная структура системы управления предприятием – это разделение предприятия на подразделения с целью упорядочения взаимодействия звеньев, установления подчиненности и полномочий.

Организационная структура системы управления предприятием может быть линейной, линейно-штабная, дивизионной, бригадной, проектной, матричной, многомерной.

Виды организационных структур управления

Линейная структура имеет несколько уровней руководства. Подразделения подчиняются одному вышестоящему органу управления. Данный орган самостоятельно выполняет управленческие функции. Структура характерна для производственных подразделений.

Линейно-штабная организационная структура является развитием линейной и призвана ликвидировать ее важнейший недостаток, связанный с отсутствием звеньев стратегического планирования. Линейно - штабная структура включает в себя специализированные подразделения (штабы), которые не обладают правами принятия решений и руководства какими – либо нижестоящими подразделениями, а лишь помогают соответствующему руководителю в выполнении отдельных функций, прежде всего, функций стратегического планирования и анализа.

Ключевыми фигурами в управлении организациями с ***дивизионной структурой*** *являются уже не руководители функциональных подразделений, а менеджеры, возглавляющие производственные отделения (дивизионы).*

Структуризация по дивизионам, как правило, производится по одному из критериев: по выпускаемой продукции (изделиям или услугам) – продуктовая специализация; по ориентации на определенные группы потребителей - потребительская специализация; по обслуживаемым территориям - региональная специализация.

Основой этой структуры управления является организация работ по рабочим группам ***(бригадам)****. Основными принципами такой организации управления являются:*

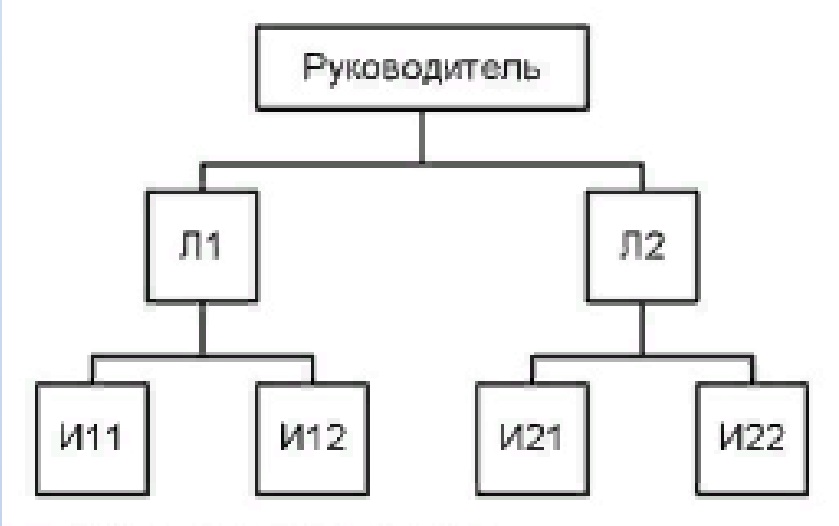
- *автономная работа рабочих групп (бригад);*
- *самостоятельное принятие решений рабочими группами и координация деятельности по горизонтали;*

Проектная структура системы управления на предприятии создается для решения конкретной задачи. Суть ее заключается в том, что исполнители по всем сферам деятельности предприятия непосредственно подчинены руководителю проекта. Это временная структура. Когда проект завершен, команда исполнителей распадается.

В матричной организации члены проектной группы подчиняются руководителю проекта и руководителям функциональных отделов, в которых они работают. Руководитель проекта обладает проектными полномочиями и отвечает за интеграцию всех видов деятельности и ресурсов, относящихся к проекту.

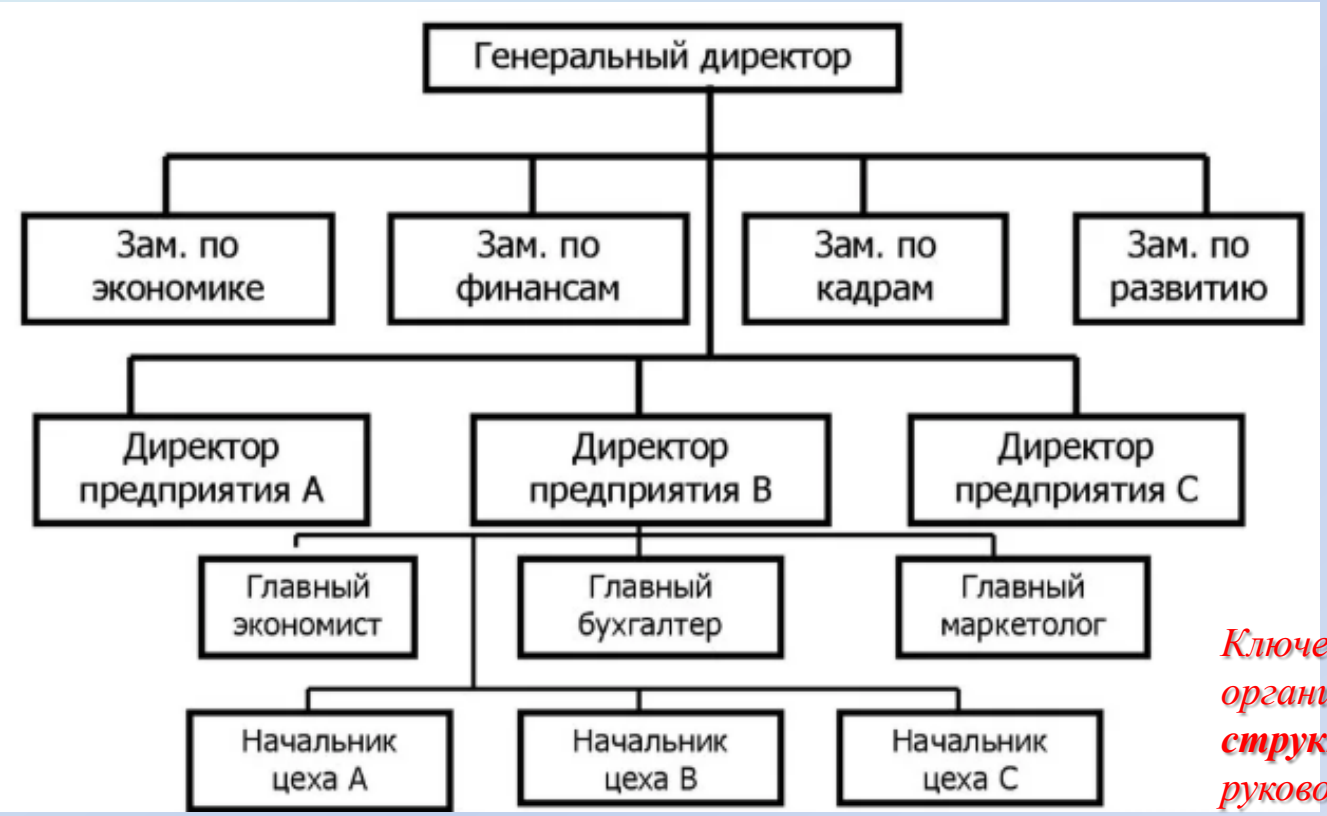
Многомерная организационная структура. Традиционно организационная структура охватывает два вида взаимоотношений: ответственность (кто за что отвечает) и подчинение (кто перед кем отчитывается). Организация с такой структурой может быть представлена в виде дерева, при этом обязанности изображаются прямоугольниками, относительное расположение которых показывает уровень полномочий, а линии, соединяющие эти прямоугольники, - распределение полномочий.

Линейная организационная структура



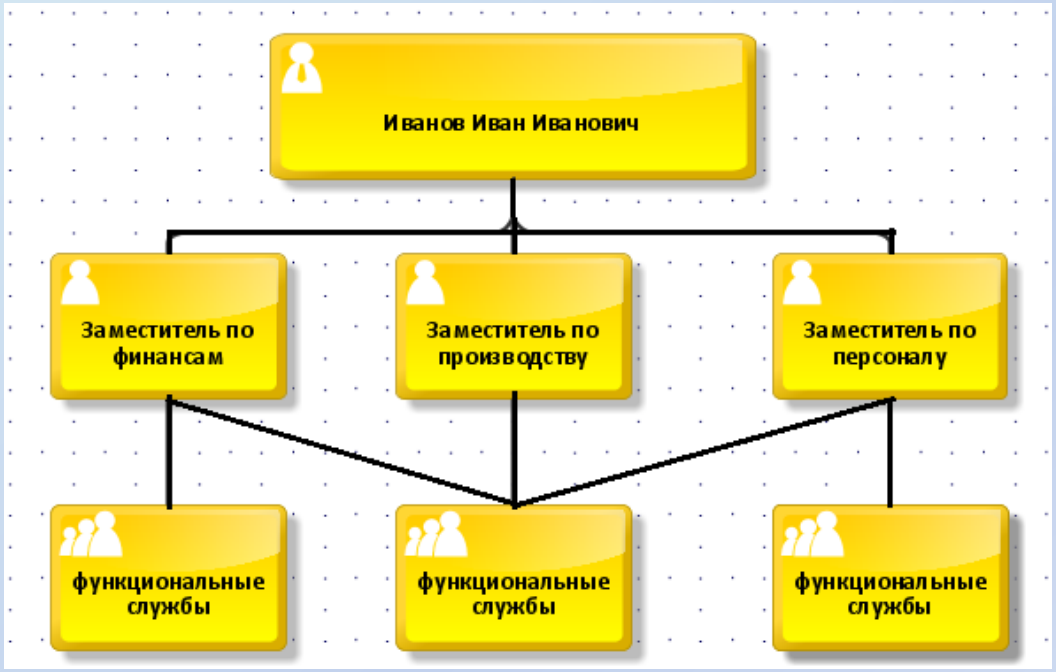
Л1, Л2 – линейные руководители
И11,И12 – исполнители первого подразделения
И21, И22 – исполнители второго подразделения

Дивизионная структура управления



*Ключевыми фигурами в управлении организациями с **дивизионной структурой** являются уже не руководители функциональных подразделений, а менеджеры, возглавляющие производственные отделения (дивизионы).*

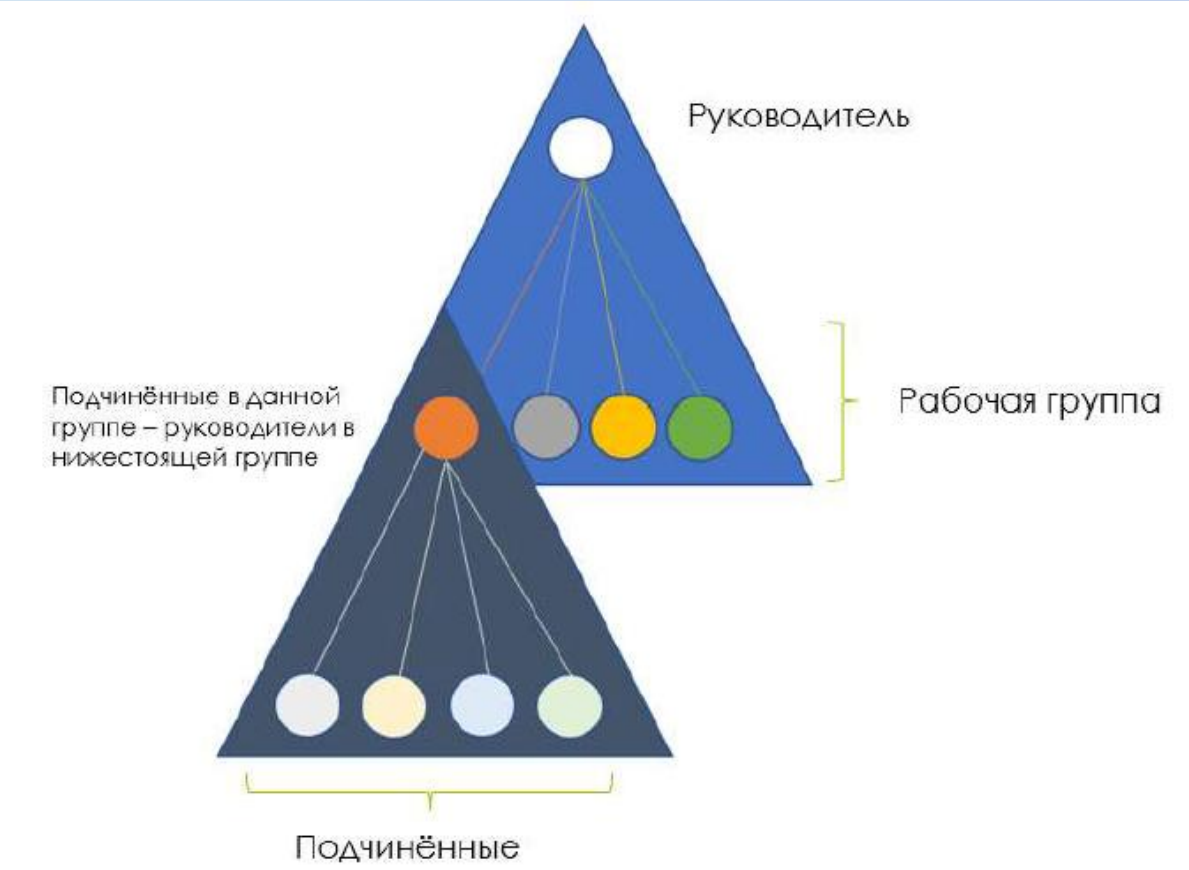
Бригадная (кросс - функциональная) структура управления



- автономная работа рабочих групп (бригад);

- самостоятельное принятие решений рабочими группами и координация деятельности по горизонтали.

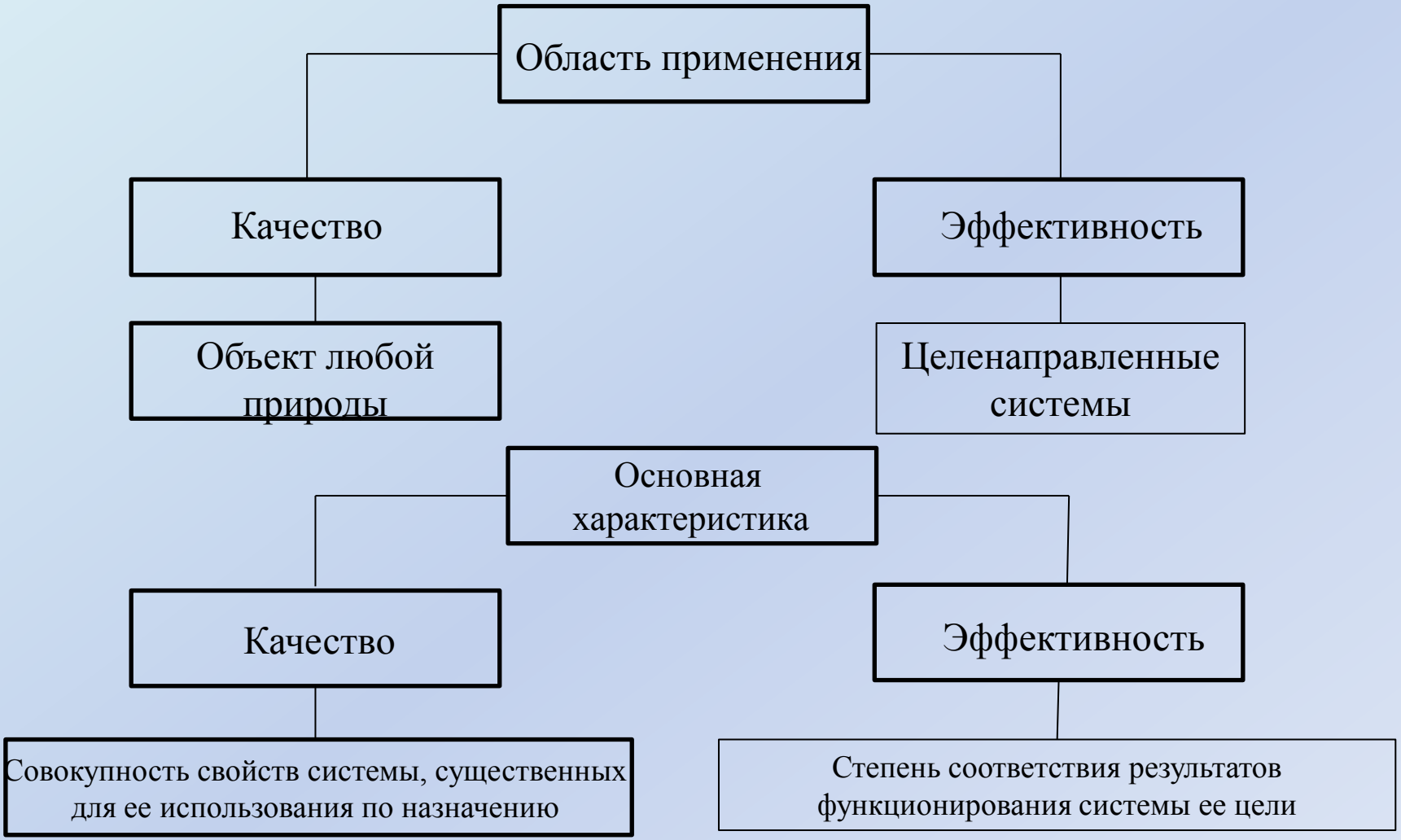
Структура организации, состоящей из рабочих групп (бригадная)

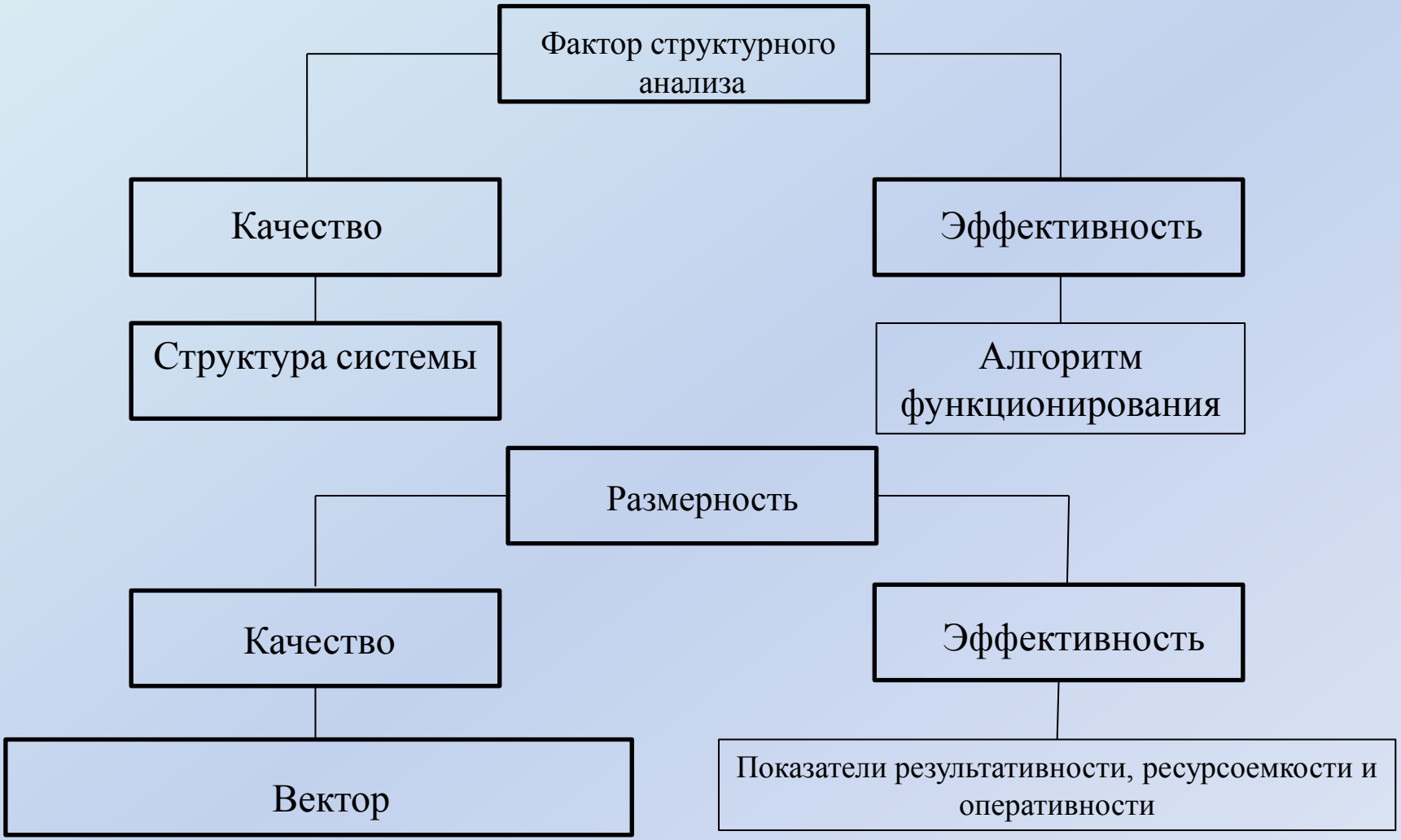


4. Определение понятий качества и эффективности систем

Качество - свойства или совокупность свойств системы, обусловливающих ее пригодность для использования по назначению.

Эффективность - свойство процессов функционирования системы, характеризующее его приспособленность к достижению цели системы


Область применения
Качество
Эффективность
Объект любой природы
Целенаправленные системы
Основная характеристика
Качество
Эффективность
Совокупность свойств системы, существенных для ее использования по назначению
Степень соответствия результатов функционирования системы ее цели


Фактор структурного анализа
Качество
Эффективность
Структура системы
Алгоритм функционирования
Размерность
Качество
Эффективность
Вектор
Показатели результативности, ресурсоемкости и оперативности



Критерий качества – это показатель существенных свойств системы и правило его оценивания.

Пусть $Y^* = \langle y_1^*, y_2^*, ..., y_n^* \rangle$ - вектор идеальной системы.

Тогда область адекватности показателя качества определяется

$$\delta \subseteq \left| Y^{\text{доп}} / Y^* \right| / \left| Y^* \right|,$$

где δ - радиус области адекватности.

Типы критериев качества

1. Критерий пригодности

$$K^{\text{приг}}: (\forall_i)\left(y_i^j \in \delta \middle| \delta_i \to y_i^{\text{доп}}, i = \overline{1,n}\right)$$

Правило, согласно которому j - ая система считается пригодной, если значения всех i - х частных показателей y_i^j этой системы принадлежит области адекватности δ, а радиус этой области соответствует допустимым значениям всех частных показателей.

2. Критерий оптимальности

$$K^{\text{опт}}: (\exists_i)\left(y_i^j \in \delta \middle| \delta_i \to \delta^{\text{доп}}\right)$$

Правило, согласно которому j - я система считается оптимальной по i - у показателю качества: если существует хотя бы один частный показатель y_i^j, значение которого принадлежит области адекватности, а радиус этой области по этому показателю оптимален.

3. Критерий превосходства

$$K^{\text{прев}}: (\forall_i)\left(y_i^j \in \delta \middle| \delta_i \to \delta^{\text{опт}}, i = \overline{1,n}\right)$$

Правило, согласно которому j - ая система считается лучшей, если все значения частных показателей качества y_i^j принадлежат области адекватности, а радиус этой области оптимален по всем показателям.

Формально $K^{\text{прев}} \subset K^{\text{опт}} \subset K^{\text{приг}}$.

При оценивании качества систем с управлением целесообразно выделить нескольких уровней качества, проранжированных в порядке возрастания сложности рассматриваемых свойств.

1. ПОРОДИНА С.В. ИССЛЕДОВАНИЕ СИСТЕМ УПРАВЛЕНИЯ: УЧЕБНО-МЕТОДИЧЕСКОЕ ПОСОБИЕ. – НИЖНИЙ НОВГОРОД: НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМ. Н.И. ЛОБАЧЕВСКОГО, 2018. – 84 С.

2. ЧЕРНЫШЕВ Л.А., КОВАЛЕВ Р.Н. ИССЛЕДОВАНИЕ СИСТЕМ УПРАВЛЕНИЯ: УЧЕБ. ПОСОБИЕ / Л.А.ЧЕРНЫШЕВ, Р.Н. КОВАЛЕВ. ЕКАТЕРИНБУРГ: УГЛТУ, 2012. 323 С. ISBN 978-5-94984-409-0

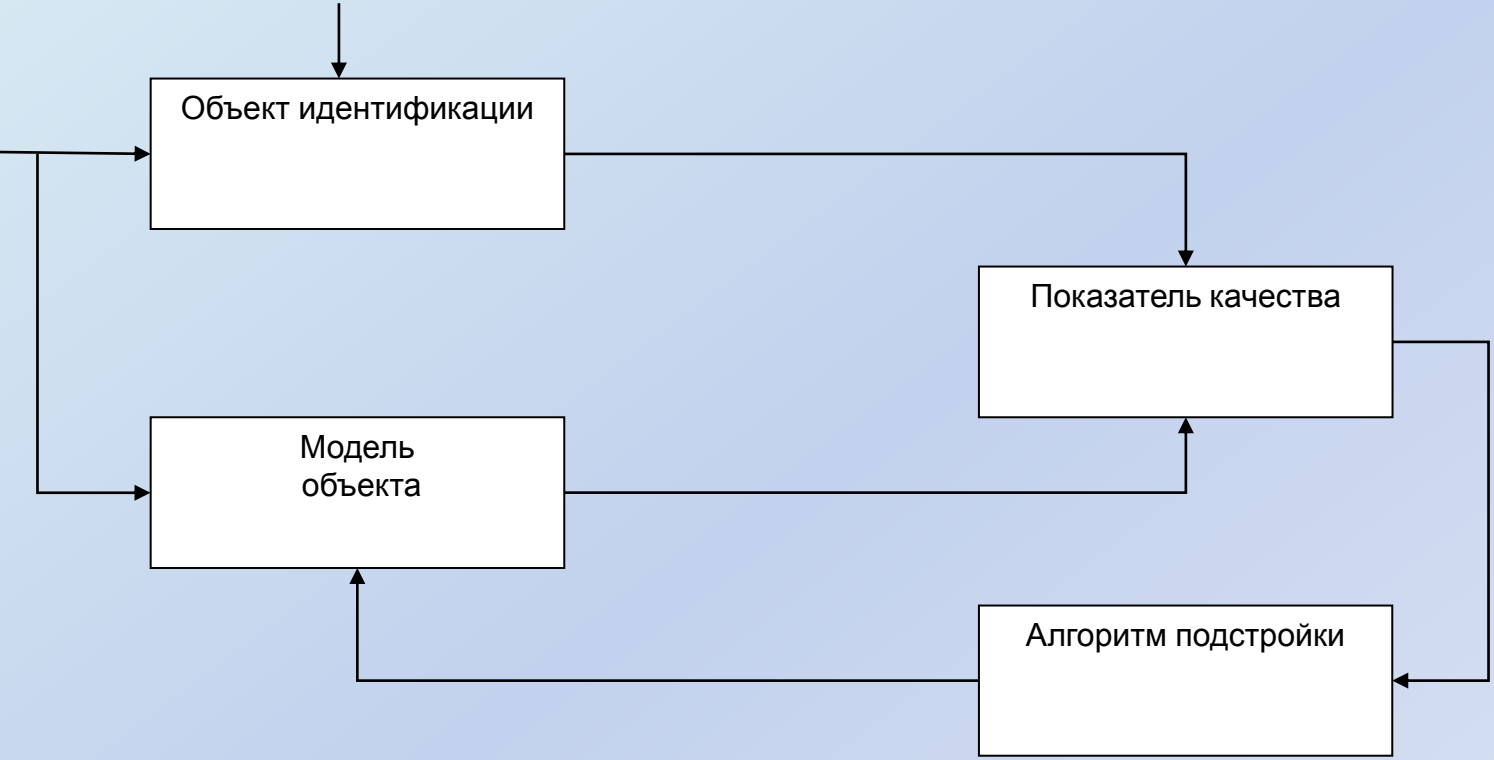
3. ИССЛЕДОВАНИЕ СИСТЕМ УПРАВЛЕНИЯ: УПРАВЛЕНИЕ ИННОВАЦИЯМИ И ИНВЕСТИЦИЯМИ: УЧЕБНОЕ ПОСОБИЕ / ПОД РЕД. В.М.ЧЕТВЕРИКОВА. – М.: МИЭМ, 2011. – 106 С.

Лекция 8. Оценивание систем

СОДЕРЖАНИЕ ЛЕКЦИИ:

- Шкала уровней качества систем.
- Принцип Парето и множество Парето.

Общая схема построения модели



2. Схема построения модели

Обычно схема построения модели основана на построенном сравнении модели с реальным объектом. Рассогласование между объектом и моделью выражено через показатель качества. Подстройка преобразуется алгоритмом подстройки в измененной модели.

Объект идентификации присутствует в виде набора зарегистрированных данных, а модель реализуется самостоятельно. В качестве специальных средств обработки реализуется алгоритм подстройки. Подстройка выполняется в условиях действия возмущающихся сил. Реальный объект может присутствовать и косвенно в виде наборов зарегистрированных данных, к которым выполняется подстройка.

Отсутствие рассогласования и его малое значение означает, что модель соответствует объекту.

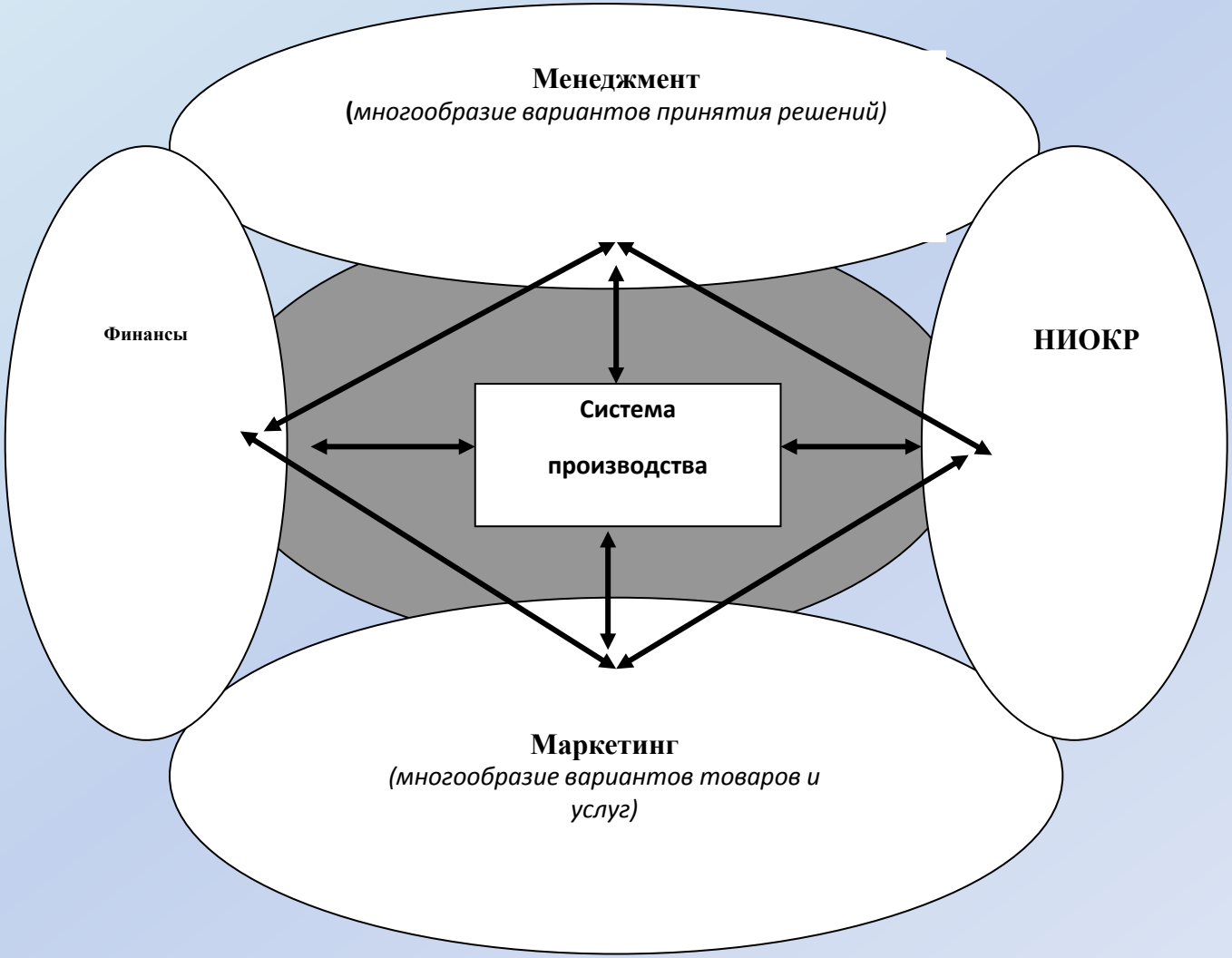
2. Понятие целевой функции

Целевое предназначение всего СА состоит в том, чтобы в результате осуществить выбор. **Выбор (или принятие решения) – суть поставленной задачи СА**, конечный итог всей работы. **Процедура принятия решения – это действие над множеством альтернатив**, в результате которого получается подмножество выбранных альтернатив. Желательно, чтобы это была 1 альтернатива.

Сужение множества альтернатив возможно, если есть способ сравнения альтернатив между собой и определения наиболее предпочтительных.

Сравнение альтернатив сводится к сравнению результатов расчетов соответствующих критериев. Если предположить, что выбор любой альтернативы приводит к однозначно определяемым последствиям и заданный критерий численно выражает оценку этих последствий, то наилучшей альтернативой является та, которая обладает наибольшим значением критерия.

Пример - Многообразие вариантов функционирования сложной системы



Информационная модель взаимодействия компонентов рыночной системы

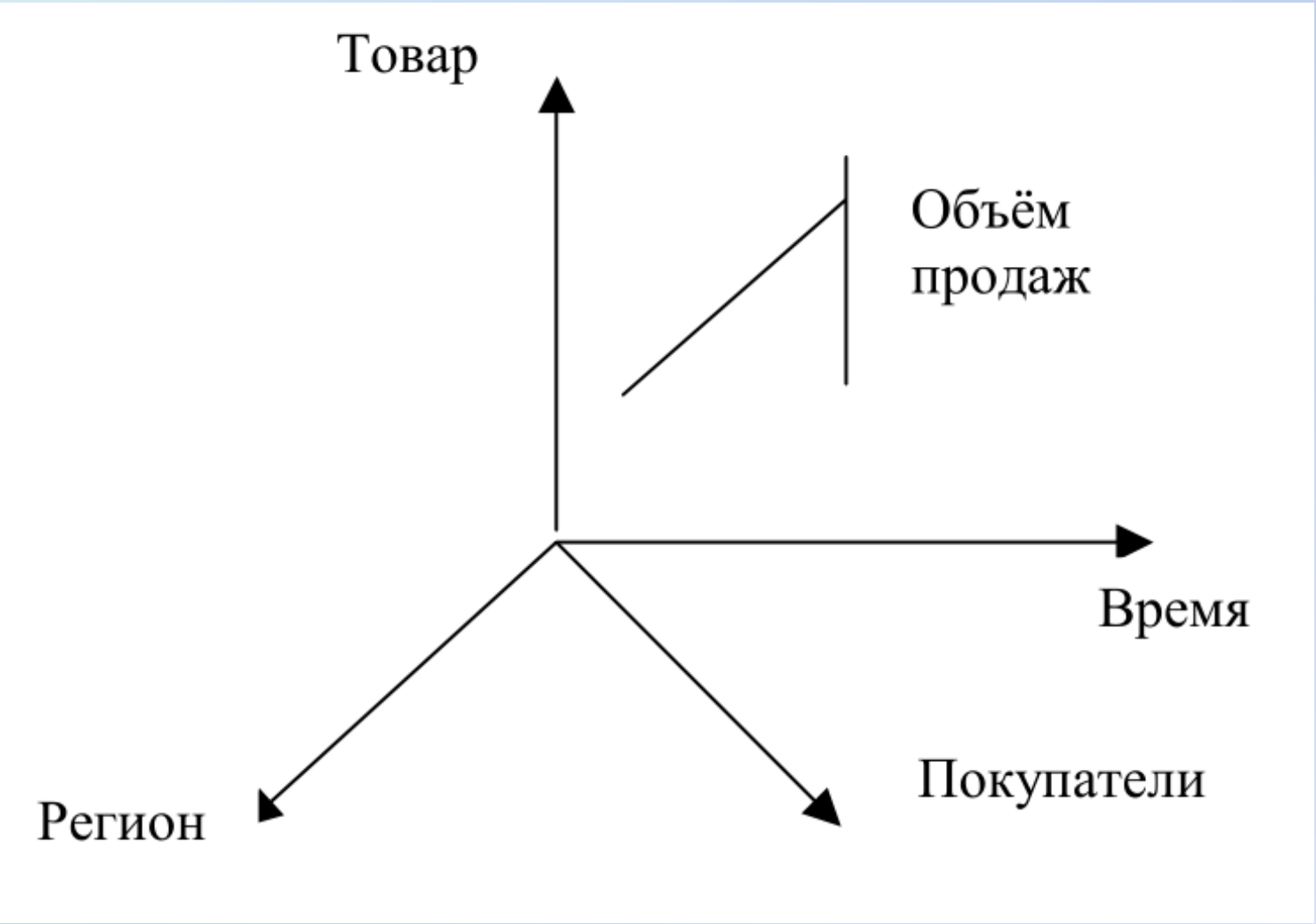
Понятие целевой функции

Выбор альтернативы на основе критериального подхода предполагает, что выполненными является несколько условий: известен критерий, задан способ сравнения вариантов и метод нахождения наилучшего из них. Однако этого оказывается недостаточно при решении задач выбора необходимо учитывать условия, при которых осуществляется выбор, и ограничения задачи, т.к. их изменение решения при одном и том же критерии.

Сравнение с оптимальными параметрами помогает решить вопрос о целесообразности дальнейших усилий по улучшению показателей качества исследуемого объекта. Однако у оптимизационного подхода есть свои ограничения, требующие внимательного и осторожного отношения.

До появления методов многоцелевой оптимизации, когда приходилось сталкиваться с проблемной многокритериальности, все критерии кроме 1-го фиксировались в виде ограничений, так что оптимизация производилась по 1-ому из них, который признавался доминирующим.

Пример многокритериальности



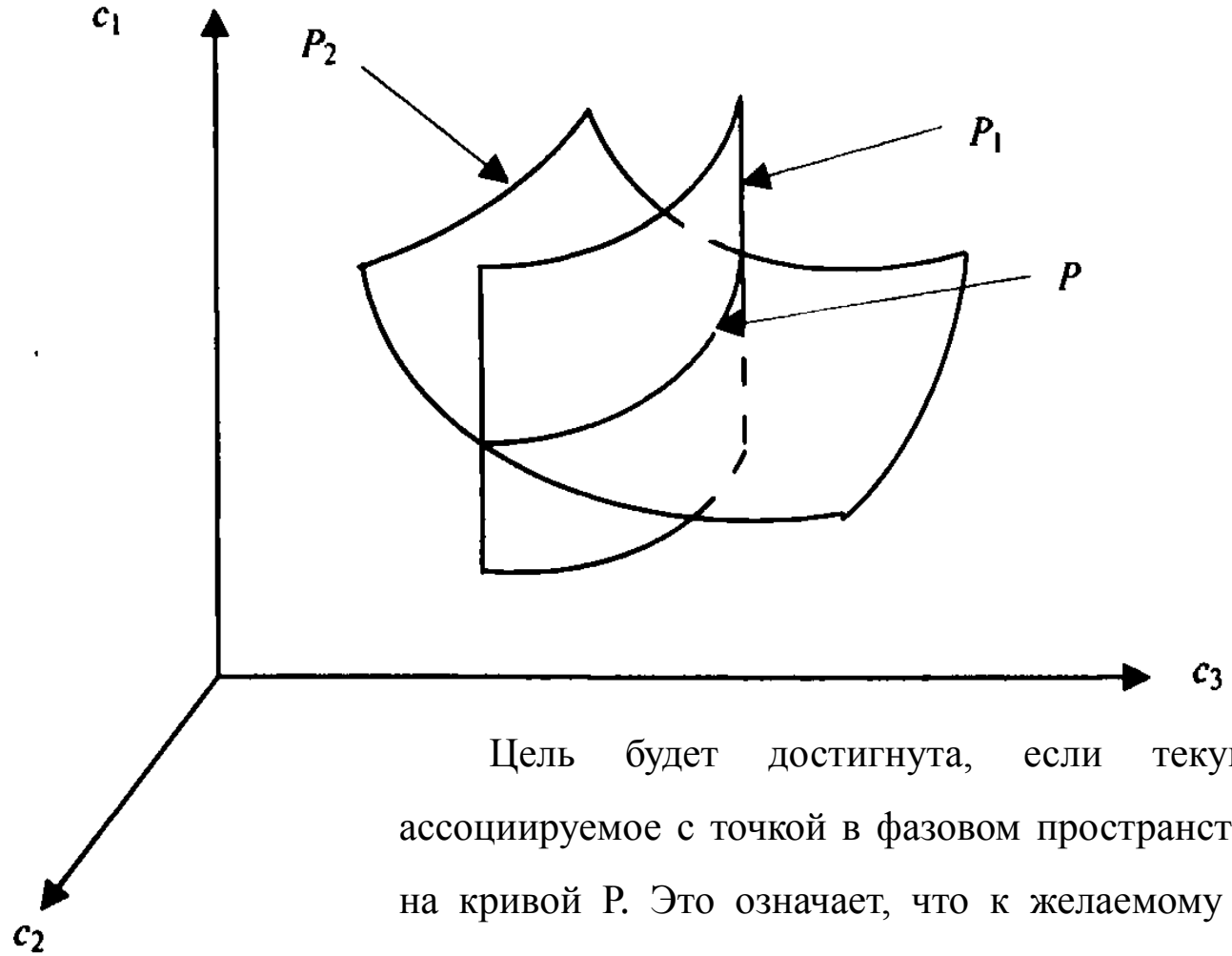
Выбор целевой функции

Процесс целеполагания связан с формулировкой требований, предъявляемых к окончанию управляемого периода, в виде конкретных значений параметров состояния. Каждый параметр имеет свою область определения, которая представляет собой некоторый диапазон значений на координате фазового пространства.

Цель системы, описываемая несколькими совместными требованиями в этом фазовом пространстве, есть не что иное, как пересечение поверхностей, каждая из которых имеет смысл функционального объединения параметров состояния, удовлетворяющих выполнению одного требования. На рисунке приведен пример определения цели системы, удовлетворяющей двум требованиям, представленным поверхностями P_1 и P_2.

Линия пересечения поверхностей P описывает комбинацию состояния системы, которые одновременно удовлетворяют оба требования.

Геометрия Целеполагания



Цель будет достигнута, если текущее состояние системы, ассоциируемое с точкой в фазовом пространстве, попадает в любое место на кривой Р. Это означает, что к желаемому результату могут привести многие траектории, и какая из них предпочтительнее — будет ясно в результате решения оптимизационной задачи.

Выбор целевой функции

На начальных этапах целеполагания глобальную цель системы предпочтительнее задать в виде сетевой или древовидной структуры. Сетевое представление последовательности подцелей требует хорошего знания исследуемой системы, специфики ее функционирования и технологии продуцирования конечных результатов. Иногда сетевая целевая структура может формироваться не сразу, а постепенно. По мере достижения одних подцелей формируются последующие, располагающиеся в направлении глобальной цели.

Такое поэтапное целеполагание возможно тогда, когда руководитель не уверен в возможностях вверенного ему коллектива и вынужден каждый раз корректировать ближайшие цели с поправкой на мнения и возможности его членов.

Вместе с тем наибольшее распространение получили древовидные структуры представления глобальной цели системы.

2. Понятие целевой функции

Построение дерева целей составляет одну из ключевых задач системного анализа.

При этом необходимо следить, чтобы на каждом уровне иерархии глобальной цели сохранялась ее целостность. Однако это не означает, что достижение некоторой цели вышестоящего уровня может быть полностью обеспечено достижением подчиненных ей подцелей. Цели системы, как и сама система, имеют свойство эмерджентности и представляют собой нечто большее, чем простая сумма подчиненных подцелей.

Любая промежуточная вершина дерева целей может рассматриваться с двух позиций: как цель для зависящих от нее вершин нижестоящего уровня и как средство достижения цели вершины выше стоящего уровня. В этой связи часто вершинам различных уровней присваивают различные названия, например, «цели», «задачи», «мероприятия», «проекты», или просто фиксируют «цели 1-го уровня», «цели 2-го уровня» и т.д.

1. Шкала уровней качества систем.


1
1
1
2
3
3
2
2
3
2
3
2
3

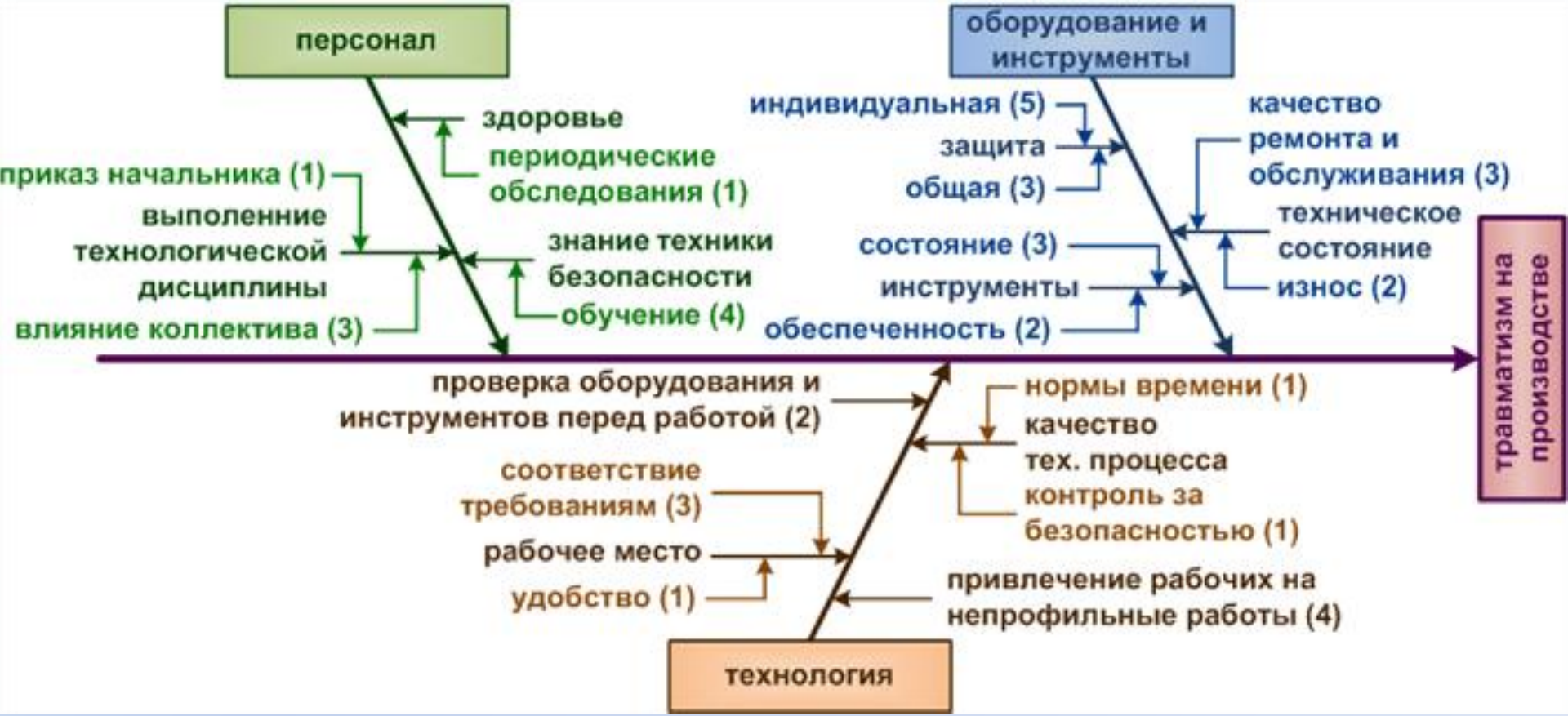
1. Шкала уровней качества систем.

Как видно из произведенной оценки основными факторами, влияющими на травматизм, являются: наличие и техническое состояние индивидуальных средств защиты, обучение работников технике безопасности, привлечение рабочих на непрофильные работы. Решением этих вопросов и нужно заниматься в первую очередь.

Естественно, что для разных сфер деятельности, предприятий и периодов времени результаты получатся другие, даже для достижения абсолютно одинаковой цели.

Необходимо оценивать степень влияния факторов на рассматриваемый показатель не для общих случаев, а конкретно для возникшей ситуации, для чего проанализировать как можно большую выборку данных для того места в котором производится улучшение.

1. Шкала уровней качества систем.



2. Понятие целевой функции

Требуемое качество системы задается правилами, которым должны удовлетворять показатели существенных свойств, а проверки их выполнения называется оцениванием качества системы.

Т.О., критерий качества – это показатель существенных свойств системы и правило его оценивания.

1. Шкала уровней качества систем.



1. Шкала уровней качества систем. Качество Управляемость

Следующим уровнем шкалы качества системы является управляемость – способность системы переходить за конечное время в требуемое состояние под влиянием управляющих воздействий.

Управляемость обеспечивается, прежде всего, прямой и обратной связью и объединяет такие свойства системы как оперативность, точность, инерционность и т.д. На этом уровне качества для сложной системы управляемость включает способность принятия решений по формированию управляющих воздействий.

1. Шкала уровней качества систем. Качество Способности

Следующим уровнем по шкале качеств является способность, определяющая возможность системы по достижению требуемого результата на основе имеющихся ресурсов в заданный период времени.

Данное качество характеризуется такими свойствами, как результативность, ресурсоѐмкость и оперативность.

Способность это по сути, потенциальная эффективность функционирования системы – способность получить требуемый результат при идеальном способе использования ресурсов и в отсутствии воздействия внешней среды.

1. Шкала уровней качества систем. Качество Способности

Наиболее сложным качеством системы является **самоорганизация**.

Самоорганизующиеся система способна изменять свою структуру, параметры , алгоритм функционирования и т.д.

Свойствами самоорганизации является **адаптируемость, самообучаемость, способность к распознаванию ситуации**.

Следует отметить, что уровень качества выбирает исследователь в зависимости от целей исследования, наличия информации и условия применения системы.

2. Показатели и критерии эффективности систем

Термин **«эффективность»** связывается и с системой, и с операцией и с решением.

Эти понятия можно считать эквивалентными.

Под эффективностью можно понимать комплексное свойство процесса функционирования системы, как степень приспособленности к достижению цели.

В общем случае **оценка функциональных свойств систем проводится в двух аспектах**:

- результат функционирования (операции);

- алгоритм, обеспечивающий получение результата.

Результат функционирования и алгоритм, обеспечивающий его получение, оцениваются по показателям результативности, ресурсоемкости и оперативности.

Результативность обуславливает еѐ получаемым целевым эффектом, ради которого функционирует система.

2. Показатели и критерии эффективности систем

Результативность обуславливает еѐ получаемым целевым эффектом, ради которого функционирует система.

Ресурсоемкость R характеризуется ресурсами всех видов (людскими, техническими, энергетическими, информационными, финансовыми и т.д.), используемыми для получения целевого эффекта.

Оперативность О определяется расходом времени, потребного для достижения цели функционирования системы.

Вектор эффективности системы определяется:

Yэф=<Yэ,YR,Yo>. **Выбор критерия эффективности – центральный, самый ответственный момент исследования систем**.

Процесс выбора критерия является субъективным, творческим процессом.

2. Понятие целевой функции

Математическое выражение критерия эффективности называют целевой функцией, поскольку её экстремум является отражением цели функционирования системы.

Отсюда следует, что формирование критерия эффективности решений требует:

- определить цель решения проблемы;
- найти множество управляемых и неуправляемых характеристик (параметров) системы;
- определить показатели исхода операции.

Хотя конкретные операции достаточно многообразны, существует ряд общих принципов, которыми необходимо руководствоваться при формировании критериев эффективности.

2. Понятие целевой функции

В зависимости от типа систем и внешних воздействий операции могут быть:

- детерминированными;

- вероятностными;

- неопределенными.

В связи с этим выделяют **3 группы критериев эффективности**:

1. В условиях определенности, если критерии отражают один строго определенный исход детерминированной операции;

2. В условиях риска, если критерии являются дискретными или непрерывными случайными величинами с известными законами распределения в вероятностной операции;

3. В условиях неопределенности, если критерии являются случайными величинами, законы распределения которых неизвестны.

2. Понятие целевой функции

Критерии пригодности для оценки детерминированных операций определяет правило, по которому операция считается эффективной, если все частные показатели исхода операции принадлежат области адекватности.

Критерий оптимальности для оценки детерминированной операции определяет правило, по которому операция считается эффективной, если все частные показатели принадлежат области адекватности, а радиус области адекватности оптимален.

Критерий пригодности для оценки эффективности вероятностной операции определяет правило, по которому операция считается эффективной, если вероятность достижения цели по показателям эффективности не меньше требуемой.

Критерий оптимальности для оценки вероятностной операции определяет правило, по которому операция считается эффективной, если вероятность достижения цели по показателям эффективности равна вероятности достижения цели с оптимальными значениями этих показателей.

2. Понятие целевой функции

Методика оценки эффективности систем в неопределенных операциях составляет один из разделов теории принятия решений.

Общие требования к показателям эффективности:

- соответствие цели;

- полнота;

- измеряемость;

- явность физического смысла;

- неизбыточность;

- чувствительность.

Оценка сложных систем в условиях неопределенности

Особенностями оценки сложных систем в условиях неопределенности являются:

1. Наличие в управляющей системе в качестве элемента ЛПР, осуществляющему управление на основе субъективных моделей, которые приводят к большому разнообразию поведения системы.

2. Алгоритм управления строит сама система управления, преследуя помимо целей старшей системы свои цели, не всегда совпадающие с внешними.

3. На этом этапе оценки ситуации в ряде случаев исходят не из фактической ситуации, а из той модели, которую использует ЛПР.

4. В процессе принятия решений большую роль играют логические рассуждения ЛПР, не поддающиеся формализации классическими методами математики.

5. При выборе управляющего воздействия ЛПР может оперировать нечеткими понятиями, отношениями и высказываниями.

6. В большинстве классов задач управление АСУ отсутствуют объективные критерии оценивания достижения целевого и текущего состояния ОУ, а также статистических данных для определения вероятностных законов для конкретного принятого решения.

Принятие решений в условиях неопределенности.

Стохастические факторы, приводящие к принятию решения в условиях риска, и неопределенные факторы приводят к разбросу возможных исходов результатов управления.

Но стохастические факторы полностью описываются известной стохастической информацией, эта информация и позволяет выбрать лучшее в среднем решение. Применительно к неопределенным факторам подобная информация отсутствует.

2. Принцип Парето и множество Парето.

Аксиома Парето:

Когда имеется один критерий оптимальности, стремление ЛПР обычно проявляется в том, чтобы получить *наибольшее*, либо *наименьшее* значение этого критерия.

> При решении экономических задач такой показатель, как затраты обычно стремятся минимизировать, а доход – максимизировать.

Изучение экстремальных задач можно ограничить лишь одним классом – либо задачами максимизации, либо задачами минимизации. Если задан не один, а сразу несколько критериев оптимальности, то для определенности для каждого из них необходимо указать «направление заинтересованности» ЛПР.

Множество Парето

Решение $x^* \in X$ называется *оптимальным по Парето* (*парето - оптимальным*), если не существует такого возможного решения $x \in X$, для которого имеет место неравенство $f(x) \geq f(x^*)$.

Все парето - оптимальные решения образуют *множество Парето*, обозначаемое $P_f(X)$.

$$Pf(X) = \{x \in X \mid \text{не существует такого } x \in X, \text{ что } f(x) \geq f(x^*)\}$$

Парето-оптимальное решение – это такое допустимое решение, которое не может быть улучшено (увеличено) ни по одному из имеющихся критериев без ухудшения (уменьшения) по какому-то хотя бы одному другому критерию.

Понятие оптимальности по Парето играет важную роль в математической экономике. Именно в этой области часто вместо парето-оптимальности используют наименования эффективное решение и множество эффективных решений.

Парето-оптимальность и эффективность в математической экономике нередко оказываются синонимами.

В зависимости от структуры множества X и вида векторного критерия f множество парето-оптимальных решений может:

- оказаться пустым (не содержать ни одного элемента);

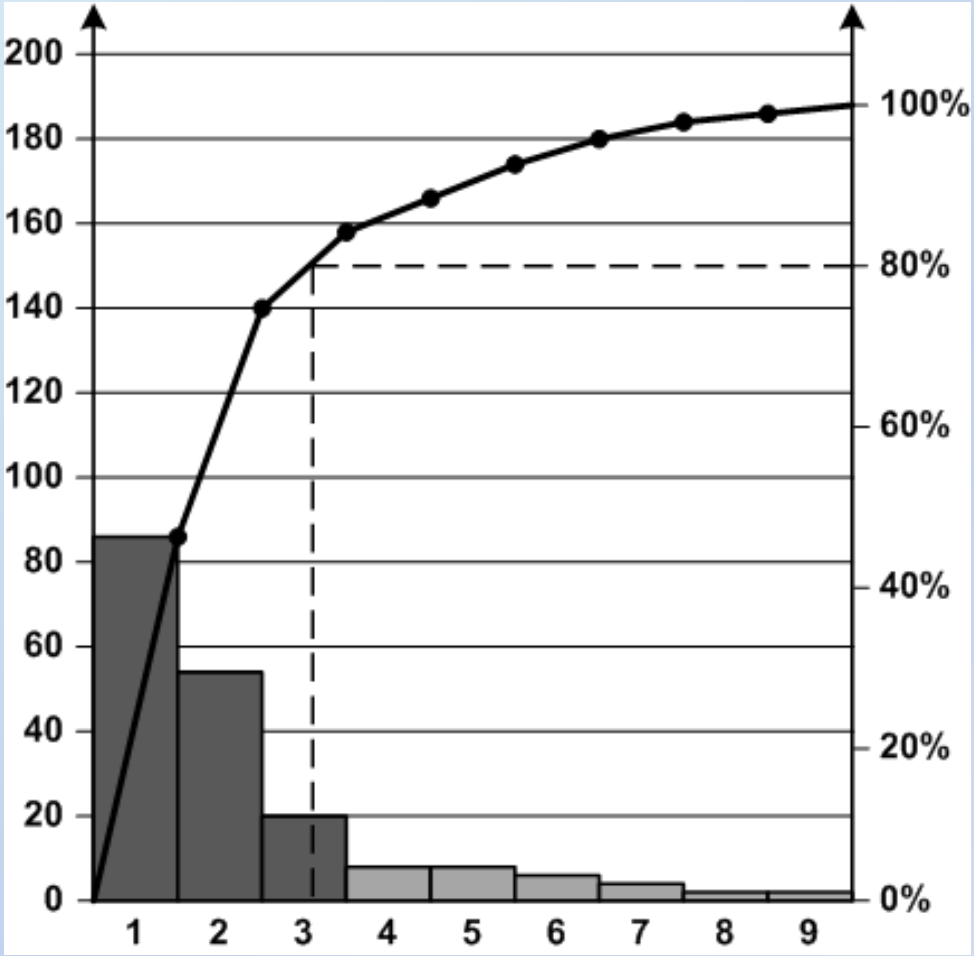
- быть одноэлементным множеством;

- состоять из некоторого конечного числа решений;

- содержать бесконечное число возможных решений;

- совпадать с множеством возможных решений X.

Диаграмма Парето -



Инструмент, позволяющий разделить факторы, влияющие на возникшую проблему, на важные и несущественные для распределения усилий по ее решению.

Сама диаграмма является разновидностью столбчатого графика с кумулятивной кривой, в которой факторы распределены в порядке уменьшения значимости (силы влияния на объект анализа).

В основе диаграммы Парето лежит принцип 80/20, согласно которому 20% причин приводят к 80% проблем, поэтому целью построения диаграммы является выявление этих причин для концентрации усилий по их устранению.

Основная литература

1. Флегонтов, А. В. Моделирование информационных систем. Unified Modeling Language : учебное пособие / А. В. Флегонтов, И. Ю. Матюшичев. — 2-е изд., стер. — Санкт-Петербург : Лань, 2022. — 112 с.
2. Гвоздева, Т. В. Проектирование информационных систем: технология автоматизированного проектирования. Лабораторный практикум : учебное пособие / Т. В. Гвоздева, Б. А. Баллод. — 2-е изд., стер. — Санкт-Петербург : Лань, 2020. — 156 с.
3. Леоненков, А. В. Объектно-ориентированный анализ и проектирование с использованием UML и IBM Rational Rose : учебное пособие / А. В. Леоненков. — 3-е изд. — Москва : Интернет-Университет Информационных Технологий (ИНТУИТ), Ай Пи Ар Медиа, 2020. — 317 с.
4. Вайсфельд Мэтт Объектно-ориентированный подход. 5-е межд. изд. — СПб.: Питер, 2020. — 256 с.: ил. — (Серия «Библиотека программиста»). ISBN 978-5-4461-1431-3
5. Волкова, В. Н. Теория систем и системный анализ : учебник для вузов / В. Н. Волкова, А. А. Денисов. — 3-е изд. — Москва : Издательство Юрайт, 2022. — 562 с.
6. Алексеева, М. Б. Теория систем и системный анализ : учебник и практикум для вузов / М. Б. Алексеева, П. П. Ветренко. — Москва : Издательство Юрайт, 2022. — 304 с.
7. Смоленцева Т.Е. и др.//Практикум АКМС/Online-edu.mirea/mod/vieww/php?d=435947 Москва 2022
8. Смоленцева Т.Е. и др.//Рабочая тетрадь АКМС/Online-edu.mirea/mod/vieww/php?d=435947 Москва 2022
9. ГОСТ Р 43.0.30—2022 Информационное обеспечение техники и операторской деятельности. Системность. Федеральное Агентство по техническому регулированию и метрологии Москва Российский институт стандартизации 2022 г. Оформление. ФГБУ «РОТ», 2022

Дополнительная литература

1. Исследование систем управления: учебное пособие / Л.Х. Мухсинова; Оренбургский гос. ун-т -- Оренбург, 2013. – 459 с.
2. Юрченко А.В. Исследование систем управления: Учебно-методическое пособие. — Ярославль: Типография ООО «С - Принт 72», 2008. - 104 с.
3. 1. Коцюба И.Ю., Чунаев А.В., Шиков А.Н. Методы оценки и измерения характеристик информационных систем. Учебное пособие. – СПб: Университет ИТМО, 2015. – 264 с
4. Дирша Е.В. Исследование систем управления: Курс лекций: Учебное пособие для студентов дневной и заочной форм обучения направлений 080500, 080200 «Менеджмент» / Рубцовский индустриальный институт. – Рубцовск, 2012. – 114 с
5. О.Г. Инюшкина / Проектирование информационных систем (на примере методов структурного системного анализа): учебное пособие / О.Г. Инюшкина, Екатеринбург: «Форт-Диалог Исеть», 2014. 240 с
6. Горлушкина Н.Н. Системный анализ и моделирование информационных процессов и систем. – СПб: Университет ИТМО, 2016. – 120 с.
7. Баринов, В.А. Теория систем и системный анализ в управлении организациями: справочник: учебное пособие для студентов вузов, обучающихся по специальности "Прикладная информатика (по областям) В.А. Баринов, Л.С. Болотова, В.Н. Волкова, А.А. Денисов В.А. Дуболазов/ Москва: Финансы и статистика, 2021
8. Шелухин, О.И. Моделирование информационных систем: учебное пособие / О.И. Шелухин. — 2-е изд., перераб. и доп. — Москва: Горячая линия-Телеком, 2018. — 516 с.

СПАСИБО ЗА ВНИМАНИЕ!